

Python: Noldan boshlab — Boshlovchilar uchun

Bu — Python'ni **mutlaqo noldan** o'rgatadigan qo'llanma. Hech qanday oldingi dasturlash tajribasi talab qilinmaydi. Bu yerda:

- Boshqa tillar bilan solishtirish **yo'q** — faqat sof Python. Boshqa til bilmasang ham hammasi tushunarli.
- Har bir tushuncha sodda tilda, kundalik hayotdan misollar bilan tushuntiriladi.
- Sekin-asta, qadam-baqadam — har modul oldingisiga tayanadi.
- Har modulda 20 ta masala va to'liq yechimlar bor.

Yo'l xaritasi

#	Modul	Mavzu	Holat
01	01-asoslar.md	print, o'zgaruvchilar, tiplar, input, f-string	✓ Tayyor
02	02-boshqaruv-funksiyalar.md	if/elif/else, for, while, funksiyalar	✓ Tayyor
03	03-malumot-tuzilmalari.md	ro'yxat, lug'at, to'plam, tuple	✓ Tayyor
04	04-stringlar.md	matn metodlari, slicing, qidiruv	✓ Tayyor
05	05-oop.md	klasslar va obyektlar	✓ Tayyor
06	06-modullar-muhit.md	modullar, <code>pip</code> , virtual muhit	✓ Tayyor
07	07-xatolar-context.md	xatolarni boshqarish (<code>try/except</code>)	✓ Tayyor
08	08-fayl-malumot.md	fayllar, JSON, CSV	✓ Tayyor

#	Modul	Mavzu	Holat
09	<code>09-iterator-generator-decorator.md</code>	generator va dekoratorlar	✓ Tayyor
10	<code>10-typing-functional.md</code>	tip ko'rsatmalari, <code>map/filter</code>	✓ Tayyor
11	<code>11-standart-kutubxona.md</code>	tayyor kutubxonalar	✓ Tayyor
12	<code>12-concurrency-async.md</code>	parallel ishlash, <code>asyncio</code>	✓ Tayyor
13	<code>13-testlash.md</code>	dasturni test qilish	✓ Tayyor
14	<code>14-web-capstone.md</code>	FastAPI bilan veb API	✓ Tayyor
15	<code>15-malumotlar-bazasi.md</code>	ma'lumotlar bazasi, SQL	✓ Tayyor

Boshlashdan oldin

Python o'rnatilganini tekshir (terminalda):

```
python --version
```

Agar `Python 3.x.x` chiqsa — tayyor. Chiqmasa, python.org dan yuklab o'rnat.

Kodni ishga tushirishning ikki yo'li: 1. **Interaktiv (REPL)** — terminalda `python` deb yoz, `>>>` belgisiga buyruq yozib darhol natija ko'r. 2. **Fayl** — `dastur.py` fayl yarat, kod yoz, `python dastur.py` bilan ishga tushir.

Birinchi moduldan boshla: [01 — Asoslar](#) →

01 — Asoslar: sintaksis, tiplar, "Pythonic" tafakkur

[← README](#) | [Keyingi: Boshqaruv va funksiyalar](#) →

1.1 Chekinish (indentation) = sintaksis

Python'da kod bloklari **chekinish** (qatorni bo'sh joy bilan ichkariga surish) orqali aniqlanadi. Qavslar ({ }) ishlatilmaydi — qaysi qatorlar blok ichida ekanini aynan chekinish belgilaydi.

```
if x > 0:
    print("musbat")      # 4 probel - bu blokni bildiradi
    print("hali ham ichida")
    print("blokdan tashqarida")
```

Qoidalar: - Standart — **4 probel** (tab emas; aralashtirib bo'lmaydi). - **:** blok ochilishini bildiradi (if, for, def, class va h.k.). - Noto'g'ri chekinish → `IndentationError`.

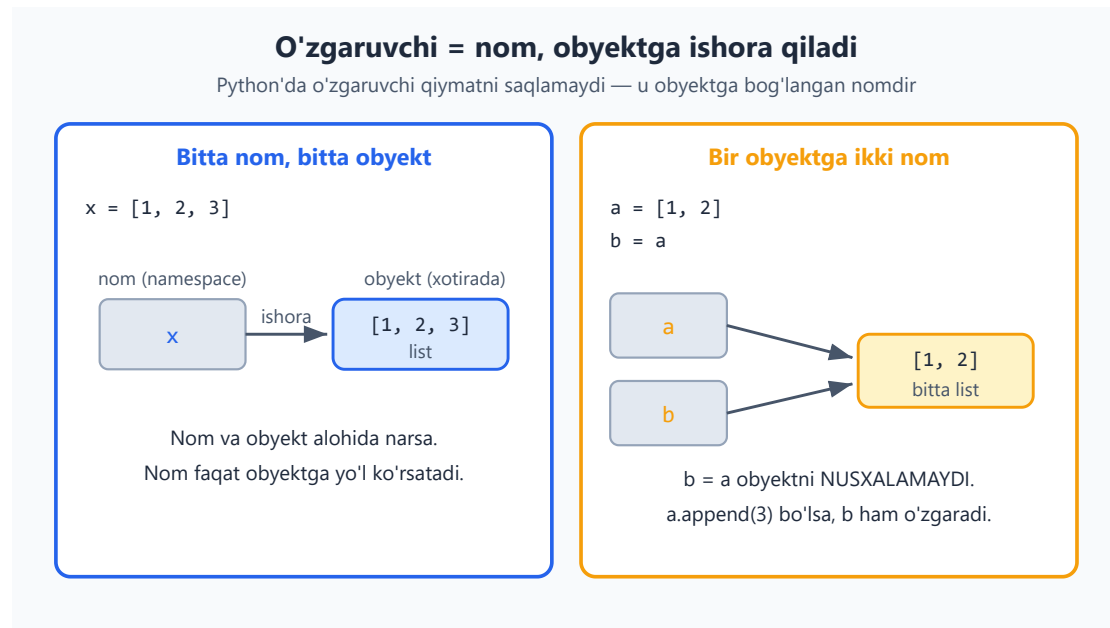
Why: Bu "kosmetik" emas — Python sintaksisini vizual tuzilmaga bog'laydi, shuning uchun "format urushlari" bo'lmaydi va kod doim bir xil ko'rinadi.

1.2 O'zgaruvchilar va dinamik tiplash

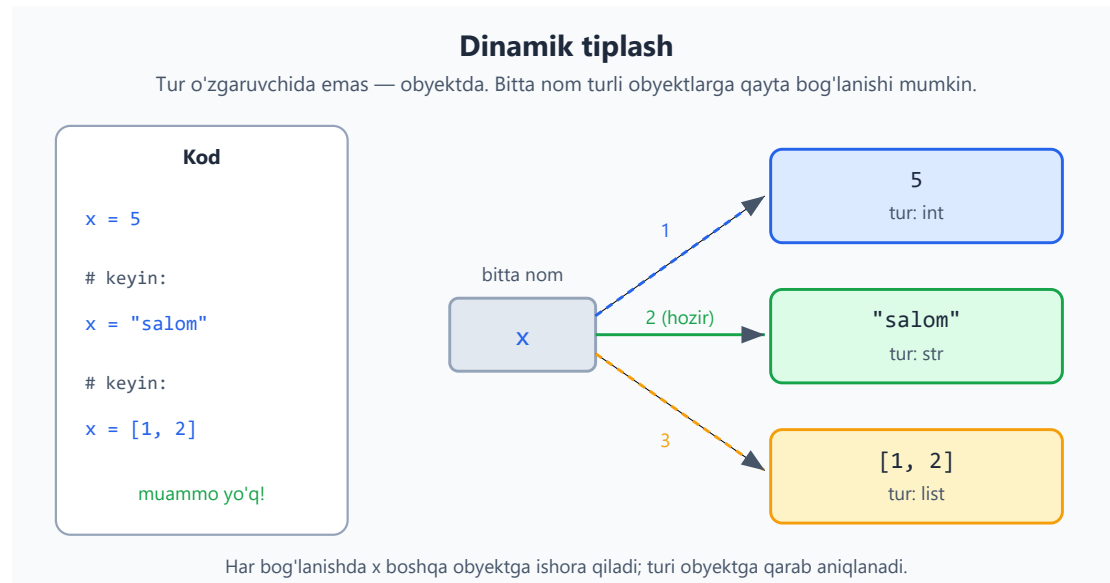
```
x = 5          # int
x = "salom"    # endi str - muammo yo'q
name = "Oqil"
PI = 3.14159    # konstanta: katta harf bilan yozish - kelishuv, til majburlamaydi
```

- O'zgaruvchini oldindan e'lon qilish (declare) shart emas — nom yozib, = bilan qiymat berasan, tamom. Nom oldiga maxsus belgi qo'yilmaydi, qator oxiriga ; ham qo'yilmaydi.
- Python **dinamik tiplangan** (o'zgaruvchining turi qiymatga qarab avtomatik aniqlanadi), lekin ayni paytda **kuchli tiplangan** (strongly typed): "3" + 5 xato beradi, turlar avtomatik aralashtirilmaydi.

Python'da o'zgaruvchi qiymatni o'z ichida saqlamaydi — u obyektga **ishora qiluvchi nom**dir (reference model). Shuning uchun `b = a` obyektни nusxalamaydi, balki bitta obyektga ikkinchi nom beradi:



Tur nomda emas, **obyekt**da turgani uchun bir nomni ketma-ket har xil turdagi obyektlarga bog'lash mumkin — dinamik tiplash aynan shu:



```
"3" + 5      # TypeError! Python matn va sonni avtomatik qo'shmaydi
"3" + str(5)  # "35" ✓
int("3") + 5  # 8   ✓
```

Type hints (tip ko'rsatmalari)

Python'da tiplar majburiy emas, lekin **type hints** yozish — professional standart. Bu funksiya qanday turdagi ma'lumot kutishini va nima qaytarishini ochiq ko'rsatadi:

```
age: int = 25
name: str = "Oqil"

def greet(name: str) -> str:
    return f"Salom, {name}"
```

Type hints dastur ishlayotganda (runtime'da) **majburlanmaydi** — ular `mypy` / `Pylance` kabi statik tekshiruvchilar uchun. Lekin baribir doim yoz — kod o'qilishi osonlashadi va xatolar oldindan topiladi.

1.3 Raqamli tiplar

```
a = 10          # int — CHEKSIZ aniqlik (big integer muammosi yo'q!)
b = 3.14        # float (64-bit)
c = 2 + 3j      # complex (kamdan-kam kerak)

2 ** 100        # 1267650600228229401496703205376 — muammosiz,
                # overflow yo'q
```

Diqqat: Python'da butun son (`int`) **cheksiz aniqlikka** ega — son qanchalik katta bo'lmasin, "overflow" (to'lib ketish) muammosi yo'q. Bu katta hisob-kitoblarda juda qulay.

Bo'lish — eng ko'p chalkashtiriladigan joy

```
7 / 2          # 3.5 — '/' har doim float (kasr son) qaytaradi
7 // 2         # 3   — butun bo'lish (floor division)
7 % 2          # 1   — qoldiq
-7 // 2        # -4  — pastga yaxlitlaydi, 0 tomon emas!
divmod(7, 2)   # (3, 1) — butun va qoldiq birga
```

1.4 Boolean va "truthiness"

```
True, False    # bosh harf bilan!
None           # "hech narsa" / bo'sh qiymat

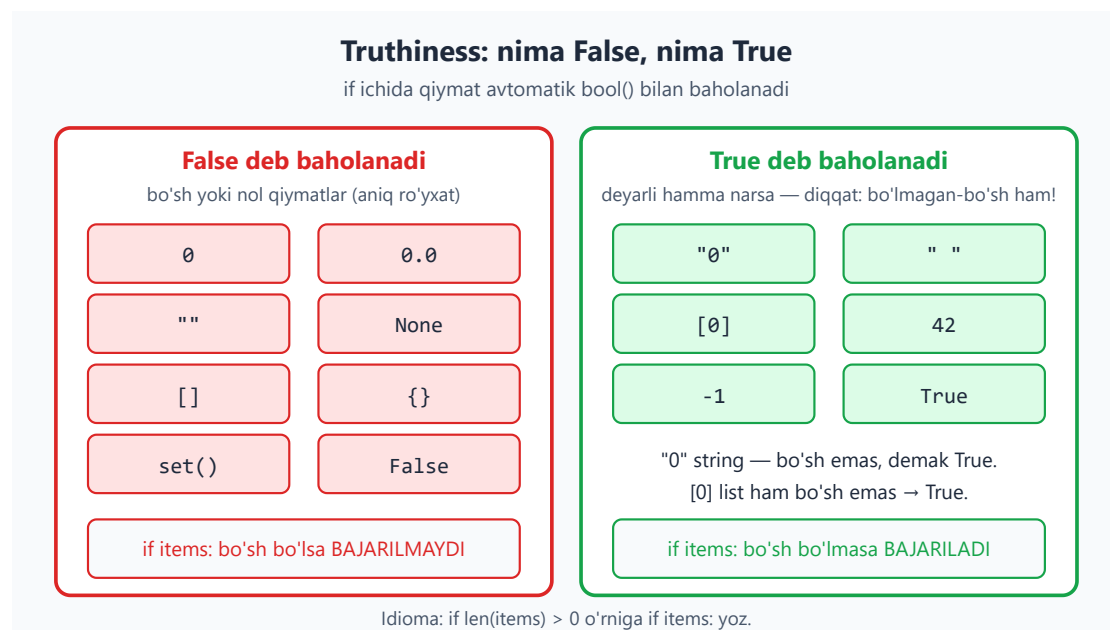
# Mantiqiy operatorlar — so'z bilan:
x and y        # &&
```

```
x or y          # ||
not x           # !
```

Truthiness — bo'sh yoki nol qiymatlar `if` ichida `False` deb baholanadi (aniq qoidalar bilan):

```
# False deb baholanadi:
bool(0), bool(0.0), bool(""), bool([]), bool({}), bool(None),
bool(set())
# True deb baholanadi: deyarli hamma narsa, jumladan "0" (string!),
[0], " "
```

Quyidagi diagramma qaysi qiymatlar `False`, qaysilari `True` deb baholanishini aniq ko'rsatadi (diqqat: `"0"` va `[0]` bo'sh emas, demak `True`):



Idioma: `if len(items) > 0`: o'rniga `if items`: yoz. Bo'sh list `False`. Bu Pythonic.

`and` / `or` qiymat qaytaradi (short-circuit), faqat bool emas:

```
name = user_input or "Mehmon" # bo'sh bo'lsa "Mehmon" (qisqa, qulay yozuv)
```

1.5 `None`, `==` va `is`

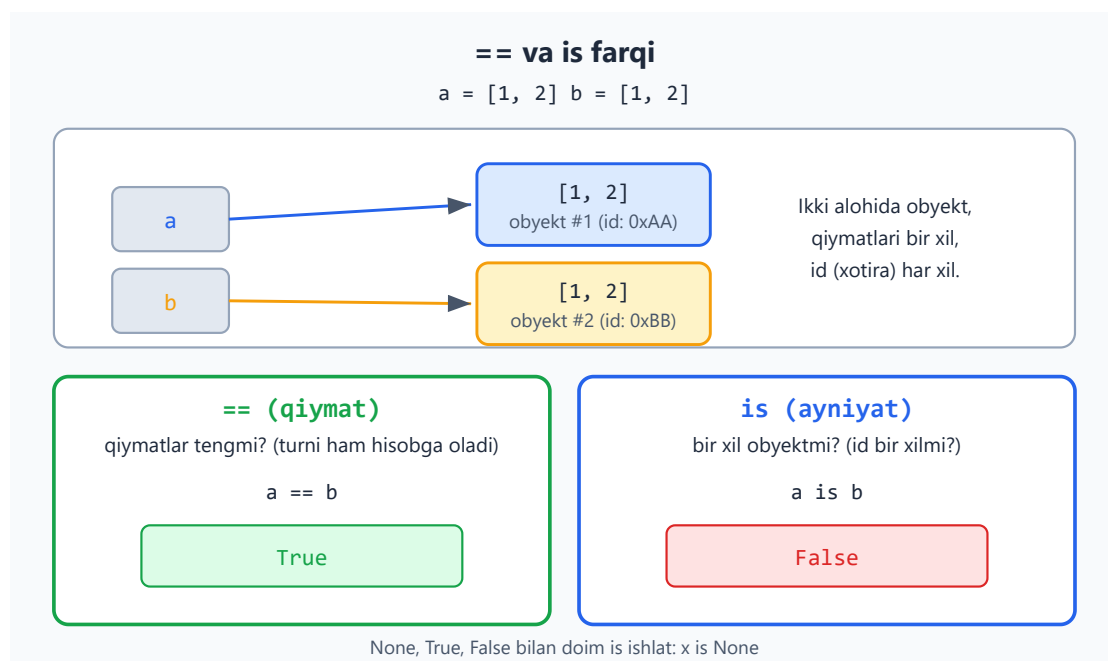
```
x = None
x == None # ishlaydi, lekin...
```

```
x is None      # ✅ to'g'ri uslub – identite (xotira) tekshiruvi
```

- `==` – **qiymat** tengligi (qiymatlarni solishtiradi va turni ham hisobga oladi: `1 == "1" → False`).
- `is` – **bir xil obyektmi** (xotirada bir joymi). `None`, `True`, `False` bilan doim `is` ishlat.

```
a = [1, 2]
b = [1, 2]
a == b      # True  – qiymatlar teng
a is b      # False – alohida obyektlar
```

Bu farqni vizual ko'rinishda: `a` va `b` qiymati bir xil, ammo xotirada ikki **alohida obyekt** – shuning uchun `== True`, `is` esa `False` qaytaradi:



1.6 Tip konvertatsiyasi

```
int("42")      # 42
int("42", 16)   # 66 – 16-lik sanoq sistemasidan
float("3.14")    # 3.14
str(42)         # "42"
bool(0)         # False
list("abc")     # ['a', 'b', 'c']

# Xavfsiz konvertatsiya:
int("abc")      # ValueError – try/except kerak (07-modulda)
```

1.7 Kiritish/chiqarish (I/O)

```
print("Salom", "dunyo")           # Salom dunyo (probel bilan ajraladi)
print("a", "b", sep="-")          # a-b
print("yangi qatorsiz", end="")   # \n qo'shmaydi
print(f"{name} - {age} yosh")     # f-string (eng muhim!)

ism = input("Isming: ")           # har doim STRING qaytaradi
yosh = int(input("Yoshing: "))    # raqam kerak bo'lsa konvertatsiya qil
```

`input()` **har doim string qaytaradi** — foydalanuvchi raqam yozsa ham, u matn bo'lib keladi. Raqam kerak bo'lsa `int()` / `float()` bilan o'ra.

1.8 f-string — formatlashning to'g'ri yo'li

```
name = "Oqil"
age = 25
price = 1234567.891

f"{name} {age} yoshda"           # Oqil 25 yoshda
f"{price:,.2f}"                  # 1,234,567.89 - minglik
ajratgich + 2 kasr
f"{age:03d}"                     # 025 - 3 xonali, nol
bilan
f"{0.1234:.1%}"                 # 12.3%
f"{name=}"                      # name='Oqil' - debug uchun
(3.8+)
f"{2 + 2}"                      # 4 - ichida ifoda yozish mumkin
```

f-string — matn ichiga to'g'ridan-to'g'ri o'zgaruvchi va ifoda joylashning eng toza yo'li: `f"..."` ichida `{...}` qavslar orasiga istalgan ifoda va formatlashni yozasan. **Eski usullar** (`.format()`, `%`) ni unut — f-string ishlat.

1.9 Bir nechta qiymat berish (multiple assignment)

```
a, b, c = 1, 2, 3                # bir vaqtda
a, b = b, a                      # almashtirish - vaqtinchalik
o'zgaruvchisiz!
x = y = z = 0                   # hammasi 0
```



```
first, *rest = [1, 2, 3, 4] # first=1, rest=[2,3,4] (unpacking)
```

`a, b = b, a` — ikki o'zgaruvchini almashtirishning eng nafis yo'li. Ko'p tillarda buning uchun vaqtinchalik uchinchi o'zgaruvchi kerak; Python'da bitta qator yetadi.

1.10 Walrus operatori `:=` (3.8+)

Ifoda ichida o'zgaruvchiga qiymat berish:

```
# Oddiy:
data = input()
while data != "quit":
    print(data)
    data = input()

# Walrus bilan:
while (data := input()) != "quit":
    print(data)
```

Kerak bo'lgandagina ishlat — o'qilishini buzmasin.

1.11 Izohlar va docstring

```
# Bir qatorli izoh

"""
Ko'p qatorli — texnik jihatdan string,
lekin izoh sifatida ishlatiladi.
"""

def hisobla(x: int) -> int:
    """Funksiya docstring'i — help() shuni ko'rsatadi."""
    return x * 2

help(hisobla) # docstring'ni chiqaradi
```

Masalalar (20 ta)

Yechimga qaramasdan ishla. REPL'da sina. Yechimlar fayl oxirida.

Oson (1–7):

1. Foydalanuvchidan ism va yoshni so'ra, "`<ism> <yosh> yoshda`" ko'rinishida chiqar (f-string bilan).
2. Ikki raqamni foydalanuvchidan ol, ularning yig'indisi, ayirmasi, ko'paytmasi, bo'linmasi (`/`), butun bo'linmasi (`//`) va qoldig'ini chiqar.
3. `a = 5`, `b = 10` — vaqtinchalik o'zgaruvchisiz ularni almashtir va natijani chiqar.
4. Sekunddagi vaqtni (masalan 3725) soat, daqiqa, sekundga aylantir: `1:02:05`.
5. Berilgan narx (`1234567.5`) ni minglik ajratgich va 2 kasr bilan chiqar: `1,234,567.50`.
6. `"3"` va `5` ni qo'shib `8` chiqar (tip muammosini hal qilib).
7. Doira radiusini ol, yuzasi va perimetrini hisobla (`PI = 3.14159`).

O'rta (8–14):

1. Foydalanuvchidan haroratni Selsiyda ol, Farengeytga aylantir: `F = C * 9/5 + 32`.
2. Ikki xonali sonni ol, uning raqamlari yig'indisini hisobla (masalan `47 → 11`).
(*Bo'lish/qoldiq bilan, stringsiz.*)
3. `bool()` ni quyidagilarga qo'llab, qaysi biri `True` / `False` ekanini bashorat qil, keyin tekshir: `0`, `" "`, `"0"`, `[]`, `[0]`, `None`, `" "`, `0.0`.
4. `x = 0`, `y = "default"` — `x or y` va `x and y` nima qaytaradi? Bashorat qilib tekshir va sababini tushuntir.
5. Foydalanuvchidan 3 ta baho ol (`input`), o'rtachasini 2 kasr bilan chiqar.
6. `a = [1,2]`, `b = [1,2]`, `c = a` bo'lsa: `a == b`, `a is b`, `a is c` — har birini bashorat qilib tekshir.
7. Bir qatorli kodda 5 ta o'zgaruvchiga (`a, b, c, d, e`) `[10, 20, 30, 40, 50]` listidan qiymat ber (unpacking).

Murakkab (15–20):

1. `first, *middle, last = [1, 2, 3, 4, 5]` — har biri nimaga teng? Yozib tekshir, keyin `*middle` ning tipi nima ekanini ayt.
2. Foydalanuvchidan summa (so'm) va valyuta kursini ol, dollarga aylantirib `$<qiymat>` ko'rinishida 2 kasr bilan chiqar.
3. Walrus operatori bilan: foydalanuvchidan son so'rab, "quit" deguncha har sonning kvadratini chiqaruvchi sikl yoz.
4. `2 ** 1000` ni hisobla — natija nechta xonali ekanini top. (*Maslahat: stringga aylantirib uzunligini ol.*)

5. Bir xonali son `n` (1–9) berilganda, uning quyidagi jadvalini chiqar: `n x 1 = n` dan `n x 9 = 9n` gacha (f-string, hozircha `for` ishlatmasdan 9 ta `print` bilan ham bo'ladi — yoki `for` bilsang, ishlat).
6. `divmod()` dan foydalanib, 1000 daqiqani kun:soat:daqiqaga aylantir (masalan `1500 → 1 kun 1 soat 0 daqiqa`).
-

 Yechimlar



```
# 1
ism = input("Isming: ")
yosh = input("Yoshing: ")
print(f"{ism} {yosh} yoshda")

# 2
a = int(input())
b = int(input())
print(a + b, a - b, a * b, a / b, a // b, a % b)

# 3
a, b = 5, 10
a, b = b, a
print(a, b)          # 10 5

# 4
t = 3725
soat = t // 3600
daqqa = (t % 3600) // 60
sekund = t % 60
print(f"{soat}:{daqqa:02d}:{sekund:02d}") # 1:02:05

# 5
narx = 1234567.5
print(f"{narx:,.2f}")          # 1,234,567.50

# 6
print(int("3") + 5)           # 8

# 7
PI = 3.14159
r = float(input("Radius: "))
print(f"Yuza: {PI * r ** 2:.2f}, Perimetr: {2 * PI * r:.2f}")

# 8
c = float(input("Selsiy: "))
print(f"{c * 9/5 + 32:.1f} F")

# 9
n = 47
print(n // 10 + n % 10)       # 11

# 10
for v in (0, "", "0", [], [0], None, " ", 0.0):
    print(repr(v), "->", bool(v))
# 0->False, ''->False, '0'->True, []->False, [0]->True, None->False, ' '->True, 0.0->False

# 11
x, y = 0, "default"
print(x or y)          # "default" (x falsy bo'lgani uchun y qaytadi)
print(x and y)         # 0 (x falsy bo'lsa and to'xtaydi, x ni qaytaradi)

# 12
a, b, c = int(input()), int(input()), int(input())
print(f"{(a + b + c) / 3:.2f}")
```

```

# 13
a, b = [1, 2], [1, 2]
c = a
print(a == b)      # True   qiymat teng
print(a is b)      # False  alohida obyekt
print(a is c)      # True   c - a ning o'zi

# 14
a, b, c, d, e = [10, 20, 30, 40, 50]
print(a, b, c, d, e)

# 15
first, *middle, last = [1, 2, 3, 4, 5]
print(first, middle, last)  # 1 [2, 3, 4] 5
print(type(middle))        # <class 'list'> - har doim list

# 16
summa = float(input("So'm: "))
kurs = float(input("Kurs: "))
print(f"${summa / kurs:.2f}")

# 17
while (son := input("Son ('quit' to'xtatadi): ")) != "quit":
    print(int(son) ** 2)

# 18
print(len(str(2 ** 1000)))  # 302

# 19
n = int(input("Son (1-9): "))
for i in range(1, 10):
    print(f"{n} x {i} = {n * i}")

# 20
daqiqqa = 1500
kun, qoldiq = divmod(daqiqqa, 24 * 60)
soat, daq = divmod(qoldiq, 60)
print(f"{kun} kun {soat} soat {daq} daqiqqa")  # 1 kun 1 soat 0 daqiqqa

```

← README | Keyingi: Boshqaruv va funksiyalar →

02 — Boshqaruv va funksiyalar

Hozirgacha kod yuqoridan pastga, har bir qator ketma-ket bajarilardi. Endi dasturlashning yuragini o'rganamiz: **shartga qarab qaror qabul qilish** (`if`) va **takrorlash** (`for`, `while`). Shundan keyin o'z **funksiyalaringni** yozasan.

Bu modulda: `if/elif/else` bilan tanlash, `for` va `while` sikllari, `break/continue`, va funksiyalar yaratish.

2.1 Bloklar va bo'sh joy (indentatsiya)

1-modulda aytib o'tgandik: Python'da kod **bloklari** chap tomondagi bo'sh joy (indentatsiya) bilan ajraladi. Endi buni amalda ko'ramiz.

Blok — bir guruh qator bo'lib, ular birga bajariladi. Blok ochilishidan oldin `:` (ikki nuqta) qo'yiladi, blok ichidagi qatorlar esa **4 ta bo'sh joy** bilan suriladi:

```
yosh = 20

if yosh >= 18:
    print("Voyaga yetgan")      # 4 ta bo'sh joy - bu qator if blokiga
    tegishli                    # 4 ta bo'sh joy - bu qator if blokiga
    print("Ovoz berishi mumkin") # bu ham if blokida
    print("Bu qator har doim bajariladi") # surilmagan - blokdan
    tashqarida
```

Qoidalar: - Blok ichi aniq **4 ta bo'sh joy** bilan suriladi (Tab tugmasi emas — ko'pchilik muharrirlar Tab'ni 4 bo'sh joyga aylantiradi). - Noto'g'ri bo'sh joy → `IndentationError` xatosi. - Bu Python qoidasi, xohish emas. Lekin foydasi bor: kod doim toza va bir xil ko'rinadi.

2.2 `if` / `elif` / `else` — qaror qabul qilish

`if` — "agar" degani. Shart `True` bo'lsa, blok bajariladi:

```
yosh = 20

if yosh >= 18:
    print("Kirishingiz mumkin")
```

`else` — "aks holda". Shart bajarilmasa, `else` bloki ishlaydi:

```
yosh = 15

if yosh >= 18:
    print("Kirishingiz mumkin")
else:
    print("Kirish mumkin emas")    # shu chiqadi
```

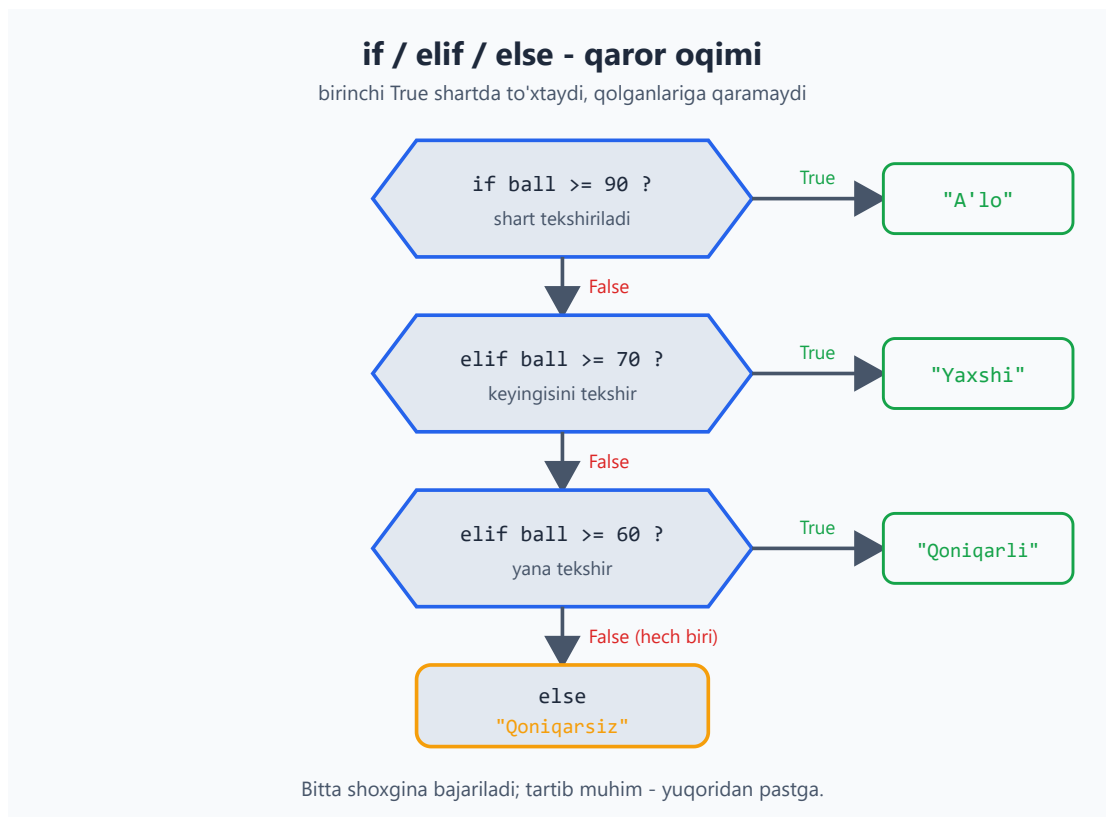
`elif` ("aks holda agar") — bir nechta shartni ketma-ket tekshirish uchun:

```
ball = 75

if ball >= 90:
    print("A'lo")
elif ball >= 70:
    print("Yaxshi")                # shu chiqadi (75 >= 70)
elif ball >= 60:
    print("Qoniqarli")
else:
    print("Qoniqarsiz")
```

Python yuqoridan pastga tekshiradi va **birinchi to'g'ri** shartda to'xtaydi. Qolganlariga qaramaydi. Shuning uchun shartlar tartibi muhim.

Quyidagi blok-sxema shartlarning yuqoridan pastga qanday tekshirilishini ko'rsatadi:



Shartlarni `and`, `or`, `not` bilan birlashtirish (1-modulda ko'rgan edik):

```
yosh = 25
chipta = True

if yosh >= 18 and chipta:
    print("Konsertga kirishingiz mumkin")
```

2.3 `for` sikli — ketma-ketlik bo'ylab takrorlash

`for` sikli biror ketma-ketlik (sonlar, ro'yxat, matn...) bo'ylab harakatlanib, har bir element uchun blokni bajaradi.

`range()` bilan sonlar bo'ylab:

```
for i in range(5):          # 0, 1, 2, 3, 4 (5 ning O'ZI kirmaydi!)
    print(i)
```

`range()` ning shakllari:

```
range(5)          # 0, 1, 2, 3, 4
range(1, 6)       # 1, 2, 3, 4, 5    (1 dan boshlab, 6 gacha lekin 6
kirmaydi)
range(0, 10, 2)   # 0, 2, 4, 6, 8    (2 qadam bilan)
```

Matn yoki ro'yxat bo'ylab (ro'yxatni 3-modulda batafsil ko'rasan):

```
for harf in "salom":
    print(harf)          # s, a, l, o, m — har harf alohida qatorda

mevalar = ["olma", "banan", "uzum"]
for meva in mevalar:
    print(meva)
```

`for i in range(n)` — "n marta takrorla" degani. `i` har aylanishda navbatdagi qiymatni oladi. O'zgaruvchi nomini ixtiyoriy tanlaysan (`i`, `son`, `harf` ...).

2.4 `while` sikli — shart bajarilguncha takrorlash

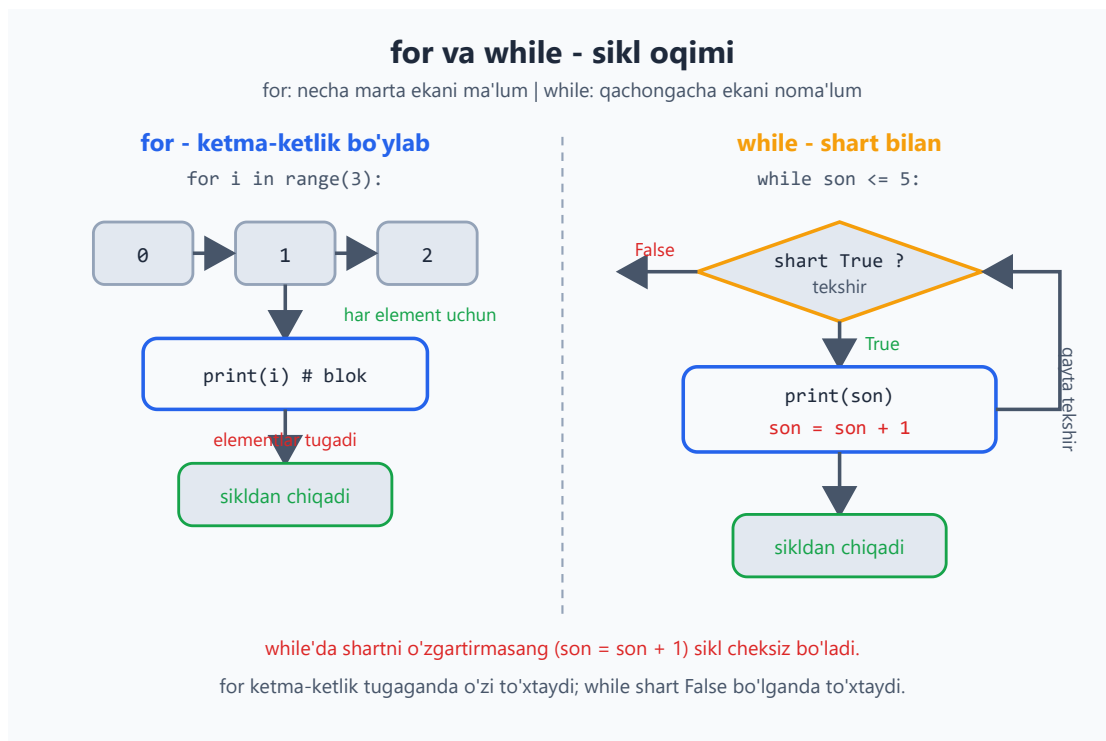
`while` — shart `True` bo'lib turguncha takrorlaydi:


```
son = 1
while son <= 5:
    print(son)
    son = son + 1      # MUHIM: shartni o'zgartirmasang, sikl cheksiz bo'ladi!
```

Diqqat — cheksiz sikl: agar shart hech qachon `False` bo'lmasa, dastur to'xtamay qoladi. Yuqoridagi misolda `son = son + 1` bo'lmasa, `son` doim 1 bo'lib qoladi va sikl tugamaydi. Har `while` da shartni o'zgartirayotganingni tekshir.

`for` qancha marta takrorlashni oldindan bilganda, `while` esa "qachongacha"ni bilmaganda qulay (masalan, foydalanuvchi to'g'ri javob berguncha).

Quyidagi sxema ikki siklning oqimini yonma-yon solishtiradi:



2.5 `break` va `continue`

Sikl ichida oqimni boshqarish:

```
# break - siklni butunlay to'xtatadi
for i in range(10):
    if i == 5:
        break          # 5 ga yetganda chiqadi
    print(i)           # 0, 1, 2, 3, 4
```

```
# continue – shu aylanishni o'tkazib, keyingisiga o'tadi
for i in range(5):
    if i == 2:
        continue          # 2 ni o'tkazib yuboradi
    print(i)               # 0, 1, 3, 4
```

2.6 Funksiyalar — kodni qayta ishlatish

Funksiya — bir vazifani bajaradigan, nom berilgan kod bo'lagi. Bir marta yozasan, keyin xohlagancha chaqirasan. Bu takrorni kamaytiradi va kodni tartibli qiladi.

```
def salomlash():          # def bilan e'lon qilinadi
    print("Salom!")
    print("Xush kelibsiz!")

salomlash()               # funksiyani chaqirish – kod shu yerda
                           bajariladi
salomlash()               # yana bir marta
```

Parametrlar — funksiyaga ma'lumot uzatish:

```
def salomlash(ism):       # ism – parametr
    print(f"Salom, {ism}!")

salomlash("Aziz")         # Salom, Aziz!
salomlash("Malika")       # Salom, Malika!
```

return — funksiyadan natija qaytarish:

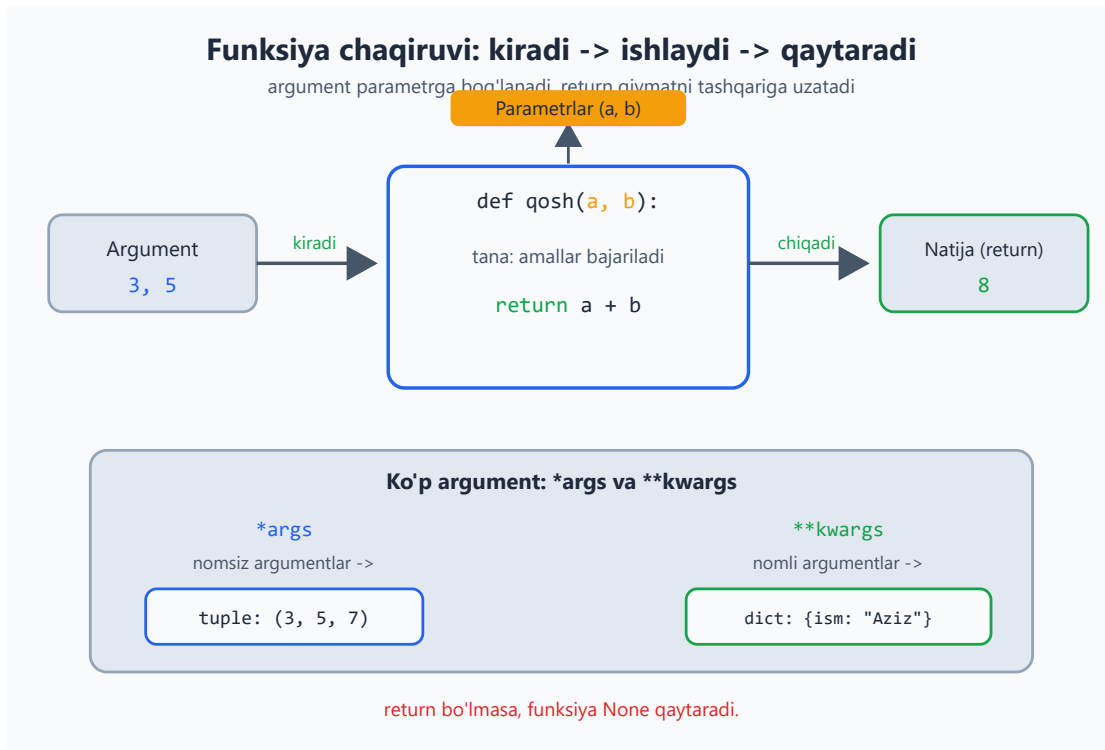
```
def qosh(a, b):
    return a + b           # natijani qaytaradi

natija = qosh(3, 5)        # natija = 8
print(natija)              # 8
print(qosh(10, 20))        # 30
```

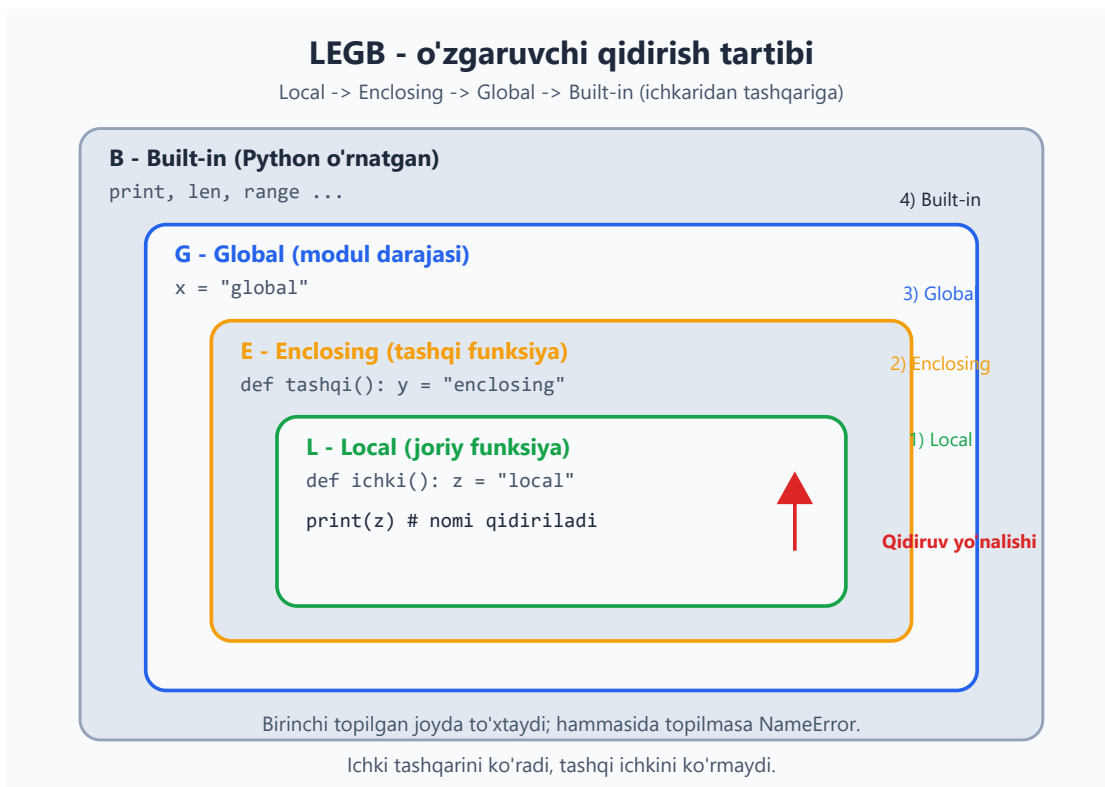
print va **return** farqi — yangi boshlovchilar buni tez chalkashtiradi: - **print** — qiymatni **ekranga ko'rsatadi**, lekin qaytarmaydi. - **return** — qiymatni **qaytaradi**, uni o'zgaruvchiga saqlab, keyin ishlatish mumkin.

Hisob-kitob qiluvchi funksiyalar odatda **return** ishlatadi, shunda natijani keyinroq foydalanasan.

Quyidagi sxema argument parametrqa qanday kirib, **return** natijani qanday qaytarishini, hamda ko'p argument uchun ***args** / ****kwargs** ni ko'rsatadi:



Funksiya ichida biror nomni ishlatganingda, Python uni qaysi tartibda qidiradi?
 Quyidagi sxema **LEGB** qoidasini ko'rsatadi — Python nomni Local -> Enclosing -> Global -> Built-in tartibida, ichkaridan tashqariga qarab qidiradi va birinchi topilgan joyda to'xtaydi:



2.7 Standart parametr qiymatlari

Parametrga oldindan qiymat berib qo'yish mumkin — chaqirilganda berilmasa, shu ishlatiladi:

```
def salomlash(ism, salom="Salom"):  
    print(f"{salom}, {ism}!")  
  
salomlash("Aziz")           # Salom, Aziz!      (standart  
ishlatildi)  
salomlash("Aziz", "Assalomu alaykum") # Assalomu alaykum, Aziz!
```

Standart qiymatli parametrlar funksiyani moslashuvchan qiladi: ko'p hollarda standart yetadi, kerak bo'lganda boshqasini berasan.

Masalalar (20 ta)

Bu masalalar shu modul va 1-modul mavzulariga asoslangan. Yechimga qaramasdan ishlashga harakat qil.

Oson (1–7):

1. Foydalanuvchidan son so'ra. Agar musbat bo'lsa "musbat", aks holda "musbat emas" deb chiqar.
2. Foydalanuvchidan yoshini so'ra. 18 dan katta yoki teng bo'lsa "voyaga yetgan", aks holda "voyaga yetmagan" deb yoz.
3. `for` sikli bilan 1 dan 10 gacha sonlarni chiqar.
4. `for` va `range` bilan 1 dan 10 gacha **juft** sonlarni chiqar (maslahat: `range(2, 11, 2)`).
5. `while` sikli bilan 5 dan 1 gacha teskari sanab chiqar.
6. `salom()` nomli funksiya yoz — chaqirilganda "Salom, dunyo!" chiqarsin. Uni 3 marta chaqir.
7. Ikkita sonni qabul qilib, ularning ko'paytmasini **qaytaradigan** (`return`) funksiya yoz.

O'rta (8–14):

1. Foydalanuvchidan ball so'ra (0–100). `if/elif/else` bilan baho qo'y: 90+ → "A'lo", 70+ → "Yaxshi", 60+ → "Qoniqarli", aks holda "Qoniqarsiz".
2. `for` sikli bilan 1 dan 100 gacha sonlarning yig'indisini hisobla va chiqar.

3. Foydalanuvchidan son so'ra, uning ko'paytuv jadvalini (1 dan 9 gacha) chiqar (`for` bilan, masalan `7 x 1 = 7`).
4. `while` sikli bilan foydalanuvchi "stop" deb yozguncha har safar undan biror so'z so'rab, uni qaytarib chiqar.
5. Sonni qabul qilib, juft yoki toq ekanini **qaytaradigan** funksiya yoz (maslahat: `%` ishlat, natija matn bo'lsin).
6. `for` va `break` bilan 1 dan boshlab sonlarni chiqar, lekin son 7 ga yetganda to'xta.
7. `for` va `continue` bilan 1 dan 10 gacha sonlarni chiqar, lekin 5 ni o'tkazib yubor.

Murakkab (15–20):

1. Foydalanuvchidan son so'rab, u **tub son** (faqat 1 va o'ziga bo'linadigan) ekanini tekshiruvchi dastur yoz (maslahat: `for` bilan 2 dan son-1 gacha bo'linishini tekshir).
2. Ikki sonni va amalni (`+`, `-`, `*`, `/`) qabul qiladigan oddiy kalkulyator funksiya yoz (`if/elif` bilan amalni tanla, natijani `return` qil).
3. `for` sikli bilan 1 dan 50 gacha sonlardan faqat 3 ga **ham**, 5 ga **ham** bo'linadiganlarini chiqar.
4. Foydalanuvchidan parol so'ra. To'g'ri parol (`"python123"`) kiritilmaguncha `while` bilan qayta-qayta so'ra. To'g'ri bo'lganda "Kirish muvaffaqiyatli" deb yoz.
5. Berilgan son uchun **faktorial** hisoblaydigan funksiya yoz (`n!` = $1 \times 2 \times \dots \times n$; masalan `5! = 120`). `for` yoki `while` bilan.
6. 1 dan 100 gacha "FizzBuzz" o'yini: 3 ga bo'linsa "Fizz", 5 ga bo'linsa "Buzz", ikkalasiga bo'linsa "FizzBuzz", aks holda sonning o'zini chiqar.

 Yechimlar



```
# 1
son = int(input("Son: "))
if son > 0:
    print("musbat")
else:
    print("musbat emas")

# 2
yosh = int(input("Yosh: "))
if yosh >= 18:
    print("voyaga yetgan")
else:
    print("voyaga yetmagan")

# 3
for i in range(1, 11):
    print(i)

# 4
for i in range(2, 11, 2):
    print(i)

# 5
son = 5
while son >= 1:
    print(son)
    son = son - 1

# 6
def salom():
    print("Salom, dunyo!")
salom()
salom()
salom()

# 7
def kopaytir(a, b):
    return a * b
print(kopaytir(4, 5))    # 20

# 8
ball = int(input("Ball: "))
if ball >= 90:
    print("A'lo")
elif ball >= 70:
    print("Yaxshi")
elif ball >= 60:
    print("Qoniqarli")
else:
    print("Qoniqarsiz")

# 9
yigindi = 0
for i in range(1, 101):
    yigindi = yigindi + i
print(yigindi)          # 5050
```

```

# 10
n = int(input("Son: "))
for i in range(1, 10):
    print(f"{n} x {i} = {n * i}")

# 11
while True:
    soz = input("So'z ('stop' to'xtatadi): ")
    if soz == "stop":
        break
    print(soz)

# 12
def juft_toq(n):
    if n % 2 == 0:
        return "juft"
    else:
        return "toq"
print(juft_toq(10))      # juft

# 13
for i in range(1, 20):
    if i == 7:
        break
    print(i)              # 1..6

# 14
for i in range(1, 11):
    if i == 5:
        continue
    print(i)              # 5 dan tashqari hammasi

# 15
son = int(input("Son: "))
tubmi = True
if son < 2:
    tubmi = False
for i in range(2, son):
    if son % i == 0:
        tubmi = False
        break
print("tub son" if tubmi else "tub son emas")

# 16
def kalkulyator(a, b, amal):
    if amal == "+":
        return a + b
    elif amal == "-":
        return a - b
    elif amal == "*":
        return a * b
    elif amal == "/":
        return a / b
print(kalkulyator(10, 3, "*"))  # 30

# 17
for i in range(1, 51):
    if i % 3 == 0 and i % 5 == 0:
        print(i)            # 15, 30, 45

# 18

```

```
while True:
    parol = input("Parol: ")
    if parol == "python123":
        print("Kirish muvaffaqiyatli")
        break

# 19
def faktorial(n):
    natija = 1
    for i in range(1, n + 1):
        natija = natija * i
    return natija
print(faktorial(5))      # 120

# 20
for i in range(1, 101):
    if i % 3 == 0 and i % 5 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```


03 — Ma'lumot tuzilmalari

Hozirgacha bitta o'zgaruvchida bitta qiymat saqladik. Lekin ko'pincha **bir nechta qiymatni birga** saqlash kerak bo'ladi — masalan, o'quvchilar ro'yxati yoki mahsulot narxlari. Buning uchun Python'da maxsus tuzilmalar bor: **ro'yxat**, **lug'at**, **to'plam** va **tuple**.

Bu modulda: ro'yxat (list) va uning metodlari, slicing, lug'at (dict), to'plam (set), tuple, va list comprehension.

3.1 Ro'yxat (list) — eng ko'p ishlatiladigan tuzilma

Ro'yxat — tartiblangan qiymatlar to'plami. Kvadrat qavs `[ ]` ichida, vergul bilan ajratiladi:

```
mevalar = ["olma", "banan", "uzum"]
sonlar = [10, 20, 30, 40]
aralash = [1, "salom", 3.14, True]    # turli tiplar ham bo'lishi
mumkin
bosh = []                             # bo'sh ro'yxat
```

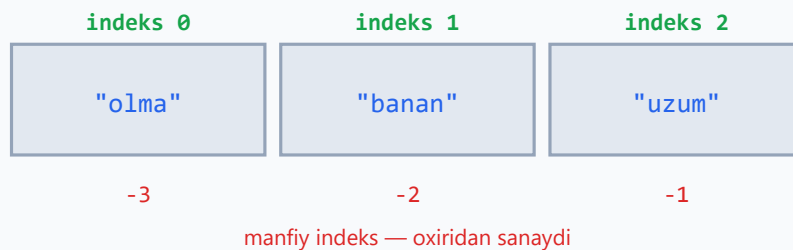
Elementga murojaat — har bir element o'z **indeksi** (raqami) bilan. Diqqat: indeks **0 dan** boshlanadi:

```
mevalar = ["olma", "banan", "uzum"]
print(mevalar[0])    # olma    (birinchi element)
print(mevalar[1])    # banan
print(mevalar[2])    # uzum
print(mevalar[-1])   # uzum    (oxirgisi — manfiy indeks oxiridan
sanaydi)
print(mevalar[-2])   # banan
```

Ro'yxat ichida har bir element o'z indeksiga bog'langan; quyidagi diagramma musbat va manfiy indekslarining bir xil katakka qanday ishora qilishini ko'rsatadi:

list — indeks → element

mevalar = ["olma", "banan", "uzum"]



`mevalar[0] → "olma"`
`mevalar[-1] → "uzum"`

`len(mevalar) → 3`
indeks 0 dan boshlanadi

Elementni o'zgartirish:

```
mevalar[0] = "anor"
print(mevalar)      # ['anor', 'banan', 'uzum']
```

Uzunligini bilish — `len()`:

```
print(len(mevalar)) # 3
```

Ro'yxat bo'ylab aylanish (`for` bilan, 2-modulda ko'rgan edik):

```
for meva in mevalar:
    print(meva)
```

3.2 Ro'yxat metodlari

Ro'yxatga amal qilish uchun tayyor metodlar bor. Metod — `royxat.metod()` ko'rinishida chaqiriladi:

```
sonlar = [3, 1, 2]

sonlar.append(4)      # oxiriga qo'shadi → [3, 1, 2, 4]
sonlar.insert(0, 9)   # 0-indeksga qo'yadi → [9, 3, 1, 2, 4]
sonlar.remove(1)      # qiymat 1 ni o'chiradi → [9, 3, 2, 4]
oxirgi = sonlar.pop() # oxirgini olib tashlaydi va qaytaradi →
oxirgi=4
sonlar.sort()         # tartiblaydi → [2, 3, 9]
sonlar.reverse()      # teskari qiladi → [9, 3, 2]

print(len(sonlar))    # nechta element
```

```
print(3 in sonlar)      # True - 3 ro'yxatda bormi?
print(sonlar.count(3))  # 3 nechta marta uchraydi
```

Eng ko'p ishlatiladigani — `append()` . Ko'pincha bo'sh ro'yxatdan boshlab, sikl ichida `append` bilan to'ldirasan:

```
kvadratlar = []
for i in range(1, 6):
    kvadratlar.append(i * i)
print(kvadratlar)      # [1, 4, 9, 16, 25]
```

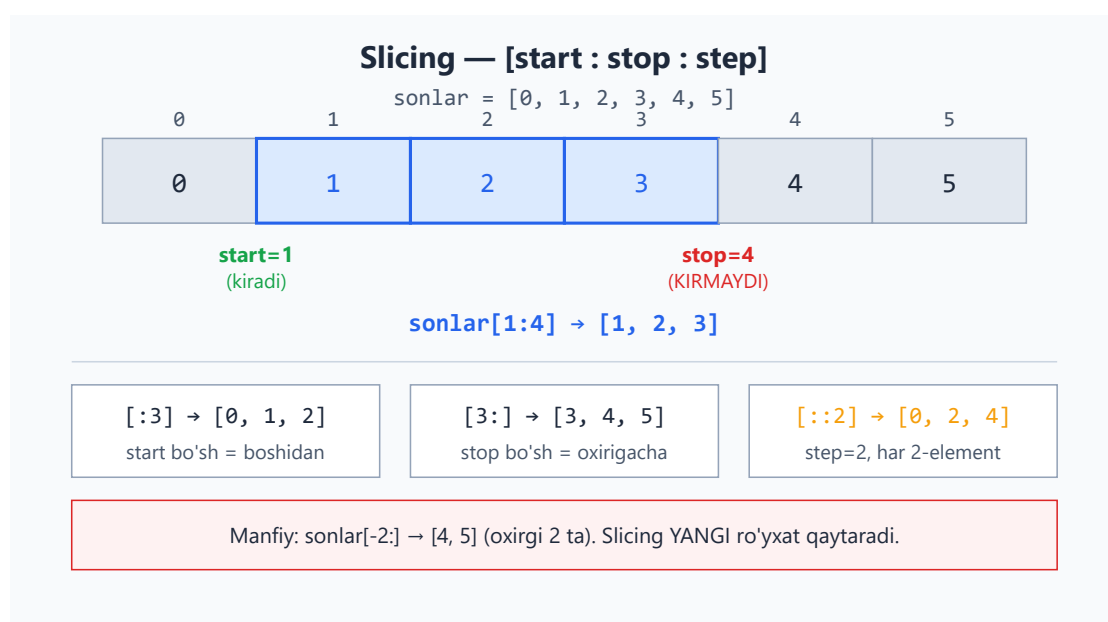
3.3 Slicing — ro'yxatning bir qismini olish

Slicing (kesib olish) — ro'yxatdan bir qismini ajratib olish. `[boshlanish:tugash]` ko'rinishida (tugash indeksi **kirmaydi**):

```
sonlar = [0, 1, 2, 3, 4, 5]

print(sonlar[1:4])      # [1, 2, 3]      (1-indeksdan 4 gacha, 4
kirmaydi)
print(sonlar[:3])       # [0, 1, 2]      (boshidan 3 gacha)
print(sonlar[3:])       # [3, 4, 5]      (3-indeksdan oxirigacha)
print(sonlar[-2:])      # [4, 5]        (oxirgi 2 ta)
print(sonlar[::-2])     # [0, 2, 4]      (har 2-element)
```

Quyidagi diagramma `[start:stop:step]` qanday ishlashini ko'rsatadi — `start` kiradi, `stop` esa kirmaydi:



Slicing matnlarda ham xuddi shunday ishlaydi (4-modulda ko'rasan). U **yangi** ro'yxat qaytaradi, aslini o'zgartirmaydi.

3.4 Lug'at (dict) — kalit–qiymat juftliklari

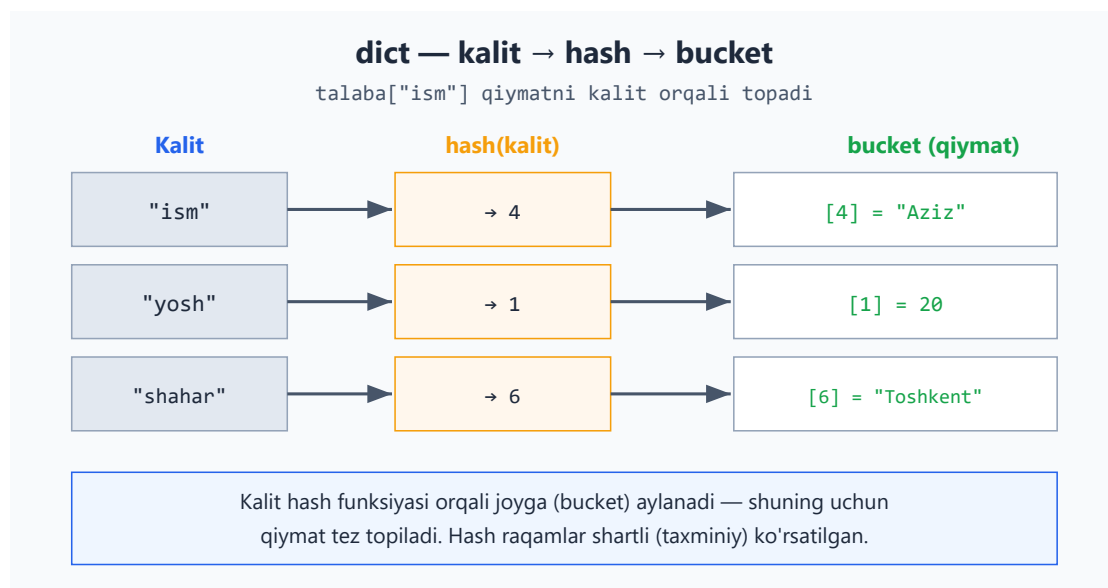
Lug'at — har bir qiymat **nom** (kalit) bilan saqlanadigan tuzilma. Ro'yxatda elementlar raqam (indeks) bilan, lug'atda esa **kalit** bilan topiladi. Jinalak qavs `{ }` ichida **kalit: qiymat** ko'rinishida:

```
talaba = {  
    "ism": "Aziz",  
    "yosh": 20,  
    "shahar": "Toshkent",  
}
```

Qiymatga murojaat — kalit orqali:

```
print(talaba["ism"])      # Aziz  
print(talaba["yosh"])     # 20
```

Lug'at qiymatni shu qadar tez topishining sababi — kalit ichkarida **hash** qiymatiga aylantiriladi va shu orqali saqlangan joy (bucket) aniqlanadi:



Qo'shish va o'zgartirish:

```
talaba["telefon"] = "998901234567" # yangi kalit qo'shadi  
talaba["yosh"] = 21                # mavjud kalitni o'zgartiradi
```

Xavfsiz murojaat — `get()` (kalit yo'q bo'lsa xato bermaydi):

```
print(talaba.get("email"))          # None (kalit yo'q, lekin xato yo'q)
print(talaba.get("email", "yo'q"))  # yo'q (standart qiymat berish mumkin)
# Solishtirish uchun: talaba["email"] — KeyError xatosi berardi
```

Lug'at bo'ylab aylanish:

```
for kalit in talaba:
    print(kalit, "=", talaba[kalit])

# yoki kalit va qiymatni birga:
for kalit, qiymat in talaba.items():
    print(f"{kalit}: {qiymat}")
```

Qachon ro'yxat, qachon lug'at? Tartibli, bir xil turdagi narsalar uchun (mevalar, ballar) — ro'yxat. Har bir narsaning "nomi"/xususiyati bo'lsa (ism, yosh, shahar) — lug'at.

3.5 To'plam (set) — takrorlanmas qiymatlar

To'plam — takrorlanmaydigan qiymatlar to'plami. Tartib yo'q, takror yo'q. Jingalak qavs `{ }` (lekin kalit-qiymat emas, faqat qiymatlar):

```
ranglar = {"qizil", "yashil", "qizil", "ko'k"}
print(ranglar)          # {'qizil', 'yashil', 'ko'k'} — takror avtomatik o'chdi
```

Eng foydali ishlatilishi — **takrorlarni olib tashlash**:

```
sonlar = [1, 2, 2, 3, 3, 3, 4]
noyob = set(sonlar)
print(noyob)          # {1, 2, 3, 4}
print(len(noyob))     # 4 — nechta xil son bor
```

To'plam amallari:

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)          # {2, 3} — umumiy (kesishma)
print(a | b)          # {1, 2, 3, 4} — birlashma
print(a - b)          # {1} — a da bor, b da yo'q
```

3.6 Tuple — o'zgarmas ro'yxat

Tuple — ro'yxatga o'xshaydi, lekin **o'zgartirib bo'lmaydi** (qiymat berib qo'yilgandan keyin qo'shish/o'chirish/o'zgartirish mumkin emas). Oddiy qavs () ichida:

```
koordinata = (41.31, 69.28)
print(koordinata[0])      # 41.31
# koordinata[0] = 50      # XATO — tuple o'zgarmas
```

Qachon tuple? O'zgarmasligi kerak bo'lgan ma'lumotlar uchun — masalan koordinata, RGB rang, yoki bir nechta qiymatni birga qaytarish. Tasodifan o'zgartirib qo'yishdan himoya qiladi.

Endi to'rttala tuzilmani bir joyda taqqoslab ko'ramiz — qaysi biri o'zgaruvchan, tartibli yoki takrorga ruxsat berishini quyidagi jadval umumlashtiradi:

Tort tuzilma — taqqoslash

Xususiyat	list []	dict { : }	set { }	tuple ()
O'zgaruvchanmi?	Ha	Ha	Ha	Yo'q
Tartibli?	Ha	Ha	Yo'q	Ha
Takror mumkinmi?	Ha	Kalit yo'q, qiymat ha	Yo'q	Ha
Misol	[1, 2, 3]	{"a": 1}	{1, 2, 3}	(1, 2)

Faqat tuple o'zgarmas; faqat set tartibsiz va takrorsiz.

Funksiyadan bir nechta qiymat qaytarishda tuple qulay:

```
def min_max(sonlar):
    return min(sonlar), max(sonlar) # tuple qaytaradi

eng_kichik, eng_katta = min_max([5, 2, 8, 1])
print(eng_kichik, eng_katta)      # 1 8
```

3.7 List comprehension — ro'yxatni qisqa yo'l bilan yaratish

Ko'pincha bir ro'yxatdan boshqasini yasaymiz. Buni `for` bilan ham qilsa bo'ladi, lekin **list comprehension** qisqaroq:

```
# Oddiy usul (for bilan):
kvadratlar = []
for i in range(1, 6):
    kvadratlar.append(i * i)

# List comprehension bilan – bir qatorda:
kvadratlar = [i * i for i in range(1, 6)]
print(kvadratlar)          # [1, 4, 9, 16, 25]
```

Shart qo'shish ham mumkin (`if` bilan):

```
juftlar = [i for i in range(1, 11) if i % 2 == 0]
print(juftlar)          # [2, 4, 6, 8, 10]
```

Boshida `for` bilan yozaver – comprehension ko'zga o'rganib qolgach, qisqaroq variantga o'tasan. Ikkalasi bir xil natija beradi.

Masalalar (20 ta)

Bu masalalar 1–3 modullar mavzulariga asoslangan.

Oson (1–7):

- `["olma", "banan", "uzum"]` ro'yxatini yarat. Birinchi va oxirgi elementini chiqar.
- Bo'sh ro'yxat yarat, unga `append` bilan 3 ta son qo'sh, keyin ro'yxatni chiqar.
- `[5, 2, 8, 1, 9]` ro'yxatini `sort` bilan tartibla va chiqar. Keyin `len` bilan uzunligini chiqar.
- `[10, 20, 30, 40, 50]` ro'yxatidan slicing bilan o'rtadagi 3 tasini (`[20, 30, 40]`) ol.
- `talaba` lug'atini yarat (`ism`, `yosh`, `shahar`). Har bir kalitning qiymatini chiqar.
- `[1, 1, 2, 3, 3, 3, 4]` ro'yxatidan `set` bilan takrorlarni olib tashla va nechta xil son borligini chiqar.
- Bir lug'atga yangi kalit qo'sh va mavjud kalitning qiymatini o'zgartir.

O'rta (8–14):

- Sonlar ro'yxatining yig'indisini va o'rtachasini hisobla (`sum()` va `len()` ishlat).
- Ro'yxatdagi eng katta va eng kichik sonni top (`max()`, `min()`).
- Foydalanuvchidan 5 ta son so'rab (siki bilan), ularni ro'yxatga `append` qil, keyin ro'yxatni tartiblab chiqar.

4. Lug'at bo'ylab `items()` bilan aylanib, har bir juftlikni `kalit: qiymat` ko'rinishida chiqar.
5. List comprehension bilan 1 dan 20 gacha sonlarning kvadratlari ro'yxatini yasa.
6. List comprehension va `if` bilan 1 dan 30 gacha 3 ga bo'linadigan sonlar ro'yxatini yasa.
7. Ikkita ro'yxatning umumiy elementlarini top (`set` va `&` ishlat): `[1,2,3,4]` va `[3,4,5,6]`.

Murakkab (15–20):

1. So'zlar ro'yxati berilgan. Har bir so'z necha marta uchraganini sanab, lug'atga yoz (maslahat: lug'at + `get`). Masalan `["a", "b", "a", "c", "a"]` → `{"a":3, "b":1, "c":1}`.
2. Talabalar ro'yxati (har biri lug'at: `ism`, `ball`). Ballari 60 dan yuqorilarning ismlarini chiqar.
3. Sonlar ro'yxatidan faqat juftlarini yangi ro'yxatga ajrat (ham `for` bilan, ham list comprehension bilan — ikki usulda).
4. Foydalanuvchidan so'zlar kiritishini so'ra ("stop" deguncha), ularni ro'yxatga yig'. Oxirida nechta so'z kiritilganini va ularni alifbo tartibida chiqar.
5. Bir lug'atda mahsulot nomi → narxi saqlangan. Barcha narxlar yig'indisini hisobla va eng qimmat mahsulot nomini top.
6. Telefon kitobchasi: foydalanuvchi "qo'shish", "qidirish" yoki "chiqish" tanlasin. Qo'shishda ism va raqamni lug'atga saqla, qidirishda ism bo'yicha raqamni chiqar (`while` sikli bilan davomli ishlasin).

 Yechimlar



```
# 1
mevalar = ["olma", "banan", "uzum"]
print(mevalar[0], mevalar[-1])    # olma uzum

# 2
sonlar = []
sonlar.append(10)
sonlar.append(20)
sonlar.append(30)
print(sonlar)                    # [10, 20, 30]

# 3
sonlar = [5, 2, 8, 1, 9]
sonlar.sort()
print(sonlar)                    # [1, 2, 5, 8, 9]
print(len(sonlar))              # 5

# 4
sonlar = [10, 20, 30, 40, 50]
print(sonlar[1:4])              # [20, 30, 40]

# 5
talaba = {"ism": "Aziz", "yosh": 20, "shahar": "Toshkent"}
print(talaba["ism"])
print(talaba["yosh"])
print(talaba["shahar"])

# 6
sonlar = [1, 1, 2, 3, 3, 3, 4]
print(len(set(sonlar)))         # 4

# 7
talaba = {"ism": "Aziz", "yosh": 20}
talaba["shahar"] = "Toshkent"   # qo'shish
talaba["yosh"] = 21             # o'zgartirish
print(talaba)

# 8
sonlar = [10, 20, 30, 40]
print(sum(sonlar))              # 100
print(sum(sonlar) / len(sonlar)) # 25.0

# 9
sonlar = [5, 2, 8, 1, 9]
print(max(sonlar), min(sonlar)) # 9 1

# 10
sonlar = []
for i in range(5):
    sonlar.append(int(input(f"{i+1}-son: ")))
sonlar.sort()
print(sonlar)

# 11
talaba = {"ism": "Aziz", "yosh": 20, "shahar": "Toshkent"}
for kalit, qiymat in talaba.items():
    print(f"{kalit}: {qiymat}")
```

```

# 12
kvadratlar = [i * i for i in range(1, 21)]
print(kvadratlar)

# 13
uchga = [i for i in range(1, 31) if i % 3 == 0]
print(uchga)                                # [3, 6, 9, ..., 30]

# 14
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
print(set(a) & set(b))                       # {3, 4}

# 15
sozlar = ["a", "b", "a", "c", "a"]
hisob = {}
for soz in sozlar:
    hisob[soz] = hisob.get(soz, 0) + 1
print(hisob)                                # {'a': 3, 'b': 1, 'c': 1}

# 16
talabalar = [
    {"ism": "Aziz", "ball": 85},
    {"ism": "Malika", "ball": 55},
    {"ism": "Bobur", "ball": 72},
]
for t in talabalar:
    if t["ball"] > 60:
        print(t["ism"])                     # Aziz, Bobur

# 17
sonlar = [1, 2, 3, 4, 5, 6]
# for bilan:
juftlar = []
for s in sonlar:
    if s % 2 == 0:
        juftlar.append(s)
print(juftlar)                              # [2, 4, 6]
# comprehension bilan:
juftlar2 = [s for s in sonlar if s % 2 == 0]
print(juftlar2)                             # [2, 4, 6]

# 18
sozlar = []
while True:
    soz = input("So'z ('stop' to'xtatadi): ")
    if soz == "stop":
        break
    sozlar.append(soz)
sozlar.sort()
print(f"{len(sozlar)} ta so'z:", sozlar)

# 19
mahsulotlar = {"olma": 12000, "non": 4000, "go'sht": 90000}
print("Jami:", sum(mahsulotlar.values()))
eng_qimmat = ""
eng_narx = 0
for nom, narx in mahsulotlar.items():
    if narx > eng_narx:
        eng_narx = narx

```

```
        eng_qimmat = nom
    print("Eng qimmat:", eng_qimmat)    # go'sht

# 20
kitobcha = {}
while True:
    amal = input("qo'shish / qidirish / chiqish: ")
    if amal == "chiqish":
        break
    elif amal == "qo'shish":
        ism = input("Ism: ")
        raqam = input("Raqam: ")
        kitobcha[ism] = raqam
    elif amal == "qidirish":
        ism = input("Ism: ")
        print(kitobcha.get(ism, "topilmadi"))
```

[← Boshqaruv va funksiyalar](#) | [Boshlovchilar README ↑](#) | [Keyingi: Stringlar →](#)

04 — Stringlar (matn bilan ishlash)

Matn (string) — dasturlashda eng ko'p ishlatiladigan ma'lumot turlaridan biri: ismlar, xabarlar, fayl matni, foydalanuvchi kiritgan ma'lumot. Bu modulda matnni qanday tahlil qilish, o'zgartirish va formatlashni o'rganasan.

Bu modulda: matn metodlari (katta/kichik harf, bo'sh joy tozalash, almashtirish), matnni bo'laklarga ajratish va birlashtirish, slicing, va matn ichidan qidirish.

4.1 Matn asoslari

Matn qo'shtirnoq `" "` yoki bittirnoq `' '` ichida yoziladi (farqi yo'q, lekin bittasini tanlab izchil ishlat):

```
ism = "Aziz"
jumla = 'Bugun havo issiq'
```

Matn ichida qo'shtirnoq kerak bo'lsa, tashqarisiga bittirnoq ishlat (yoki teskari):

```
gap = 'U "salom" dedi'
gap2 = "It's a book"
```

Uzunligi — `len()`:

```
print(len("salom"))    # 5
```

Matnni birlashtirish va takrorlash:

```
ism = "Aziz"
familiya = "Karimov"
print(ism + " " + familiya)    # Aziz Karimov    (+ birlashtiradi)
print("=" * 20)                # =====    (* takrorlaydi)
```

Diqqat: `+` faqat matnni matn bilan birlashtiradi. Matnga son qo'shmoqchi bo'lsang, avval sonni `str()` bilan matnga aylantir:

```
yosh = 25
print("Yosh: " + str(yosh))    # Yosh: 25
# yoki osonroq — f-string:
print(f"Yosh: {yosh}")        # Yosh: 25
```

4.2 Matn metodlari

Matnga amal qiladigan ko'plab tayyor metodlar bor. `matn.metod()` ko'rinishida chaqiriladi:

```
gap = "  Salom Dunyo  "

print(gap.upper())      # "  SALOM DUNYO  " - katta harf
print(gap.lower())      # "  salom dunyo  " - kichik harf
print(gap.strip())      # "Salom Dunyo"    - chetdagi bo'sh
                        joylarni olib tashlaydi
print(gap.replace("Dunyo", "Olam"))    # "  Salom Olam  " -
                        almashtiradi
print(gap.title())      # "  Salom Dunyo  " - har so'z bosh harf
                        bilan
```

Muhim: matn metodlari aslini **o'zgartirmaydi** — yangi matn qaytaradi. Natijani saqlash uchun o'zgaruvchiga ber:

```
gap = "  Salom  "
gap = gap.strip()      # natijani qayta saqladik
print(gap)             # "Salom"
```

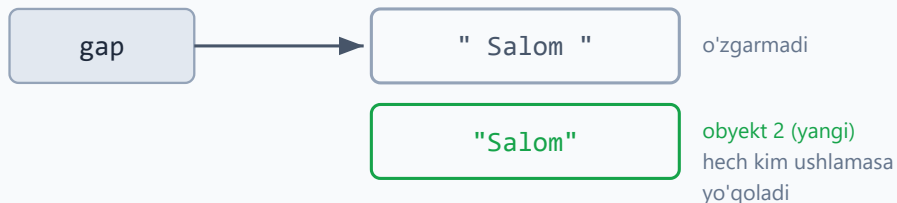
Quyidagi diagramma matn **o'zgarmasligini** (immutable) ko'rsatadi: metod aslini tegmaydi, balki yangi obyekt yaratadi va u faqat o'zgaruvchiga qayta saqlasang qoladi.

Matn o'zgarmas (immutable): metod yangi obyekt yaratadi

1) `gap = " Salom "`



2) `gap.strip()` — yangi obyekt qaytaradi (aslini tegmaydi)



3) `gap = gap.strip()` — nomni yangi obyektga ulaymiz



Tekshiruv metodlari (`True / False` qaytaradi):

```
"salom".startswith("sal")    # True    - "sal" bilan boshlanadimi?
"rasm.jpg".endswith(".jpg")  # True    - ".jpg" bilan tugaydimi?
"12345".isdigit()            # True    - faqat raqamlardan iboratmi?
"salom".isalpha()            # True    - faqat harflardanmi?
"Salom" in "Salom Dunyo"     # True    - ichida bormi?
```

4.3 Matnni bo'laklarga ajratish va birlashtirish

split() — matnni bo'laklarga ajratib, ro'yxat qaytaradi:

```
gap = "olma banan uzum"
sozlar = gap.split()          # bo'sh joy bo'yicha ajratadi
print(sozlar)                 # ['olma', 'banan', 'uzum']

sana = "2026-06-09"
qismlar = sana.split("-")    # "-" bo'yicha ajratadi
print(qismlar)               # ['2026', '06', '09']
```

join() — ro'yxatni matnga birlashtiradi (`split` ning teskarisi):

```
sozlar = ["olma", "banan", "uzum"]
print(", ".join(sozlar))     # "olma, banan, uzum" - vergul bilan
                              birlashtirdi
print(" ".join(sozlar))     # "olma banan uzum"
```

Eslab qol: `ajratuvchi.join(royxat)` — ajratuvchi belgi matnga ulanib turadi. Bu yangi boshlovchilarni chalkashtiradi: `join` ro'yxatning emas, **ajratuvchi matnning** metodi.

4.4 Slicing — matnning bir qismini olish

Matn ham xuddi ro'yxatdek indekslanadi (0 dan boshlab) va slicing bilan kesib olinadi:

```
gap = "Python"

print(gap[0])                # P        (birinchi harf)
print(gap[-1])               # n        (oxirgi harf)
print(gap[0:3])              # Pyt     (0 dan 3 gacha, 3 kirmaydi)
print(gap[2:])               # thon     (2-indeksdan oxirigacha)
print(gap[:3])               # Pyt     (boshidan 3 gacha)
print(gap[::-1])             # nohtyP   (teskari - matnni teskari aylantirish
                              hiylasi!)
```

Quyidagi chizg'ich indekslar qanday ishlashini ko'rsatadi: musbat indeks boshidan (0 dan), manfiy indeks oxiridan (-1 dan) sanaladi, qadam (step) esa yo'nalish va sakrashni belgilaydi.

gap = "Python" — slicing chizg'ichi

indeks → 0 1 2 3 4 5

P	y	t	h	o	n
---	---	---	---	---	---

-6 -5 -4 -3 -2 -1 ← Manfiy indeks (oxiridan)

Slicing misollari:

<code>gap[0:3]</code> → "Pyt" (3 kirmaydi)	<code>gap[-1]</code> → "n"
<code>gap[2:]</code> → "thon"	<code>gap[:3]</code> → "Pyt"

Qadam (step):

<code>gap[::-1]</code> → "nohtyP" (qadam -1: teskari aylantiradi)

Matn bo'ylab aylanish:

```
for harf in "abc":  
    print(harf)           # a, b, c
```

4.5 f-string bilan formatlash (kengaytirilgan)

1-modulda f-string'ni ko'rgan edik. Eng muhim formatlash variantlarini takrorlaymiz:

```
ism = "Aziz"  
narx = 1234567.5  
foiz = 0.85  
  
f"Salom, {ism}!"           # Salom, Aziz!  
f"{narx:,.2f}"             # 1,234,567.50   - minglik ajratgich + 2  
kasr                       # 85.0%         - foiz  
f"{ism:>10}"               # "          Aziz"   - o'ngga tekislash, 10  
belgi kenglikda            # "Aziz      "     - chapga tekislash  
f"{ism:<10}"               # "Aziz      "     - chapga tekislash  
f"{ism:^10}"               # "  Aziz  "       - markazga
```

Tekislash jadval ko'rinishidagi chiqarish uchun foydali:

```
mahsulotlar = [("Olma", 12000), ("Non", 4000), ("Go'sht", 90000)]
for nom, narx in mahsulotlar:
    print(f"{nom:<10} {narx:>10,} so'm")
# Olma          12,000 so'm
# Non           4,000 so'm
# Go'sht        90,000 so'm
```

4.6 Matn ichidan qidirish (boshlang'ich)

Oddiy qidiruv uchun yuqoridagi metodlar (`in`, `startswith`, `find`) yetadi:

```
gap = "Salom Dunyo"
print("Dunyo" in gap)      # True
print(gap.find("Dunyo"))   # 6      - qaysi indeksdan boshlanishini
                             qaytaradi
print(gap.find("yo'q"))     # -1     - topilmasa -1
print(gap.count("o"))       # 2      - necha marta uchraydi
```

Ilg'or qidiruv: murakkab namunalar (masalan "har qanday telefon raqamini top") uchun Python'da `re` (regular expressions) moduli bor. U kuchli, lekin boshlovchi uchun erta — keyingi modullarda ko'rib chiqamiz. Hozircha yuqoridagi oddiy metodlar ko'p ishni hal qiladi.

Masalalar (20 ta)

Bu masalalar 1–4 modullar mavzulariga asoslangan.

Oson (1–7):

1. Foydalanuvchidan ism va familiyani alohida so'rab, ularni bo'sh joy bilan birlashtirib chiqar.
2. Bir matnni `upper()` va `lower()` bilan katta va kichik harfda chiqar.
3. " salom " matnidagi chetdagi bo'sh joylarni `strip()` bilan olib tashlab chiqar.
4. Bir matnning uzunligini (`len`) chiqar.
5. "=" belgisini 30 marta chiqar (`*` ishlat).
6. "Python dasturlash" matnini `split()` bilan so'zlarga ajratib chiqar.
7. ["a", "b", "c"] ro'yxatini "-" bilan birlashtirib chiqar (`a-b-c`).

O'rta (8–14):

1. Foydalanuvchidan email so'ra. Agar ichida "@" bo'lsa "to'g'ri", aks holda "noto'g'ri" deb chiqar.
2. Bir so'zni slicing bilan teskari aylantir ("salom" → "molas").
3. Foydalanuvchidan jumla so'rab, undagi so'zlar sonini chiqar (`split()` + `len`).
4. Bir matndagi biror harf (masalan "a") necha marta uchrashini sana (`count`).
5. Fayl nomini (masalan "rasm.jpg") tekshir: .jpg bilan tugasa "rasm", .txt bilan tugasa "matn", aks holda "boshqa" deb chiqar.
6. Foydalanuvchidan to'liq ismni ("Aziz Karimov") so'rab, faqat ismini (birinchi so'z) chiqar (`split` + indeks).
7. Bir jumladagi har bir so'zni alohida qatorda chiqar (`split` + `for`).

Murakkab (15–20):

1. Foydalanuvchidan jumla so'rab, undagi har so'zning bosh harfini katta qil (`title()` ishlatmasdan, `split` + `for` + slicing bilan qilib ko'r).
2. Palindrom tekshir: foydalanuvchidan so'z so'rab, u teskarisiga ham bir xil o'qiladimi tekshir ("radar" → palindrom). Bo'sh joy va katta-kichik harfni hisobga olma.
3. Foydalanuvchidan jumla so'rab, undagi unli harflar (a, e, i, o, u) sonini sana.
4. Bir matndagi barcha bo'sh joylarni "_" ga almashtir (`replace`) va natijani chiqar.
5. Sana matnini ("2026-06-09") `split("-")` bilan ajratib, "9-iyun, 2026-yil" ko'rinishida chiqar (oylar nomini lug'atdan ol).
6. Oddiy "shifrlash": foydalanuvchidan so'z so'rab, har bir harfni keyingi harfga almashtir ("abc" → "bcd"). Maslahat: `ord()` harfni songa, `chr()` sonni harfga aylantiradi.

 Yechimlar



```
# 1
ism = input("Ism: ")
familiya = input("Familiya: ")
print(ism + " " + familiya)

# 2
gap = "Salom"
print(gap.upper())      # SALOM
print(gap.lower())      # salom

# 3
print("  salom  ".strip())  # "salom"

# 4
print(len("Python"))      # 6

# 5
print("=" * 30)

# 6
print("Python dasturlash".split())  # ['Python', 'dasturlash']

# 7
print("-".join(["a", "b", "c"]))      # a-b-c

# 8
email = input("Email: ")
if "@" in email:
    print("to'g'ri")
else:
    print("noto'g'ri")

# 9
soz = "salom"
print(soz[::-1])      # molas

# 10
jumla = input("Jumla: ")
print(len(jumla.split()))

# 11
gap = "banana"
print(gap.count("a"))      # 3

# 12
fayl = "rasm.jpg"
if fayl.endswith(".jpg"):
    print("rasm")
elif fayl.endswith(".txt"):
    print("matn")
else:
    print("boshqa")

# 13
toliq = input("To'liq ism: ")
print(toliq.split()[0])      # birinchi so'z
```

```

# 14
jumla = input("Jumla: ")
for soz in jumla.split():
    print(soz)

# 15
jumla = input("Jumla: ")
natija = []
for soz in jumla.split():
    natija.append(soz[0].upper() + soz[1:])
print(" ".join(natija))

# 16
soz = input("So'z: ").lower().replace(" ", "")
if soz == soz[::-1]:
    print("palindrom")
else:
    print("palindrom emas")

# 17
jumla = input("Jumla: ").lower()
hisob = 0
for harf in jumla:
    if harf in "aeiou":
        hisob += 1
print("Unlilar soni:", hisob)

# 18
gap = input("Matn: ")
print(gap.replace(" ", "_"))

# 19
sana = "2026-06-09"
yil, oy, kun = sana.split("-")
oylar = {
    "01": "yanvar", "02": "fevral", "03": "mart", "04": "aprel",
    "05": "may", "06": "iyun", "07": "iyul", "08": "avgust",
    "09": "sentabr", "10": "oktabr", "11": "noyabr", "12": "dekabr",
}
print(f"{int(kun)}-{oylar[oy]}, {yil}-yil")    # 9-iyun, 2026-yil

# 20
soz = input("So'z: ")
natija = ""
for harf in soz:
    natija += chr(ord(harf) + 1)
print(natija)    # "abc" -> "bcd"

```

05 — OOP: klasslar va obyektlar

Hozirgacha ma'lumotni alohida o'zgaruvchilarda saqladik. Lekin ko'pincha **ma'lumot va u bilan bog'liq amallarni birga** saqlash qulay. Masalan, "talaba"ning ismi, yoshi (ma'lumot) va "salomlash", "imtihon topshirish" (amallar) — bularni bitta joyda jamlash mumkin. Bu — **OOP** (obyektga yo'naltirilgan dasturlash), va u **klass** hamda **obyekt** tushunchalariga asoslanadi.

Bu modulda: klass va obyekt nima, `__init__` va `self`, atributlar va metodlar, `__str__`, hamda meros (inheritance).

5.1 Klass va obyekt nima?

Klass — bu "qolip" yoki "chizma". **Obyekt** — shu qolipdan yasalgan aniq narsa.

Misol: "Mashina" — bu klass (umumiy tushuncha, chizma). Sening qo'shningning oq Malibusi — bu obyekt (aniq mashina). Bitta chizmadan ko'p mashina yasash mumkin, har biri alohida obyekt.

```
class Mashina:          # klass - qolip
    pass                # hozircha bo'sh

mening_mashinam = Mashina() # obyekt - qolipdan yasaldi
qoshnining = Mashina()     # yana bir obyekt
```

Kelishuv: klass nomi katta harf bilan boshlanadi (`Mashina`, `Talaba`), o'zgaruvchilar kichik harf bilan.

5.2 `__init__` va `self` — obyektga ma'lumot berish

`__init__` — maxsus metod, obyekt yaratilganda **avtomatik** ishlaydi. Unda obyektning boshlang'ich ma'lumotlarini o'rnatasan. `self` — "shu obyektning o'zi" degani:

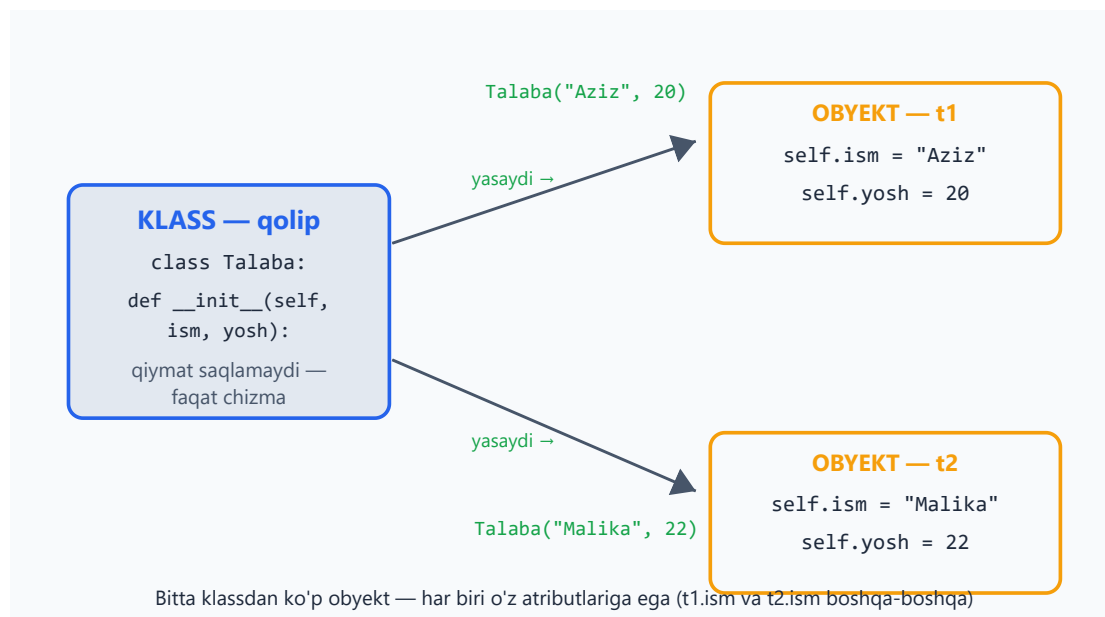
```
class Talaba:
    def __init__(self, ism, yosh):
        self.ism = ism          # bu obyektning ismi
        self.yosh = yosh        # bu obyektning yoshi

t1 = Talaba("Aziz", 20)         # __init__ avtomatik chaqiriladi
t2 = Talaba("Malika", 22)
```

```
print(t1.ism)          # Aziz
print(t2.yosh)         # 22
```

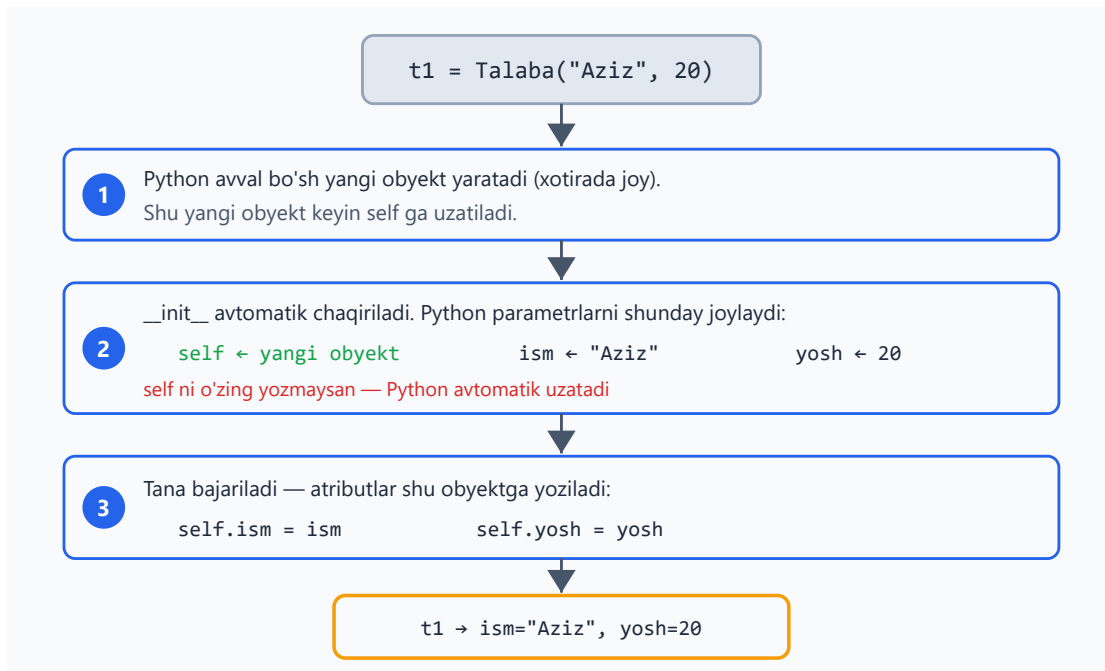
`self.ism = ism` ni shunday tushun: "shu obyektga `ism` degan xususiyat (atribut) qo'yib, unga berilgan qiymatni saqla". Har bir obyekt o'z atributlariga ega — `t1.ism` va `t2.ism` boshqa-boshqa.

Quyidagi diagramma bitta klassdan ikki alohida obyekt yasalishini va har birining o'z atributlariga ega ekanini ko'rsatadi:



self haqida: har bir metodning birinchi parametri `self` bo'ladi. Uni chaqirayotganda **o'zing yozmaysan** — Python avtomatik uzatadi. `t1.salomla()` desang, Python `self` o'rniga `t1` ni qo'yadi.

Mana `t1 = Talaba("Aziz", 20)` yozilganda Python ichkarida nima qilishi — `__init__` va `self` qadam-baqadam:



5.3 Metodlar — obyektning amallari

Metod — klass ichidagi funksiya. U obyekt bilan biror ish qiladi:

```
class Talaba:
    def __init__(self, ism, yosh):
        self.ism = ism
        self.yosh = yosh

    def salomla(self):
        # metod - birinchi parametr self
        print(f"Salom, men {self.ism}")

    def tugilgan_kun(self):
        self.yosh = self.yosh + 1
        # obyektning atributini o'zgartiradi
        print(f"{self.ism} endi {self.yosh} yoshda")

t = Talaba("Aziz", 20)
t.salomla()
t.tugilgan_kun()
```

Salom, men Aziz
Aziz endi 21 yoshda

Metod ichida boshqa atributlarga `self.` orqali murojaat qilasan. Metod qiymat ham qaytarishi mumkin (`return`):

```
class Hisob:
    def __init__(self, balans):
        self.balans = balans
```

```

def qoshish(self, summa):
    self.balans += summa

def hisobot(self):
    return f"Balans: {self.balans} so'm"

h = Hisob(1000)
h.qoshish(500)
print(h.hisobot())          # Balans: 1500 so'm

```

5.4 `__str__` — obyektни chiroyli chiqarish

Obyektни to'g'ridan-to'g'ri `print` qilsang, tushunarsiz narsa chiqadi:

```

t = Talaba("Aziz", 20)
print(t)          # <__main__.Talaba object at 0x7f...> - foydasiz

```

`__str__` metodini qo'shsang, `print` shuni ko'rsatadi:

```

class Talaba:
    def __init__(self, ism, yosh):
        self.ism = ism
        self.yosh = yosh

    def __str__(self):
        return f"Talaba: {self.ism}, {self.yosh} yosh"

t = Talaba("Aziz", 20)
print(t)          # Talaba: Aziz, 20 yosh

```

5.5 Meros (inheritance) — klassdan klass yasash

Meros — bir klass boshqasining xususiyatlarini "meros qilib olishi". Umumiy narsalarni asosiy klassga yozasan, maxsus narsalarni avlod klasslarga qoldirasan. Bu takrorini kamaytiradi.

```

class Hayvon:
    def __init__(self, ism):
        self.ism = ism

    def ovqatlanish(self):
        print(f"{self.ism} ovqatlanmoqda")

class It(Hayvon):
    # It - Hayvon'dan meros oladi

```

```

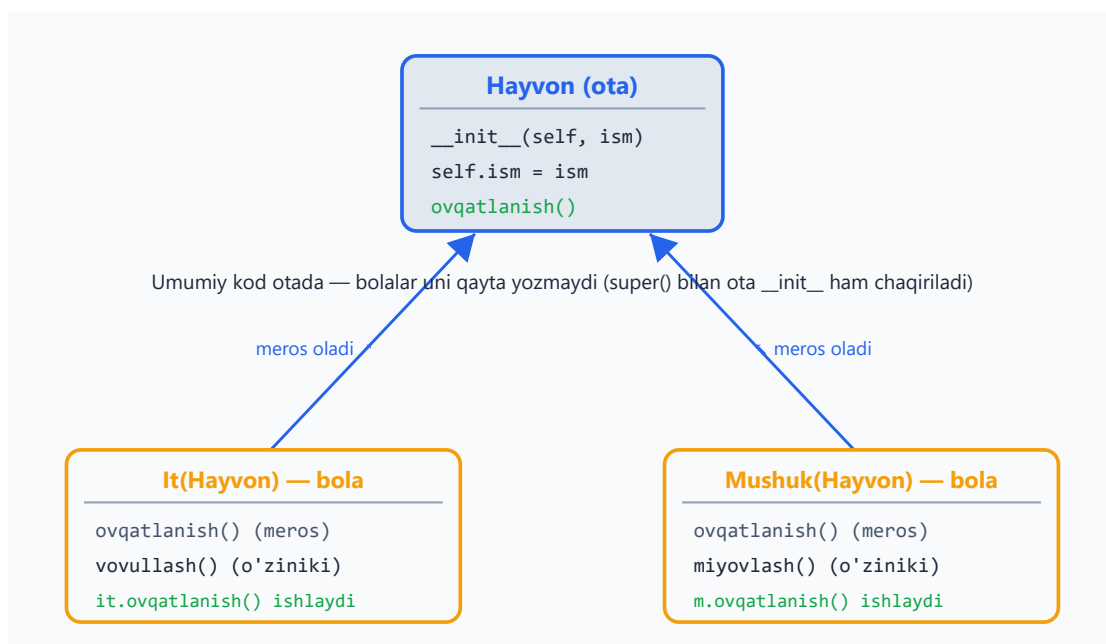
def vovullash(self):
    print(f"{self.ism}: Vov-vov!")

class Mushuk(Hayvon):
    # Mushuk ham
    def miyovlash(self):
        print(f"{self.ism}: Miyov!")

it = It("Boburjon")
it.ovqatlanish()      # Boburjon ovqatlanmoqda    - Hayvon'dan merosga
olindi
it.vovullash()        # Boburjon: Vov-vov!        - It'ning o'ziniki

```

Quyidagi meros daraxti ota klass (`Hayvon`) va undan meros oluvchi bola klasslar (`It` , `Mushuk`) qanday bog'lanishini hamda metod merosini ko'rsatadi:



`super()` — asosiy klassning `__init__` ini chaqirish uchun (avlod qo'shimcha atribut qo'shsa):

```

class It(Hayvon):
    def __init__(self, ism, zot):
        super().__init__(ism) # Hayvon'ning __init__ ini chaqiradi
                               (ism o'rnatiladi)
        self.zot = zot        # qo'shimcha atribut

it = It("Boburjon", "Labrador")
print(it.ism, it.zot)        # Boburjon Labrador

```

Qachon meros? Agar "B — bu A turi" desa bo'lsa (`It` — bu `Hayvon`, `Talaba` — bu `Inson`), meros mos keladi. Umumiy kodni qayta yozmaslik uchun foydali.

5.6 Ilg'or mavzular (hozircha qisqacha)

Python OOP'da yana ko'p imkoniyatlar bor — ularni keyinroq, tajriba ortgach o'rganasan:

- `@property` — atributga "o'qishda" hisoblanuvchi qiymat berish. - `dataclass` — faqat ma'lumot saqlovchi klasslarni qisqa yozish. - Maxsus metodlar (`__eq__` , `__len__` ...) — obyektarga `+` , `==` , `len()` kabi amallarni qo'shish.

Hozircha yuqoridagi asoslar — klass, `__init__` , metod, meros — kundalik ishning 90% ini qoplaydi.



Masalalar (20 ta)

Bu masalalar 1–5 modullar mavzulariga asoslangan.

Oson (1–7):

1. Kitob klassini yarat: `__init__` da nom va muallif atributlarini o'rnat. Bitta obyekt yaratib, atributlarini chiqar.
2. Talaba klassiga `salomla()` metodini qo'sh — "Salom, men {ism}" deb chiqarsin.
3. Doira klassini yarat: `__init__` da radius ni ol, `yuza()` metodi yuzasini $(3.14159 * r * r)$ qaytarsin.
4. Hisoblagich klassini yarat: son atributi 0 dan boshlansin, `oshir()` metodi 1 ga oshirsin.
5. It klassiga `__str__` qo'sh — "It: {ism}" ko'rinishida chiqarsin.
6. Hisob (bank hisobi) klassini yarat: balans atributi bo'lsin, `qoy(summa)` va `yech(summa)` metodlari balansni o'zgartirsin.
7. Bitta Talaba klassidan 3 ta obyekt yaratib, har birining ismini chiqar.

O'rta (8–14):

1. Talaba klassiga ballar (ro'yxat) atributini qo'sh. `ball_qosh(b)` metodi ballni ro'yxatga qo'shsin, `ortacha()` metodi o'rtachani qaytarsin.
2. Hisob klassining `yech` metodida tekshiruv qo'sh: balansdan ko'p yechib bo'lmasin (yetarli bo'lmasa "mablag' yetarli emas" deb yoz).
3. Hayvon asosiy klassini yarat (`ism + ovqatlanish()`), undan It va Mushuk avlod klasslarini yasab, har biriga o'z ovozi qo'sh.
4. `super()` ishlatib: Mashina klassi (marka) va undan ElektrMashina (marka + batareya) yarat.

5. `Toringoq` (to'rtburchak) klassini yarat: `eni`, `boyi` atributlari; `yuza()` va `perimetr()` metodlari.
6. `Kitob` klassiga `__str__` qo'sh: "{nom} – {muallif}" ko'rinishida. Kitoblar ro'yxatini yaratib, har birini `print` qil.
7. `Talaba` klassining ikkita obyektini yarat, ballari o'rtachasini taqqoslab, kim yuqori ekanini chiqar.

Murakkab (15–20):

1. `Stack` (suzgich) klassini yarat: `push(x)` (qo'shish), `pop()` (oxirgini olib qaytarish), `bosh()` (bo'shmi tekshirish) metodlari (ichida ro'yxat ishlat).
2. `Kutubxona` klassini yarat: kitoblarni ro'yxatda saqlasin; `qosh(kitob)`, `royxat()` (barchasini chiqarish), `qidir(nom)` metodlari bo'lsin.
3. `Hayvon` dan meros olgan kamida 3 ta klass yarat. Ularni bitta ro'yxatga solib, sikl bilan har birining ovozini chiqar (har biri o'zicha ovoz chiqarsin).
4. `Vaqt` klassini yarat: `soat`, `daqiq`; `__str__` "SS:MM" ko'rinishida (`{:02d}`); `daqiq_qosh(n)` metodi vaqtni to'g'ri oshirsin (60 daqiqa = 1 soat).
5. Oddiy "Do'kon" tizimi: `Mahsulot` klassi (`nom`, `narx`, `soni`) va `Savat` klassi (mahsulotlarni qo'shish, umumiy summani hisoblash).
6. `BankHisobi` klassini to'liq yoz: `qoy`, `yech` (yetarli mablag' tekshiruvi bilan), va har amalni `tarix` ro'yxatiga yozsin; `tarixni_chiqar()` metodi barcha amallarni ko'rsatsin.

 Yechimlar



```
# 1
class Kitob:
    def __init__(self, nom, muallif):
        self.nom = nom
        self.muallif = muallif
k = Kitob("O'tkan kunlar", "A. Qodiriy")
print(k.nom, "-", k.muallif)

# 2
class Talaba:
    def __init__(self, ism):
        self.ism = ism
    def salomla(self):
        print(f"Salom, men {self.ism}")
Talaba("Aziz").salomla()

# 3
class Doira:
    def __init__(self, radius):
        self.radius = radius
    def yuza(self):
        return 3.14159 * self.radius ** 2
print(Doira(5).yuza())          # 78.53975

# 4
class Hisoblagich:
    def __init__(self):
        self.son = 0
    def oshir(self):
        self.son += 1
h = Hisoblagich(); h.oshir(); h.oshir()
print(h.son)                    # 2

# 5
class It:
    def __init__(self, ism):
        self.ism = ism
    def __str__(self):
        return f"It: {self.ism}"
print(It("Boburjon"))          # It: Boburjon

# 6
class Hisob:
    def __init__(self, balans):
        self.balans = balans
    def qoy(self, summa):
        self.balans += summa
    def yech(self, summa):
        self.balans -= summa
h = Hisob(1000); h.qoy(500); h.yech(200)
print(h.balans)                # 1300

# 7
class Talaba:
    def __init__(self, ism):
        self.ism = ism
talabalar = [Talaba("Aziz"), Talaba("Malika"), Talaba("Bobur")]
```

```

for t in talabalar:
    print(t.ism)

# 8
class Talaba:
    def __init__(self, ism):
        self.ism = ism
        self.ballar = []
    def ball_qosh(self, b):
        self.ballar.append(b)
    def ortacha(self):
        return sum(self.ballar) / len(self.ballar)
t = Talaba("Aziz"); t.ball_qosh(80); t.ball_qosh(90)
print(t.ortacha())          # 85.0

# 9
class Hisob:
    def __init__(self, balans):
        self.balans = balans
    def yech(self, summa):
        if summa > self.balans:
            print("mablag' yetarli emas")
        else:
            self.balans -= summa
h = Hisob(500); h.yech(1000) # mablag' yetarli emas

# 10
class Hayvon:
    def __init__(self, ism):
        self.ism = ism
    def ovqatlanish(self):
        print(f"{self.ism} ovqatlanmoqda")
class It(Hayvon):
    def ovoz(self):
        print(f"{self.ism}: Vov!")
class Mushuk(Hayvon):
    def ovoz(self):
        print(f"{self.ism}: Miyov!")
It("Rex").ovoz()
Mushuk("Kitty").ovoz()

# 11
class Mashina:
    def __init__(self, marka):
        self.marka = marka
class ElektrMashina(Mashina):
    def __init__(self, marka, batareya):
        super().__init__(marka)
        self.batareya = batareya
e = ElektrMashina("Tesla", 100)
print(e.marka, e.batareya) # Tesla 100

# 12
class Tortburchak:
    def __init__(self, eni, boyi):
        self.eni = eni
        self.boyi = boyi
    def yuza(self):
        return self.eni * self.boyi
    def perimetr(self):
        return 2 * (self.eni + self.boyi)

```

```

t = Tortburchak(4, 6)
print(t.yuza(), t.perimetr()) # 24 20

# 13
class Kitob:
    def __init__(self, nom, muallif):
        self.nom = nom
        self.muallif = muallif
    def __str__(self):
        return f"{self.nom} - {self.muallif}"
kitoblar = [Kitob("Mehrobdan chayon", "A. Qodiriy"), Kitob("Sarob", "A.
Qahhor")]
for k in kitoblar:
    print(k)

# 14
class Talaba:
    def __init__(self, ism, ball):
        self.ism = ism
        self.ball = ball
a = Talaba("Aziz", 85)
b = Talaba("Bobur", 72)
print(a.ism if a.ball > b.ball else b.ism) # Aziz

# 15
class Stack:
    def __init__(self):
        self.items = []
    def push(self, x):
        self.items.append(x)
    def pop(self):
        return self.items.pop()
    def bosh(self):
        return len(self.items) == 0
s = Stack(); s.push(1); s.push(2)
print(s.pop(), s.bosh()) # 2 False

# 16
class Kutubxona:
    def __init__(self):
        self.kitoblar = []
    def qosh(self, kitob):
        self.kitoblar.append(kitob)
    def royxat(self):
        for k in self.kitoblar:
            print(k)
    def qidir(self, nom):
        for k in self.kitoblar:
            if k == nom:
                return True
        return False
kt = Kutubxona(); kt.qosh("Sarob"); kt.qosh("O'tkan kunlar")
print(kt.qidir("Sarob")) # True

# 17
class Hayvon:
    def __init__(self, ism):
        self.ism = ism
    def ovoz(self):
        print(f"{self.ism}: ...")
class It(Hayvon):

```

```

        def ovoz(self):
            print(f"{self.ism}: Vov!")
class Mushuk(Hayvon):
    def ovoz(self):
        print(f"{self.ism}: Miyov!")
class Sigir(Hayvon):
    def ovoz(self):
        print(f"{self.ism}: Mu!")
hayvonlar = [It("Rex"), Mushuk("Kitty"), Sigir("Zebo")]
for h in hayvonlar:
    h.ovoiz()

# 18
class Vaqt:
    def __init__(self, soat, daqiqa):
        self.soat = soat
        self.daqiqa = daqiqa
    def __str__(self):
        return f"{self.soat:02d}:{self.daqiqa:02d}"
    def daqiqa_qosh(self, n):
        jami = self.soat * 60 + self.daqiqa + n
        self.soat = (jami // 60) % 24
        self.daqiqa = jami % 60
v = Vaqt(10, 50); v.daqiqa_qosh(20)
print(v)                                # 11:10

# 19
class Mahsulot:
    def __init__(self, nom, narx, soni):
        self.nom = nom
        self.narx = narx
        self.soni = soni
class Savat:
    def __init__(self):
        self.mahsulotlar = []
    def qosh(self, m):
        self.mahsulotlar.append(m)
    def jami(self):
        return sum(m.narx * m.soni for m in self.mahsulotlar)
s = Savat()
s.qosh(Mahsulot("Olma", 12000, 2))
s.qosh(Mahsulot("Non", 4000, 3))
print(s.jami())                        # 36000

# 20
class BankHisobi:
    def __init__(self, balans=0):
        self.balans = balans
        self.tarix = []
    def qoy(self, summa):
        self.balans += summa
        self.tarix.append(f"+{summa}")
    def yech(self, summa):
        if summa > self.balans:
            self.tarix.append(f"RAD: {summa} (mablag' yetarli emas)")
        else:
            self.balans -= summa
            self.tarix.append(f"-{summa}")
    def tarixni_chiqar(self):
        for amal in self.tarix:
            print(amal)

```

```
b = BankHisobi(1000)
b.qoy(500)
b.yech(2000)
b.yech(300)
b.tarixni_chiqar()
print("Balans:", b.balans)    # 1200
```

[← Stringlar](#) | [Boshlovchilar README ↑](#) | [Keyingi: Modullar va muhit →](#)

06 — Modullar va muhit

Dasturlaring kattalashganda, butun kodni bitta faylga yozish noqulay bo'lib qoladi. Python kodni **modullarga** (alohida fayllarga) bo'lish va boshqalar yozgan tayyor kutubxonalardan foydalanish imkonini beradi. Bu modulda kodni qanday bo'lish, tashqi kutubxonalarni o'rnatish va loyihalarni toza saqlashni o'rganasan.

Bu modulda: `import`, standart kutubxonadan foydalanish, o'z modulingni yozish, `if __name__ == "__main__":`, `pip` bilan kutubxona o'rnatish, va virtual muhit (venv).

6.1 Modul nima? `import`

Modul — bu Python kodi bo'lgan `.py` fayl. Boshqa fayldagi kodni `import` orqali olib ishlatasan. Python o'zida ko'plab tayyor modullar bilan keladi (standart kutubxona).

`import` qilganingda Python modulni qator joylardan ma'lum tartib bilan qidiradi — quyidagi diagramma shu jarayonni ko'rsatadi:



```
import math                # butun modulni import qilish

print(math.sqrt(16))       # 4.0      - modul nomi.funksiya
print(math.pi)             # 3.14159...
```

Faqat kerakli qismini olish (`from`):


```
from math import sqrt, pi    # faqat sqrt va pi

print(sqrt(25))              # 5.0    - endi math. yozish shart emas
print(pi)
```

Modulga qisqa nom berish (`as`):

```
import random as r          # random ni r deb chaqiramiz

print(r.randint(1, 6))      # 1..6 oralig'ida tasodifiy son
```

Bir nechta foydali standart modul:

```
import math                 # matematik funksiyalar (sqrt, ceil, floor, pi...)
import random               # tasodifiy sonlar va tanlovlar
import datetime             # sana va vaqt
```

```
import random
print(random.choice(["olma", "banan", "uzum"]))    # tasodifiy bittasi
print(random.randint(1, 100))                     # 1..100 tasodifiy
```

6.2 O'z modulingni yozish

Har qanday `.py` fayl — modul. Masalan `hisob.py` faylini yarat:

```
# hisob.py
def qosh(a, b):
    return a + b

def kop(a, b):
    return a * b

PI = 3.14159
```

Boshqa fayldan (masalan `main.py`) uni import qilasan:

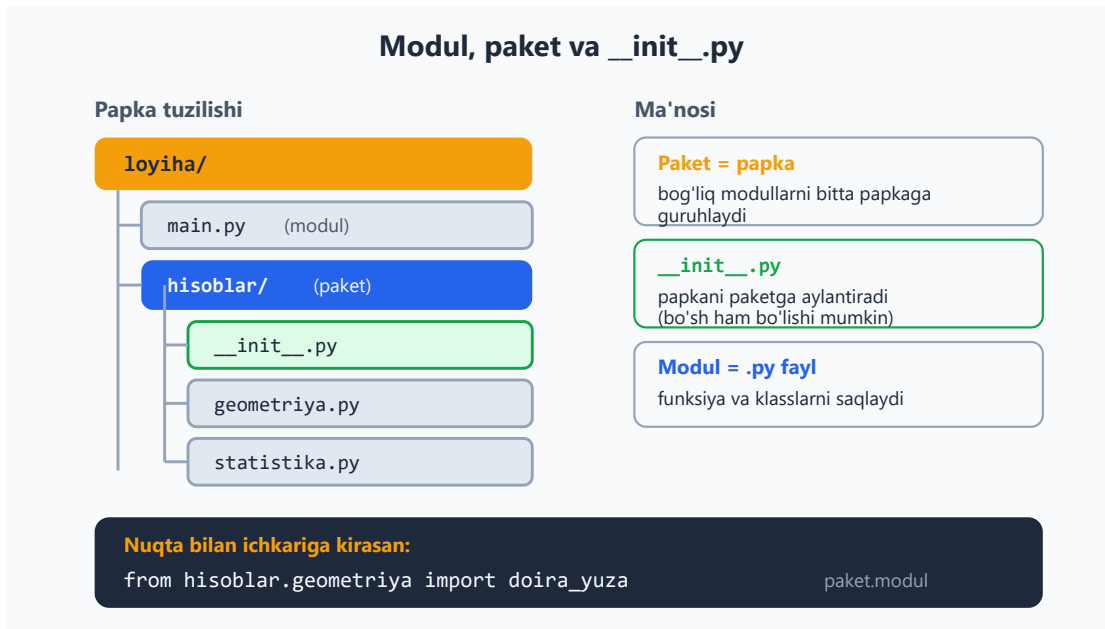
```
# main.py (hisob.py bilan bir papkada)
import hisob

print(hisob.qosh(3, 5))    # 8
print(hisob.PI)            # 3.14159

# yoki:
from hisob import qosh
print(qosh(10, 20))        # 30
```

Bu kodni tartibli saqlashning asosiy yo'li: bog'liq funksiyalarni alohida modullarga ajratasan (masalan `hisob.py`, `fayllar.py`, `foydalanuvchi.py`), keyin `main.py` da birlashtirasan.

Loyiha o'sganda bog'liq modullarni bitta papkaga — **paketga** — guruhlaysan. Papka `__init__.py` faylini olganida paketga aylanadi va ichidagi modullarga nuqta bilan murojaat qilasan:



6.3 `if __name__ == "__main__":`

Bu satrni Python fayllarida tez-tez ko'rasan. Uning ma'nosi: **"agar bu fayl to'g'ridan-to'g'ri ishga tushirilsa"** (import qilinmasa).

```
# hisob.py
def qosh(a, b):
    return a + b

if __name__ == "__main__":
    # bu blok faqat 'python hisob.py' deganda ishlaydi,
    # 'import hisob' qilinganda ishlamaydi
    print("Test:", qosh(2, 3))
```

Nega kerak? Modulingni boshqa joydan import qilganda, undagi test/namoyish kodi avtomatik ishlab ketmasligi uchun. Funksiyalar import qilinadi, lekin `if __name__ == "__main__":` ichidagi kod faqat fayl o'zi ishga tushirilganda bajariladi. Hozircha "fayl bevosita ishga tushganda ishlaydigan kod" deb tushun.

6.4 `pip` — tashqi kutubxonalarni o'rnatish

Standart kutubxonadan tashqari, jamoa yozgan minglab kutubxonalar bor (internetda, PyPI omborida). Ularni `pip` bilan o'rnatasan (terminalda):

```
pip install requests      # 'requests' kutubxonasini o'rnatadi
                           (internet so'rovlari uchun)
pip install rich           # chiroyli terminal chiqishi uchun
```

O'rnatilgandan keyin koddan ishlatasan:

```
import requests           # endi import qilish mumkin
```

Foydali buyruqlar:

```
pip list                  # o'rnatilgan kutubxonalar ro'yxati
pip install --upgrade requests # yangilash
pip uninstall requests    # o'chirish
```

6.5 Virtual muhit (`venv`) — loyihalarni ajratish

Har loyihaning o'z kutubxonalar to'plami bo'lishi kerak. Aks holda turli loyihalar bir-biriga xalaqit beradi (biriga kerakli versiya boshqasini buzishi mumkin). **Virtual muhit (`venv`)** — har loyiha uchun alohida, izolyatsiyalangan Python muhiti.

```
# 1. Virtual muhit yaratish (loyiha papkasida):
python -m venv venv

# 2. Uni faollashtirish:
#   Linux / macOS:
source venv/bin/activate
#   Windows:
venv\Scripts\activate

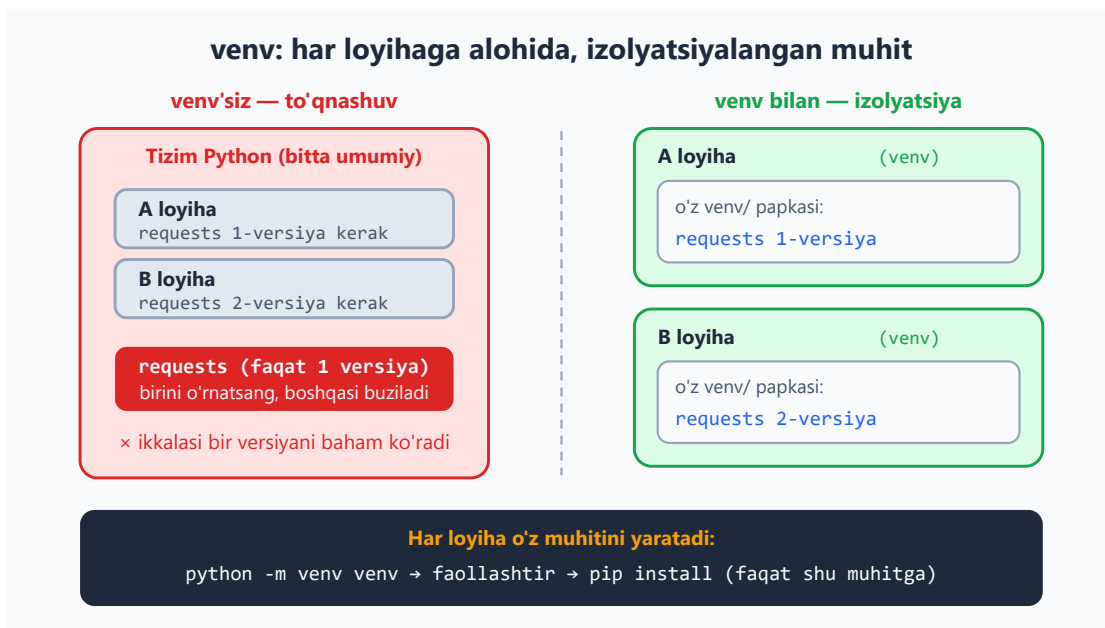
# 3. Endi pip install shu muhitga o'rnatadi (tizimga emas):
pip install requests

# 4. Ishni tugatib, chiqish:
deactivate
```

Faollashtirilganda terminalda `(venv)` belgisi paydo bo'ladi — demak izolyatsiyalangan muhitdasan.

Nega muhim? Tasavvur qil: A loyihaga kutubxonaning 1-versiyasi, B loyihaga 2-versiyasi kerak. Virtual muhitsuiz ular to'qnashadi. Har loyihaga alohida venv — har birida kerakli versiya. Bu professional amaliyot, har doim ishlat.

Quyidagi diagramma venv'siz to'qnashuv va venv bilan izolyatsiya o'rtasidagi farqni ko'rsatadi:



6.6 requirements.txt — loyiha bog'liqliklari

Loyihangda qaysi kutubxonalar kerakligini bitta faylda saqlaysan — requirements.txt. Shunda boshqa odam (yoki sen boshqa kompyuterda) hammasini bir buyruq bilan o'rnatadi:

```
# Joriy kutubxonalarni faylga yozish:
pip freeze > requirements.txt

# Boshqa joyda hammasini o'rnatish:
pip install -r requirements.txt
```

requirements.txt taxminan shunday ko'rinadi:

```
requests==2.31.0
rich==13.7.0
```

Loyihangni GitHub'ga qo'yganingda requirements.txt ni ham qo'sh — shunda boshqalar loyihangni oson ishga tushiradi. venv/ papkasini esa qo'shma (u katta va har kompyuterda qayta yaratiladi).



Masalalar (20 ta)

Ba'zi masalalar terminalda buyruq bajarishni talab qiladi. Modul yozish masalalarida ikkita fayl yaratib sina.

Oson (1–7):

1. `math` modulini import qilib, 144 ning kvadrat ildizini chiqar.
2. `random` modulidan `randint` bilan 1–6 oralig'ida tasodifiy son chiqar (zar tashlash).
3. `from math import pi` qilib, radiusi 5 bo'lgan doira yuzasini hisobla.
4. `random.choice` bilan ro'yxatdan tasodifiy element tanla.
5. `import random as r` qilib, `r.randint` bilan 3 ta tasodifiy son chiqar.
6. Terminalda `pip list` ni ishga tushirib, o'rnatilgan kutubxonalarni ko'r (kod emas, terminal).
7. `math.ceil` va `math.floor` bilan 4.3 ni yuqoriga va pastga yaxlitla.

O'rta (8–14):

1. `hisob.py` moduli yarat (`qosh`, `ayir` funksiyalari bilan), uni `main.py` dan import qilib ishlat.
2. `salom.py` modul yarat (`salomlash(ism)` funksiyasi), boshqa fayldan `from salom import salomlash` qilib chaqir.
3. `hisob.py` ga `if __name__ == "__main__":` qo'shib, ichida test kodi yoz. Faylni to'g'ridan-to'g'ri ishga tushirib tekshir.
4. `random.shuffle` bilan `[1,2,3,4,5]` ro'yxatini aralashtir.
5. `datetime` modulidan foydalanib, bugungi sanani chiqar (`datetime.date.today()`).
6. Terminalda virtual muhit yarat (`python -m venv venv`) va faollashtir.
7. Virtual muhitda biror kutubxona o'rnat (`pip install rich`) va `pip list` bilan tekshir.

Murakkab (15–20):

1. Ikkita modul yarat: `geometriya.py` (`doira_yuza`, `tortburchak_yuza`) va `main.py` (ularni import qilib ishlatadi).
2. `random` bilan oddiy "son top" o'yini: kompyuter 1–100 oralig'ida son o'ylasin, foydalanuvchi topguncha "katta"/"kichik" deb yo'naltir.

3. `utils.py` moduli yarat: kamida 3 ta foydali funksiya (masalan `katta_harf`, `teskari`, `unli_sanash`), `main.py` dan hammasini ishlat.
 4. Loyiha yarat: virtual muhit, `requirements.txt`, va `pip freeze` bilan uni to'ldir.
 5. `random` va `math` ni birga ishlatib: 5 ta tasodifiy sonning kvadrat ildizlari yig'indisini hisobla.
 6. Ikki moduldan iborat kichik loyiha: `bank.py` (`Hisob` klassi — 5-moduldan) va `main.py` (hisob yaratib, amallar bajaradi). Klassni modulga joylashtirib, import qilib ishlat.
-

 Yechimlar



```
# 1
import math
print(math.sqrt(144))          # 12.0

# 2
import random
print(random.randint(1, 6))

# 3
from math import pi
print(pi * 5 ** 2)            # 78.539...

# 4
import random
print(random.choice(["olma", "banan", "uzum"]))

# 5
import random as r
print(r.randint(1, 100), r.randint(1, 100), r.randint(1, 100))

# 6
# Terminalda: pip list

# 7
import math
print(math.ceil(4.3), math.floor(4.3))    # 5 4

# 8
# hisob.py:
# def qosh(a, b): return a + b
# def ayir(a, b): return a - b
# main.py:
import hisob
print(hisob.qosh(3, 5))          # 8
print(hisob.ayir(10, 4))        # 6

# 9
# salom.py:
# def salomlash(ism): print(f"Salom, {ism}!")
# main.py:
from salom import salomlash
salomlash("Aziz")               # Salom, Aziz!

# 10
# hisob.py:
def qosh(a, b):
    return a + b
if __name__ == "__main__":
    print("Test:", qosh(2, 3))    # faqat 'python hisob.py' deganda chiqadi

# 11
import random
sonlar = [1, 2, 3, 4, 5]
random.shuffle(sonlar)
print(sonlar)                   # masalan [3, 1, 5, 2, 4]

# 12
```

```

import datetime
print(datetime.date.today()) # masalan 2026-06-09

# 13
# Terminalda:
# python -m venv venv
# source venv/bin/activate      (Windows: venv\Scripts\activate)

# 14
# Terminalda (venv faol):
# pip install rich
# pip list

# 15
# geometriya.py:
# def doira_yuza(r): return 3.14159 * r * r
# def tortburchak_yuza(a, b): return a * b
# main.py:
import geometriya
print(geometriya.doira_yuza(5))
print(geometriya.tortburchak_yuza(4, 6))

# 16
import random
maxfiy = random.randint(1, 100)
while True:
    taxmin = int(input("Taxminingiz (1-100): "))
    if taxmin == maxfiy:
        print("Topdingiz!")
        break
    elif taxmin < maxfiy:
        print("Kattaroq son")
    else:
        print("Kichikroq son")

# 17
# utils.py:
# def katta_harf(s): return s.upper()
# def teskari(s): return s[::-1]
# def unli_sanash(s): return sum(1 for h in s.lower() if h in "aeiou")
# main.py:
from utils import katta_harf, teskari, unli_sanash
print(katta_harf("salom")) # SALOM
print(teskari("salom")) # molas
print(unli_sanash("salom")) # 2

# 18
# Terminalda:
# python -m venv venv
# source venv/bin/activate
# pip install requests
# pip freeze > requirements.txt

# 19
import random, math
sonlar = [random.randint(1, 100) for _ in range(5)]
print(sum(math.sqrt(s) for s in sonlar))

# 20
# bank.py:
# class Hisob:

```



```
#         def __init__(self, balans=0): self.balans = balans
#         def qoy(self, s): self.balans += s
#         def yech(self, s): self.balans -= s
# main.py:
from bank import Hisob
h = Hisob(1000)
h.qoy(500)
h.yech(200)
print(h.balans)                # 1300
```

[← OOP | Boshlovchilar README ↑](#) | [Keyingi: Xatolarni boshqarish →](#)

07 — Xatolarni boshqarish (`try` / `except`)

Dasturlarda xatolar muqarrar: foydalanuvchi raqam o'rniga matn kiritadi, fayl topilmaydi, nolga bo'lish sodir bo'ladi. Agar xatoni boshqarmasang, dastur **to'satdan to'xtaydi** (qulaydi). Bu modulda xatolarni ushlab, dasturni nazoratda saqlashni o'rganasan.

Bu modulda: xato (exception) nima, `try` / `except`, aniq xato turlarini ushlash, `else` / `finally`, va o'zing xato chaqirish (`raise`).

7.1 Xato nima? Nega dastur "qulaydi"?

Xato yuz berganda, agar uni ushlamasang, dastur shu yerda to'xtaydi:

```
yosh = int(input("Yoshing: "))      # foydalanuvchi "salom" yozsa...
print(yosh)
# ValueError: invalid literal for int() ... – dastur QULADI, keyingi
kod ishlamaydi
```

Tez-tez uchraydigan xato turlari:

Xato	Qachon	Misol
<code>ValueError</code>	qiymat noto'g'ri	<code>int("salom")</code>
<code>ZeroDivisionError</code>	nolga bo'lish	<code>10 / 0</code>
<code>TypeError</code>	tip mos kelmaydi	<code>"matn" + 5</code>
<code>KeyError</code>	lug'atda kalit yo'q	<code>lugat["yo'q"]</code>
<code>IndexError</code>	ro'yxatda indeks yo'q	<code>royxat[100]</code>
<code>FileNotFoundError</code>	fayl topilmadi	mavjud bo'lmagan faylni ochish

7.2 `try` / `except` — xatoni ushlash

`try` blokiga "xato bo'lishi mumkin" kodni yozasan, `except` ga xato yuz berganda nima qilishni:

```
try:
    yosh = int(input("Yoshing: "))
    print(f"Siz {yosh} yoshdasiz")
except:
    print("Iltimos, to'g'ri raqam kiriting")
```

Endi foydalanuvchi "salom" yozsa, dastur qulamaydi — `except` bloki ishlaydi va dastur davom etadi.

```
try:
    natija = 10 / 0
except:
    print("Nolga bo'lib bo'lmaydi!")    # shu chiqadi, dastur tirik qoladi
```

7.3 Aniq xato turini ushlash

Har qanday xatoni emas, **aniq turini** ushlash yaxshiroq — shunda har xil xatoga har xil javob berasan:

```
try:
    son = int(input("Son: "))
    natija = 100 / son
    print(natija)
except ValueError:
    print("Bu raqam emas!")
except ZeroDivisionError:
    print("Nolga bo'lib bo'lmaydi!")
```

Nega aniq tur? "Hamma xatoni ushla" (`except:`) ba'zan haqiqiy muammoni yashiradi — masalan kodingdagi xatoni ham "yutib" yuboradi. Aniq turni ushlasang, faqat kutgan xatoga javob berasan, kutilmagani esa ko'rinadi (va sen tuzatasan).

Bir nechta turni birga ushlash:

```
try:
    ...
except (ValueError, TypeError):
    print("Qiymat yoki tip xatosi")
```

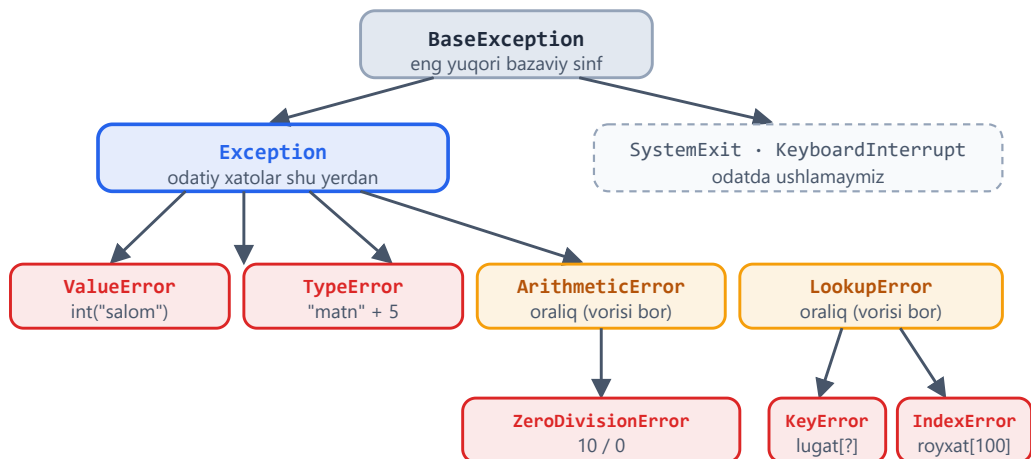
Xato haqida ma'lumot olish (`as`):

```
try:
    int("salom")
except ValueError as e:
    print(f"Xato yuz berdi: {e}")    # Xato yuz berdi: invalid literal
    for int()...
```

Xato turlari tasodifiy emas — ular vorislik daraxti (ierarxiya) bo'yicha tartibga solingan, va yuqori turni ushlasang uning ostidagi barcha turlar ham ushlanadi:

Exception ierarxiyasi (vorislik daraxti)

Yuqori turni ushlasang, uning ostidagi barcha turlar ham ushlanadi



Muhim qoida:

except Exception barcha odatiy xatolarni ushlaydi · except LookupError esa KeyError va IndexError ni (chunki ular vorislari)

7.4 else va finally

else — xato bo'lmaganda ishlaydi:

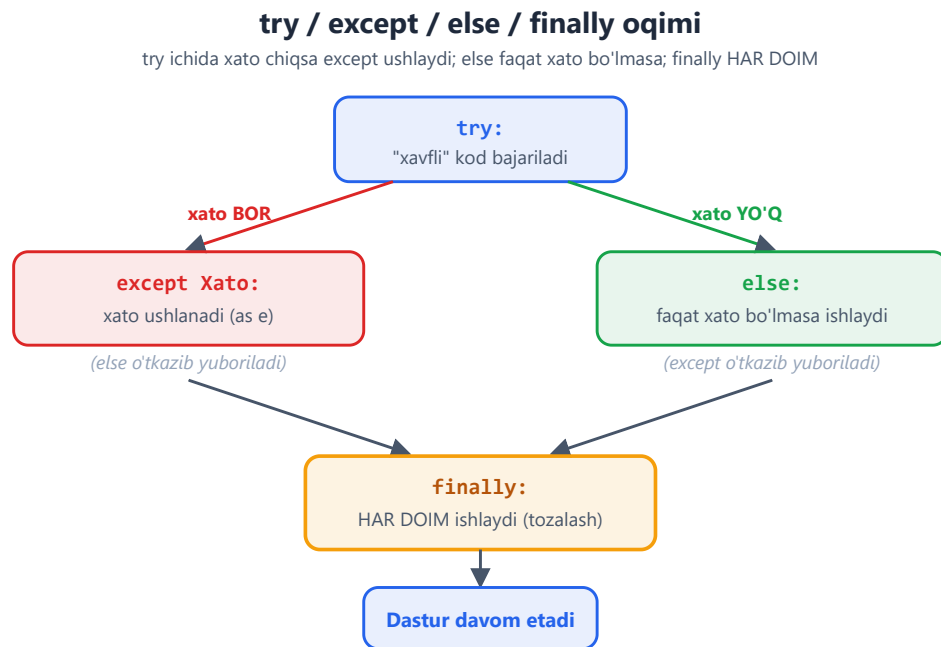
```
try:
    son = int(input("Son: "))
except ValueError:
    print("Noto'g'ri kiritish")
else:
    print(f"Rahmat! Siz {son} kiritdingiz")    # faqat xato bo'lmasa
```

finally — xato bo'ladimi-yo'qmi, **doim** ishlaydi (odatda tozalash uchun — faylni yopish va h.k.):

```
try:
    fayl_ochish()
```

```
except FileNotFoundError:
    print("Fayl topilmadi")
finally:
    print("Bu har doim bajariladi")    # tozalash kodi shu yerga
```

Quyidagi diagramma to'rt blokning oqimini ko'rsatadi — xato bo'lganda va bo'lmaganda qaysi blok ishlashini:



7.5 `raise` — o'zing xato chaqirish

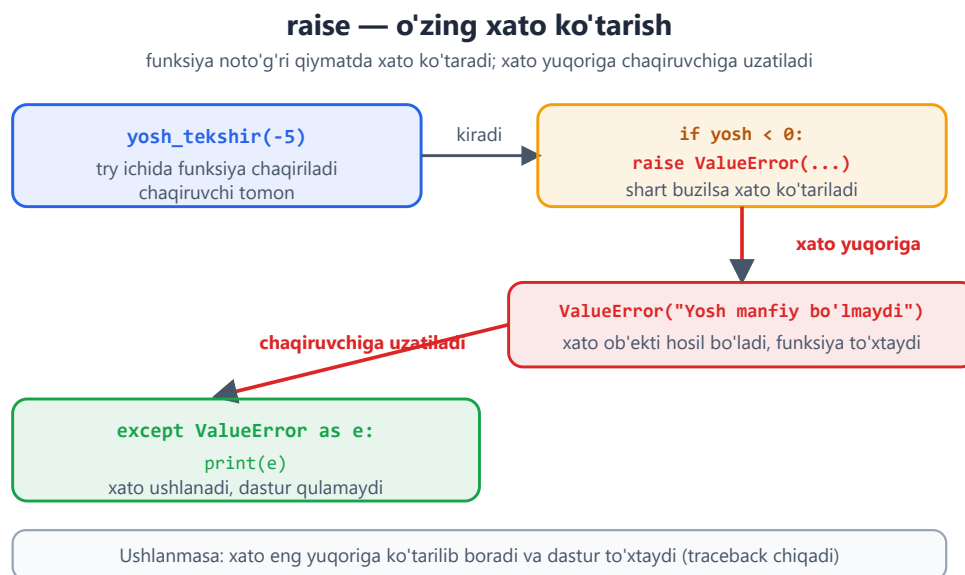
Ba'zan o'zing xato "chaqirishing" kerak — masalan, noto'g'ri qiymat berilganda dasturni to'xtatish uchun:

```
def yosh_tekshir(yosh):
    if yosh < 0:
        raise ValueError("Yosh manfiy bo'lishi mumkin emas")
    return yosh

try:
    yosh_tekshir(-5)
except ValueError as e:
    print(e)    # Yosh manfiy bo'lishi mumkin emas
```

Qachon `raise`? Funksiyanga noto'g'ri ma'lumot kelganda, uni jimgina qabul qilish o'rniga xato chaqir. Shunda muammo erta ko'rinadi va chaqiruvchi kod uni ushlab, to'g'ri javob beradi.

Quyidagi diagramma `raise` qilingan xato funksiyadan chaqiruvchi tomonga qanday uzatilishini va u yerda `except` bilan ushlanishini ko'rsatadi:



7.6 Amaliy namuna: ishonchli kiritish

Xato boshqarishning eng keng tarqalgan ishlatilishi — foydalanuvchi to'g'ri ma'lumot kiritguncha qayta-qayta so'rash:

```
while True:
    try:
        yosh = int(input("Yoshingiz: "))
        break
    except ValueError:
        print("Iltimos, butun son kiriting")

print(f"Yoshingiz: {yosh}")
```

Bu naqsh (`while True + try/except + break`) juda foydali — foydalanuvchi xato qilsa, dastur qulamaydi, balki muloyim qayta so'raydi.

Masalalar (20 ta)

Bu masalalar 1–7 modullar mavzulariga asoslangan.

Oson (1–7):

1. Foydalanuvchidan son so'ra. `try/except` bilan: raqam bo'lmasa "Bu raqam emas" deb chiqar.
2. `10 / 0` ni `try/except` ichida yozib, `ZeroDivisionError` ni ushla.
3. Foydalanuvchidan ikki son so'rab, birinchisini ikkinchisiga bo'l. Nolga bo'lishni ushlab, xabar chiqar.
4. Ro'yxat `[1, 2, 3]` ning 10-indeksiga murojaat qilib, `IndexError` ni ushla.
5. Lug'atdan mavjud bo'lmagan kalitni so'rab, `KeyError` ni ushla.
6. `int("salom")` ni `try/except ValueError` bilan ushlab, xato xabarini (`as e`) chiqar.
7. `try/except/else` ishlat: son to'g'ri kiritilsa `else` da "rahmat" deb yoz.

O'rta (8–14):

1. `while True` + `try/except` bilan: foydalanuvchi to'g'ri butun son kiritguncha qayta so'ra.
2. Bir funksiya yoz: ikki sonni bo'lsin, lekin nolga bo'lishni `try/except` bilan ushlab, `None` qaytarsin.
3. Ikki xil xatoni alohida ushla (`ValueError` va `ZeroDivisionError`), har biriga boshqa xabar ber.
4. `finally` ishlat: xato bo'ladimi-yo'qmi, oxirida "Tugadi" deb chiqaruvchi kod yoz.
5. `raise` ishlat: `yosh < 0` bo'lsa `ValueError` chaqiradigan funksiya yoz.
6. Foydalanuvchidan ro'yxat indeksini so'rab, shu indeksdagi elementni chiqar. Noto'g'ri indeks bo'lsa "Bunday element yo'q" deb yoz.
7. Bir funksiya yoz: matnni songa aylantirsin, agar aylantirib bo'lmasa 0 qaytarsin (xatoni ushlab).

Murakkab (15–20):

1. Oddiy kalkulyator: ikki son va amal so'ra. Noto'g'ri son yoki nolga bo'lishni `try/except` bilan boshqarib, foydalanuvchini quladmagan.
2. `raise` ishlat: bank hisobidan balansdan ko'p yechishga urinilsa `ValueError("mablag' yetarli emas")` chaqiruvchi funksiya yoz, uni `try/except` bilan sina.
3. Foydalanuvchidan bir nechta son so'rab (sikl bilan), faqat to'g'ri kiritilganlarini ro'yxatga qo'sh, xato kiritilganlarni o'tkazib yubor.
4. Lug'atdan xavfsiz qiymat oluvchi funksiya yoz: kalit bo'lsa qiymatni, bo'lmasa "topilmadi" qaytarsin (`try/except KeyError` bilan, keyin `get` bilan ham qilib taqqosla).

5. O'z funksiyangda kirish tekshiruvi: `bolish(a, b)` funksiyasi `b == 0` bo'lsa o'zi xato chaqirsin (`raise`), chaqiruvchi tomon esa ushlasin.
 6. "Ishonchli son kiritish" funksiyasini yoz: parametr sifatida savol matnini olsin, foydalanuvchi to'g'ri butun son kiritguncha so'rasin, va sonni qaytarsin. Bir nechta joyda qayta ishlatib ko'r.
-

 Yechimlar



```
# 1
try:
    son = int(input("Son: "))
    print(son)
except ValueError:
    print("Bu raqam emas")

# 2
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Nolga bo'lib bo'lmaydi")

# 3
try:
    a = int(input("a: "))
    b = int(input("b: "))
    print(a / b)
except ZeroDivisionError:
    print("Nolga bo'lib bo'lmaydi")

# 4
try:
    print([1, 2, 3][10])
except IndexError:
    print("Bunday indeks yo'q")

# 5
lugat = {"ism": "Aziz"}
try:
    print(lugat["yosh"])
except KeyError:
    print("Bunday kalit yo'q")

# 6
try:
    int("salom")
except ValueError as e:
    print(f"Xato: {e}")

# 7
try:
    son = int(input("Son: "))
except ValueError:
    print("Noto'g'ri")
else:
    print("rahmat")

# 8
while True:
    try:
        son = int(input("Butun son: "))
        break
    except ValueError:
        print("Iltimos, butun son kiriting")
print("Kiritildi:", son)
```

```

# 9
def bol(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return None
print(bol(10, 2), bol(10, 0))    # 5.0 None

# 10
try:
    son = int(input("Son: "))
    print(100 / son)
except ValueError:
    print("Raqam emas")
except ZeroDivisionError:
    print("Nolga bo'lib bo'lmaydi")

# 11
try:
    print("ishlayapti")
except Exception:
    print("xato")
finally:
    print("Tugadi")

# 12
def yosh_tekshir(yosh):
    if yosh < 0:
        raise ValueError("Yosh manfiy bo'lmaydi")
    return yosh
try:
    yosh_tekshir(-3)
except ValueError as e:
    print(e)

# 13
royxat = [10, 20, 30]
try:
    i = int(input("Indeks: "))
    print(royxat[i])
except (IndexError, ValueError):
    print("Bunday element yo'q")

# 14
def songa(matn):
    try:
        return int(matn)
    except ValueError:
        return 0
print(songa("42"), songa("salom"))    # 42 0

# 15
try:
    a = float(input("1-son: "))
    b = float(input("2-son: "))
    amal = input("Amal (+ - * /): ")
    if amal == "+":
        print(a + b)
    elif amal == "-":
        print(a - b)
    elif amal == "*":

```

```

        print(a * b)
    elif amal == "/":
        print(a / b)
except ValueError:
    print("Noto'g'ri son")
except ZeroDivisionError:
    print("Nolga bo'lib bo'lmaydi")

# 16
def yech(balans, summa):
    if summa > balans:
        raise ValueError("mablag' yetarli emas")
    return balans - summa
try:
    print(yech(500, 1000))
except ValueError as e:
    print(e)

# 17
sonlar = []
for _ in range(5):
    qiymat = input("Son: ")
    try:
        sonlar.append(int(qiymat))
    except ValueError:
        print(f"'{qiymat}' o'tkazib yuborildi")
print(sonlar)

# 18
lugat = {"ism": "Aziz", "yosh": 20}
def olish(kalit):
    try:
        return lugat[kalit]
    except KeyError:
        return "topilmadi"
print(olish("ism"), olish("email"))    # Aziz topilmadi
# get bilan ham:
print(lugat.get("email", "topilmadi")) # topilmadi

# 19
def bolish(a, b):
    if b == 0:
        raise ValueError("nolga bo'lish mumkin emas")
    return a / b
try:
    print(bolish(10, 0))
except ValueError as e:
    print(e)

# 20
def son_sora(savol):
    while True:
        try:
            return int(input(savol))
        except ValueError:
            print("Iltimos, butun son kiriting")
yosh = son_sora("Yoshingiz: ")
miqdor = son_sora("Miqdor: ")
print(yosh, miqdor)

```

[← Modullar va muhit](#) | [Boshlovchilar README](#) [↑](#) | [Keyingi: Fayllar, JSON, CSV](#) [→](#)

08 — Fayllar, JSON va CSV

Hozirgacha ma'lumot faqat dastur ishlab turganda xotirada saqlangan — dastur tugagach yo'qolgan. **Fayllar** ma'lumotni doimiy saqlash imkonini beradi: matn yozish, o'qish, sozlamalarni saqlash. Bu modulda fayllar bilan ishlash, hamda ma'lumotni JSON va CSV formatlarida saqlashni o'rganasan.

Bu modulda: fayl o'qish va yozish (`open`, `with`), `utf-8` kodlash, `pathlib`, JSON bilan ma'lumot saqlash/yuklash, va CSV jadvallar.

8.1 Faylga yozish

Faylni `open()` bilan ochasan. Eng to'g'ri usul — `with` bilan (u faylni avtomatik yopadi):

```
with open("salom.txt", "w", encoding="utf-8") as f:
    f.write("Salom, dunyo!\n")
    f.write("Ikkinchi qator\n")
# blok tugaganda fayl avtomatik yopiladi
```

`open()` ning ikkinchi parametri — **rejim**:

Rejim	Ma'nosi
"w"	yoziq (mavjud faylni o'chirib qaytadan yozadi)
"a"	qo'shish (mavjud faylning oxiriga qo'shadi)
"r"	o'qish (standart)

`encoding="utf-8"` — juda muhim! O'zbekcha (yoki har qanday o'zbek kirill/lotin) matn bilan ishlaganda doim `encoding="utf-8"` yoz. Aks holda ba'zi tizimlarda harflar buzilib chiqishi mumkin. Buni odat qil.

8.2 Fayldan o'qish

```
# Butun faylni bir matn sifatida o'qish:
with open("salom.txt", "r", encoding="utf-8") as f:
    matn = f.read()
```

```

print(matn)

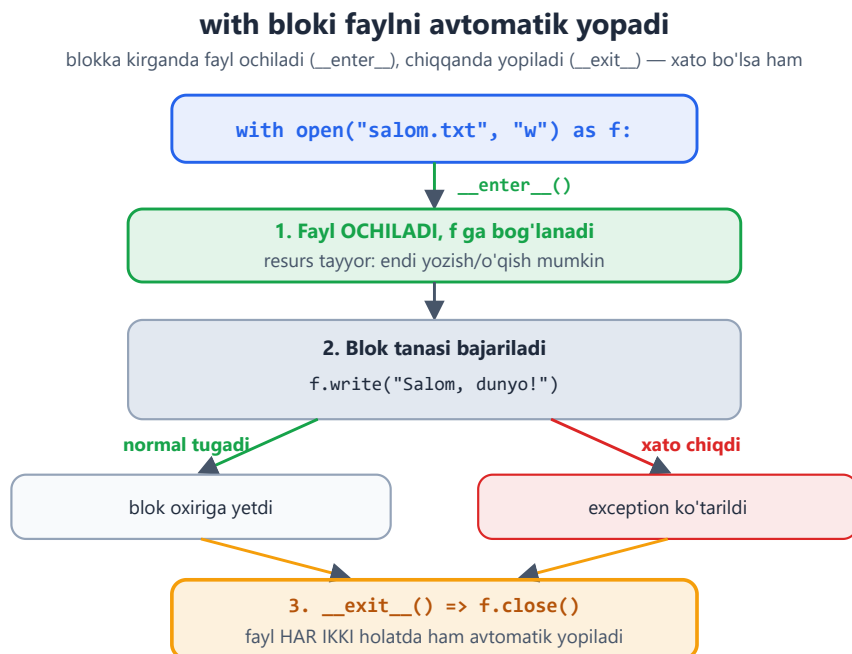
# Qator-qator o'qish (katta fayllar uchun eng yaxshi yo'l):
with open("salom.txt", "r", encoding="utf-8") as f:
    for qator in f:
        print(qator.strip())    # strip() – qator oxiridagi \n ni olib
                                tashlaydi

# Barcha qatorlarni ro'yxatga olish:
with open("salom.txt", "r", encoding="utf-8") as f:
    qatorlar = f.readlines()    # ['Salom...\n', 'Ikkinchi...\n']

```

Nega with? with bloki tugaganda fayl avtomatik va ishonchli yopiladi — xato bo'lsa ham. with siz f.close() ni o'zing yozishing kerak bo'lardi va unutish oson. Doim with ishlat.

Quyidagi diagramma with blokining ichki mexanizmini ko'rsatadi: blokka kirganda fayl ochiladi, chiqqanda esa (xato bo'lsa ham) avtomatik yopiladi.



8.3 pathlib — fayl yo'llari bilan ishlash

pathlib — fayl va papka yo'llari bilan ishlashning zamonaviy usuli:

```

from pathlib import Path

yol = Path("hujjatlar/salom.txt")

print(yol.name)    # salom.txt    – fayl nomi

```

```

print(yol.stem)          # salom      - kengaytmasiz nom
print(yol.suffix)        # .txt       - kengaytma
print(yol.exists())      # True/False - fayl mavjudmi?

# Yo'llarni / bilan birlashtirish (qulay!):
papka = Path("hujjatlar")
fayl = papka / "yangi.txt"      # hujjatlar/yangi.txt

# pathlib bilan o'qish/yozish - qisqaroq:
Path("salom.txt").write_text("Salom!", encoding="utf-8")
matn = Path("salom.txt").read_text(encoding="utf-8")

```

8.4 JSON — ma'lumotni saqlash va yuklash

JSON — ma'lumotni matn ko'rinishida saqlash formati. Python lug'at/ro'yxatlarini faylga saqlab, keyin qayta yuklash uchun ideal (masalan, dastur sozlamalari yoki foydalanuvchi ma'lumotlari).

```

import json

malumot = {
    "ism": "Aziz",
    "yosh": 20,
    "tillar": ["o'zbek", "ingliz"],
}

# Faylga saqlash:
with open("malumot.json", "w", encoding="utf-8") as f:
    json.dump(malumot, f, ensure_ascii=False, indent=2)

# Fayldan yuklash:
with open("malumot.json", "r", encoding="utf-8") as f:
    yuklangan = json.load(f)

print(yuklangan["ism"])  # Aziz

```

ensure_ascii=False — o'zbekcha harflar to'g'ri (o'qiladigan) ko'rinishda saqlanishi uchun. Busiz harflar `\u...` kodlariga aylanib ketadi. **indent=2** — faylni chiroyli, o'qish oson qilib formatlaydi.

Matn va Python obykti orasida aylantirish (faylsiz):

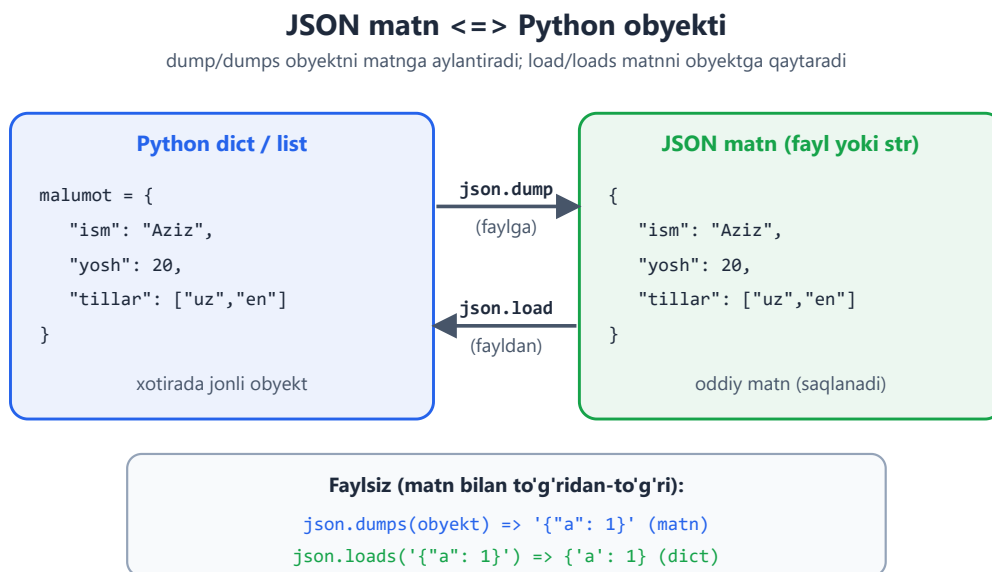
```

import json

matn = json.dumps({"a": 1})  # obyekt → JSON matn: '{"a": 1}'
obyekt = json.loads('{"a": 1}')  # JSON matn → obyekt: {'a': 1}

```

Quyidagi diagramma JSON matn va Python obykti orasidagi ikki tomonlama moslikni ko'rsatadi.



8.5 CSV — jadval ma'lumotlari

CSV — jadval ko'rinishidagi ma'lumot (Excel kabi: qatorlar va ustunlar). Har qator vergul bilan ajratilgan qiymatlar:

```
import csv

# Yozish:
with open("talabalar.csv", "w", newline="", encoding="utf-8") as f:
    yozuvchi = csv.writer(f)
    yozuvchi.writerow(["ism", "yosh", "ball"])      # sarlavha qatori
    yozuvchi.writerow(["Aziz", 20, 85])
    yozuvchi.writerow(["Malika", 22, 90])

# O'qish:
with open("talabalar.csv", "r", encoding="utf-8") as f:
    oquvchi = csv.reader(f)
    for qator in oquvchi:
        print(qator)      # ['ism', 'yosh', 'ball'], keyin ['Aziz', '20', '85'], ...
```

Kalit bilan o'qish (DictReader) — har qatorni lug'at sifatida:

```
import csv
with open("talabalar.csv", "r", encoding="utf-8") as f:
    oquvchi = csv.DictReader(f)
```



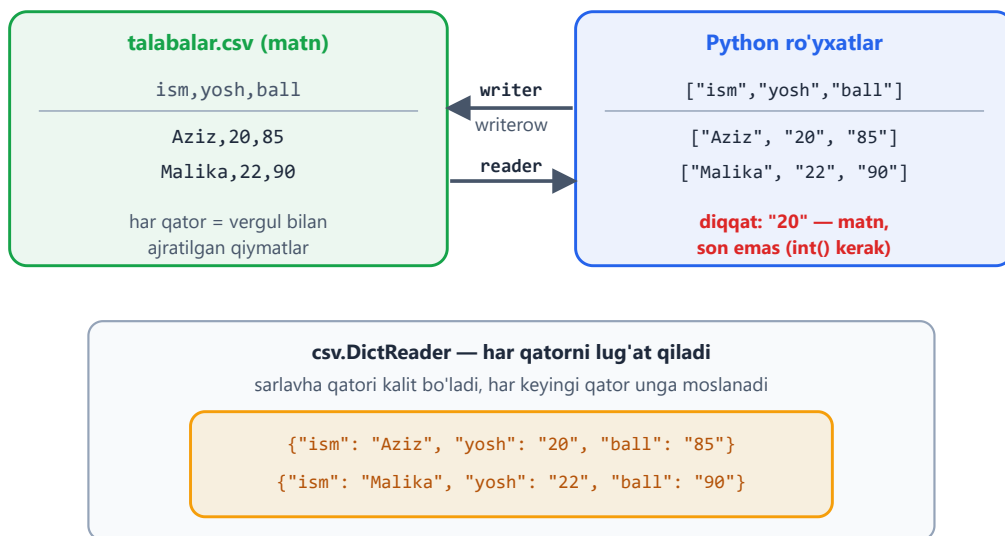
```
for qator in oquvchi:
    print(qator["ism"], qator["ball"])    # Aziz 85, Malika 90
```

`newline=""` — CSV yozishda bu parametрни qo'sh (ba'zi tizimlarda qo'shimcha bo'sh qatorlar paydo bo'lmasligi uchun). CSV'dan o'qilgan sonlar **matn** ko'rinishida keladi (`"20"`), kerak bo'lsa `int()` bilan aylantir.

Quyidagi diagramma CSV jadval va Python qatorlari orasidagi moslikni ko'rsatadi (`DictReader` har qatorni lug'atga aylantirishi bilan).

CSV jadval <=> Python qatorlari

writer ro'yxatlarni qatorga yozadi; reader qatorlarni o'qiydi (sonlar ham matn keladi)



Masalalar (20 ta)

Bu masalalar fayllar bilan ishlaydi — har birini ishga tushirib, yaratilgan fayllarni tekshir.

Oson (1–7):

1. `salom.txt` fayliga "Salom, dunyo!" deb yoz (`with open`, `"w"`, `utf-8`).
2. Shu faylni o'qib, mazmunini ekranga chiqar.
3. Faylga uchta qator yoz (har biri yangi qatorda, `\n` bilan).
4. Faylni qator-qator o'qib, har qatorni `strip()` bilan chiqar.
5. `"a"` rejimida faylga yana bir qator **qo'sh** (eskisi o'chmasin).
6. `pathlib.Path` bilan `salom.txt` faylining nomini, kengaytmasini va mavjudligini chiqar.

7. `Path.write_text` bilan faylga matn yozib, `read_text` bilan qayta o'qi.

O'rta (8–14):

1. Bir lug'atni (`ism`, `yosh`) `json.dump` bilan faylga saqla (`ensure_ascii=False`, `indent=2`).
2. Shu JSON faylni `json.load` bilan o'qib, qiymatlarini chiqar.
3. Ro'yxatdagi sonlarni faylga yoz (har biri alohida qatorda), keyin o'qib yig'indisini hisobla.
4. Foydalanuvchidan bir nechta ism so'rab (sikl bilan), ularni faylga yoz.
5. CSV faylga 3 ta talaba (ism, yosh, ball) yoz (`csv.writer`).
6. Shu CSV faylni `csv.DictReader` bilan o'qib, har talabaning ism va ballini chiqar.
7. JSON faylga sonlar ro'yxatini saqla, keyin o'qib eng kattasini top.

Murakkab (15–20):

1. Oddiy "eslatmalar" dasturi: foydalanuvchi yozgan eslatmalarni faylga qo'shib boradi (`"a"` rejimi), har safar yangi qator.
2. Lug'atlar ro'yxatini (talabalar) JSON faylga saqla, qayta yuklab, ballari 80 dan yuqorilarni chiqar.
3. CSV faylni o'qib, barcha ballarning o'rtachasini hisobla (sonlarni `int` ga aylantirishni unutma).
4. Oddiy "telefon kitobchasi": ma'lumotni JSON faylda saqla. Dastur ishga tushganda yuklasin, yangi kontakt qo'shilganda saqlasin (qo'shish/ko'rish menyusi bilan).
5. Bir matn faylini o'qib, undagi qatorlar sonini, so'zlar sonini va belgilar sonini hisobla.
6. Ma'lumotni CSV'dan o'qib, JSON formatiga o'tkazib saqlovchi dastur yoz (har CSV qatori — JSON'da bitta lug'at bo'lsin).

 Yechimlar



```
# 1
with open("salom.txt", "w", encoding="utf-8") as f:
    f.write("Salom, dunyo!")

# 2
with open("salom.txt", "r", encoding="utf-8") as f:
    print(f.read())

# 3
with open("uch.txt", "w", encoding="utf-8") as f:
    f.write("Birinchi\n")
    f.write("Ikkinchi\n")
    f.write("Uchinchi\n")

# 4
with open("uch.txt", "r", encoding="utf-8") as f:
    for qator in f:
        print(qator.strip())

# 5
with open("uch.txt", "a", encoding="utf-8") as f:
    f.write("Qo'shilgan qator\n")

# 6
from pathlib import Path
yol = Path("salom.txt")
print(yol.name, yol.suffix, yol.exists()) # salom.txt .txt True

# 7
from pathlib import Path
Path("test.txt").write_text("Salom!", encoding="utf-8")
print(Path("test.txt").read_text(encoding="utf-8")) # Salom!

# 8
import json
malumot = {"ism": "Aziz", "yosh": 20}
with open("m.json", "w", encoding="utf-8") as f:
    json.dump(malumot, f, ensure_ascii=False, indent=2)

# 9
import json
with open("m.json", "r", encoding="utf-8") as f:
    d = json.load(f)
print(d["ism"], d["yosh"]) # Aziz 20

# 10
sonlar = [10, 20, 30]
with open("sonlar.txt", "w", encoding="utf-8") as f:
    for s in sonlar:
        f.write(f"{s}\n")
with open("sonlar.txt", "r", encoding="utf-8") as f:
    jami = sum(int(q) for q in f)
print(jami) # 60

# 11
with open("ismlar.txt", "w", encoding="utf-8") as f:
    for _ in range(3):
```

```

        ism = input("Ism: ")
        f.write(ism + "\n")

# 12
import csv
with open("talabalar.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["ism", "yosh", "ball"])
    w.writerow(["Aziz", 20, 85])
    w.writerow(["Malika", 22, 90])
    w.writerow(["Bobur", 21, 78])

# 13
import csv
with open("talabalar.csv", "r", encoding="utf-8") as f:
    for q in csv.DictReader(f):
        print(q["ism"], q["ball"])

# 14
import json
with open("nums.json", "w", encoding="utf-8") as f:
    json.dump([5, 12, 8, 20, 3], f)
with open("nums.json", "r", encoding="utf-8") as f:
    nums = json.load(f)
print(max(nums))          # 20

# 15
while True:
    eslatma = input("Eslatma ('stop' to'xtatadi): ")
    if eslatma == "stop":
        break
    with open("eslatmalar.txt", "a", encoding="utf-8") as f:
        f.write(eslatma + "\n")

# 16
import json
talabalar = [
    {"ism": "Aziz", "ball": 85},
    {"ism": "Malika", "ball": 92},
    {"ism": "Bobur", "ball": 70},
]
with open("t.json", "w", encoding="utf-8") as f:
    json.dump(talabalar, f, ensure_ascii=False, indent=2)
with open("t.json", "r", encoding="utf-8") as f:
    yuklangan = json.load(f)
for t in yuklangan:
    if t["ball"] > 80:
        print(t["ism"])          # Aziz, Malika

# 17
import csv
ballar = []
with open("talabalar.csv", "r", encoding="utf-8") as f:
    for q in csv.DictReader(f):
        ballar.append(int(q["ball"]))
print(sum(ballar) / len(ballar))

# 18
import json
from pathlib import Path
FAYL = "kontaktlar.json"

```

```

def yukla():
    if Path(FAYL).exists():
        return json.loads(Path(FAYL).read_text(encoding="utf-8"))
    return {}
def saqla(k):
    Path(FAYL).write_text(json.dumps(k, ensure_ascii=False, indent=2),
encoding="utf-8")
kontaktlar = yukla()
while True:
    amal = input("qosh / korish / chiqish: ")
    if amal == "chiqish":
        break
    elif amal == "qosh":
        ism = input("Ism: ")
        kontaktlar[ism] = input("Raqam: ")
        saqla(kontaktlar)
    elif amal == "korish":
        print(kontaktlar)

# 19
with open("uch.txt", "r", encoding="utf-8") as f:
    matn = f.read()
print("Qatorlar:", len(matn.splitlines()))
print("So'zlar:", len(matn.split()))
print("Belgilar:", len(matn))

# 20
import csv, json
qatorlar = []
with open("talabalar.csv", "r", encoding="utf-8") as f:
    for q in csv.DictReader(f):
        qatorlar.append(q)
with open("talabalar.json", "w", encoding="utf-8") as f:
    json.dump(qatorlar, f, ensure_ascii=False, indent=2)
print("CSV → JSON tayyor")

```

← [Xatolarni boshqarish](#) | [Boshlovchilar README](#) ↑ | [Keyingi: Generator va dekoratorlar](#)

→

09 — Generatorlar va dekoratorlar

Bu modul Python'ning kuchliroq vositalariga kirish. **Generatorlar** — katta yoki cheksiz ketma-ketliklarni xotirani tejab ishlatish usuli. **Dekoratorlar** — funksiyaning xulqini o'zgartirmasdan unga qo'shimcha imkoniyat qo'shish vositasi. Bular boshida biroz murakkab tuyuladi — sekin, misol bilan o'rganamiz.

Bu modulda: generatorlar (`yield`), generator ifodalari, closure (yopilma), va dekoratorlar.

9.1 Generator nima? `yield`

Oddiy funksiya `return` bilan **bitta** qiymat qaytaradi va tugaydi. **Generator** esa `yield` bilan qiymatlarni **birma-bir**, kerak bo'lganda beradi — hammasini birdaniga xotiraga olmaydi.

```
def sanagich(n):
    for i in range(n):
        yield i          # return emas — yield: qiymatni "berib",
                           to'xtab turadi

for son in sanagich(5):
    print(son)           # 0, 1, 2, 3, 4
```

Nega foydali? Tasavvur qil, 1 milliard sonni qayta ishlash kerak. Oddiy ro'yxat hammasini xotiraga oladi (juda ko'p joy egallaydi). Generator esa har safar bittasini beradi — xotira deyarli ishlatilmaydi:

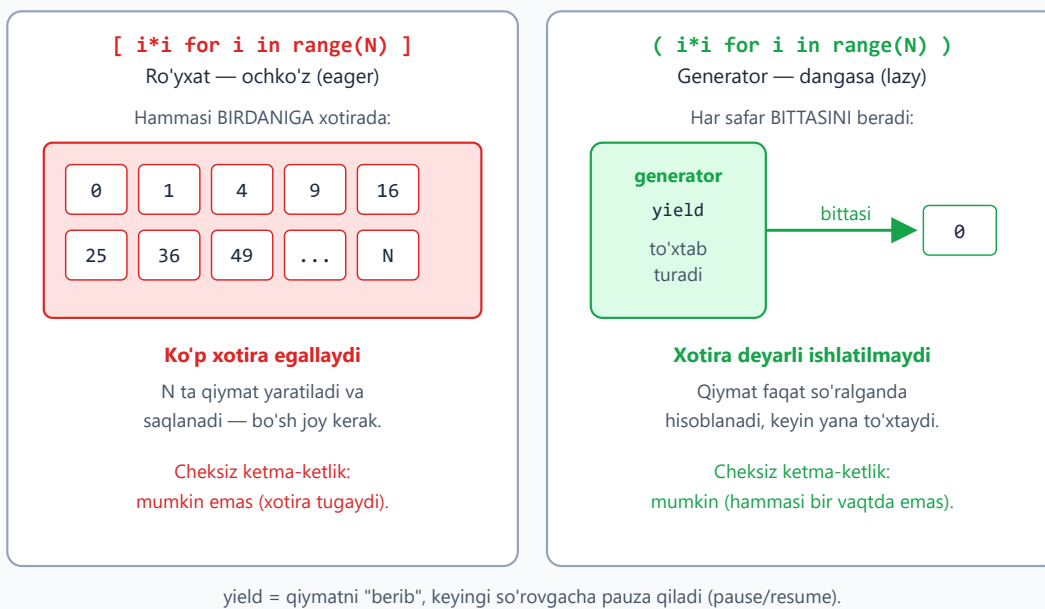
```
# Ro'yxat — hammasini xotiraga oladi:
sonlar = [i * i for i in range(1000000)]  # ko'p xotira

# Generator — bittadan beradi, xotira tejaladi:
sonlar = (i * i for i in range(1000000))  # ( ) — generator ifodasi
for s in sonlar:
    ...                                   # har safar bittasi
```

Eslab qol: generator "dangasa" (lazy) — qiymatni faqat **so'ralganda** hisoblaydi. Cheksiz ketma-ketliklar bilan ham ishlay oladi (chunki hammasini bir vaqtda yaratmaydi).

Quyidagi diagrammada ro'yxat (hammasini birdaniga xotiraga oladi) va generator (`yield` bilan to'xtab-to'xtab, bittadan beradi) yonma-yon taqqoslangan:

Generator vs ro'yxat



Generator ifodasi — list comprehension'ga o'xshaydi, lekin `[ ]` o'rniga `( )`:

```
kvadratlar = (x * x for x in range(5))    # generator
print(sum(kvadratlar))                  # 30 - yig'indi, oraliq
ro'yxatsiz
```

9.2 Closure (yopilma) — funksiya qaytaruvchi funksiya

Funksiya ichida boshqa funksiya yaratib, uni qaytarish mumkin. Ichki funksiya tashqi funksiyaning o'zgaruvchilarini "eslab qoladi" — bu **closure**:

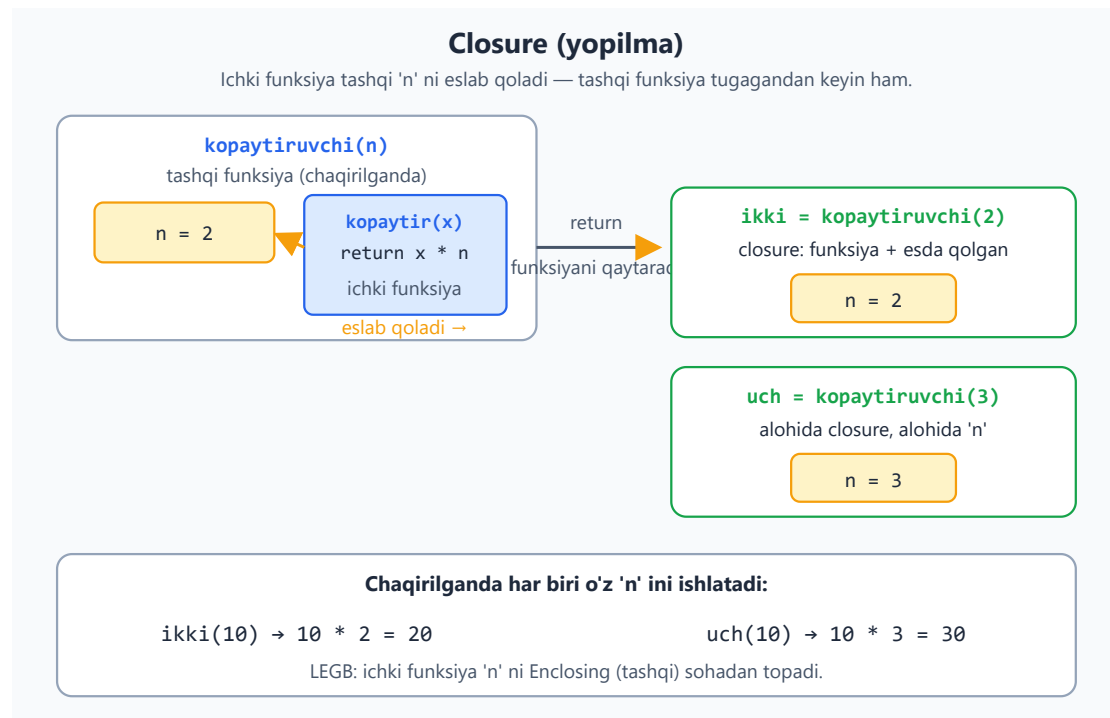
```
def kopaytiruvchi(n):
    def kopaytir(x):
        return x * n    # tashqi 'n' ni eslab qoladi
    return kopaytir     # funksiyaning O'ZINI qaytaramiz
                        (chaqirmasdan)

ikki = kopaytiruvchi(2)    # n=2 bo'lgan funksiya
uch = kopaytiruvchi(3)     # n=3 bo'lgan funksiya

print(ikki(10))           # 20
print(uch(10))            # 30
```

Bu boshida g'alati tuyuladi: funksiya **qiymat** emas, **boshqa funksiya** qaytaradi. Closure dekoratorlarni tushunish uchun zarur — keyingi bo'limga poydevor.

Quyidagi diagrammada ichki funksiya tashqi `n` ni qanday "eslab qolishi" va har bir chaqiruv o'z `n` ini olib yurishi ko'rsatilgan:



9.3 Dekoratorlar — funksiya qo'shimcha imkoniyat qo'shish

Dekorator — bir funksiya "o'rab", unga qo'shimcha xulq qo'shadigan funksiya. Masalan, funksiya ishlashidan oldin/keyin biror narsa qilish (log yozish, vaqt o'lchash) — asl funksiya o'zgartirmasdan.

Avval dekoratorning "ichini" ko'ramiz:

```
def log_qiluvchi(funk):
    def ichki(*args, **kwargs):
        print(f"{funk.__name__} chaqirilmoqda...")
        natija = funk(*args, **kwargs) # asl funksiya
    chaqiramiz
    print(f"{funk.__name__} tugadi")
    return natija
    return ichki

def salomla(ism):
    print(f"Salom, {ism}!")

salomla = log_qiluvchi(salomla) # funksiya "o'radik"
salomla("Aziz")
```



```
# Chiqish:
#   salomla chaqirilmoqda...
#   Salom, Aziz!
#   salomla tugadi
```

@ belgisi — xuddi shu ishning qisqa yozuvi:

```
def log_qiluvchi(funk):
    def ichki(*args, **kwargs):
        print(f"{funk.__name__} chaqirilmoqda...")
        natija = funk(*args, **kwargs)
        print(f"{funk.__name__} tugadi")
        return natija
    return ichki

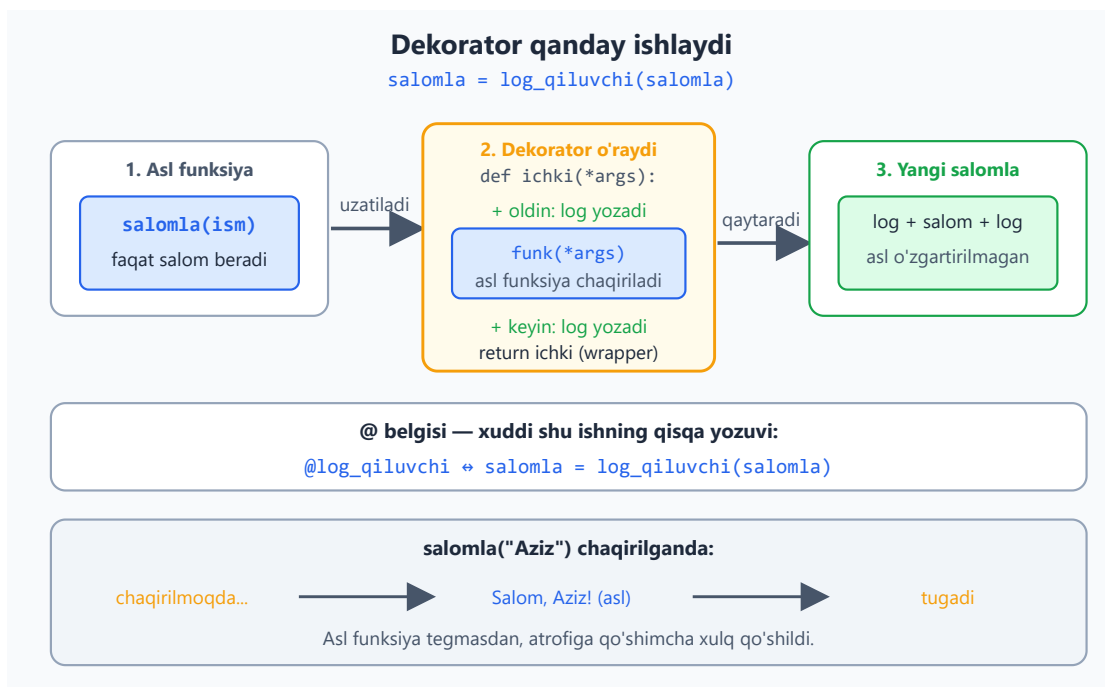
@log_qiluvchi
def salomla(ism):
    print(f"Salom, {ism}!")

salomla("Aziz")
```

= salomla = log_qiluvchi(salomla)

avtomatik o'ralgan holda ishlaydi

Quyidagi diagrammada dekorator asl funksiyani qanday o'rab, wrapper (ichki funksiya) orqali unga qo'shimcha xulq qo'shishi bosqichma-bosqich ko'rsatilgan:



args, **kwargs nima?** Bular dekorator har qanday funksiyani (qanday argument olsa ham) o'rashi uchun. ***args** — barcha oddiy argumentlar, *kwargs** — barcha nomli argumentlar. Hozircha "har qanday argumentni qabul qiladi" deb tushun.

Amaliy misol — vaqt o'lchovchi dekorator:

```
import time

def vaqt_olchash(funk):
    def ichki(*args, **kwargs):
        boshlanish = time.perf_counter()
        natija = funk(*args, **kwargs)
        davomiylik = time.perf_counter() - boshlanish
        print(f"{funk.__name__}: {davomiylik:.4f} soniya")
        return natija
    return ichki

@vaqt_olchash
def sekin_ish():
    time.sleep(1)

sekin_ish()      # sekin_ish: 1.0001 soniya
```

9.4 Tayyor foydali dekoratorlar

Python o'zida foydali dekoratorlar bilan keladi. Eng mashhuri — `lru_cache` (natijani eslab qolib, takror hisoblamaydi):

```
from functools import lru_cache

@lru_cache
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(50))      # tez ishlaydi – keshlangani uchun
# lru_cache'siz fib(50) juda sekin bo'lardi (bir xil hisob takrorlanardi)
```

Hozircha dekoratorlarni faqat "ishlatish" darajasida bilsang yetadi (`@lru_cache` kabi tayyorlarni). O'z dekoratoringni yozish — yuqorida ko'rgan namuna bo'yicha, vaqt o'tib mashq qilasan.

Masalalar (20 ta)

Bu masalalar oldingi modullarga (ayniqsa funksiyalar va sikllarga) tayanadi.

Oson (1–7):

1. `sanagich(n)` generatorini yoz: 0 dan n gacha sonlarni `yield` qilsin. `for` bilan chiqar.
2. `list()` bilan generator natijasini ro'yxatga aylantir: `list(sanagich(5))`.
3. Generator ifodasi `(x*2 for x in range(5))` yarat va `for` bilan chiqar.
4. `sum()` ni generator ifodasi bilan ishlat: 1 dan 100 gacha sonlar yig'indisi.
5. `kvadratlar(n)` generatorini yoz: 0 dan n gacha sonlarning kvadratlarini bersin.
6. `kopaytiruvchi(n)` closure yoz (n ga ko'paytiruvchi funksiya qaytarsin). `ikki = kopaytiruvchi(2)` qilib ishlat.
7. `@lru_cache` ishlatib rekursiv funksiya yoz (masalan faktorial yoki Fibonachchi).

O'rta (8–14):

1. Juft sonlarni beruvchi generator yoz: `juftlar(n)` — 0 dan n gacha faqat juftlarni `yield` qilsin.
2. Closure yoz: `qoshuvchi(n)` — n ni qo'shuvchi funksiya qaytarsin (`bash = qoshuvchi(5)`, `bash(10) → 15`).
3. Oddiy dekorator yoz: `@salom` — funksiya chaqirilishidan oldin "Salom!" deb chiqarsin.
4. Funksiya nomini chop etuvchi dekorator yoz (`funk.__name__` ishlat).
5. Vaqt o'lchovchi dekorator yoz (`time.perf_counter`), uni biror funksiya qo'lla.
6. Generator yoz: berilgan ro'yxatdan faqat musbat sonlarni `yield` qilsin.
7. `@lru_cache` bilan va usiz Fibonachchi yozib, tezligini taqqosla (`time` bilan).

Murakkab (15–20):

1. Cheksiz generator yoz: `sanoq(boshlanish)` — to'xtamasdan sonlarni `yield` qilsin. `for` + `break` bilan birinchi 5 tasini ol.
 2. Dekorator yoz: funksiya nechi marta chaqirilganini sanasin va har chaqiruvda chop etsin.
 3. Natijani 2 ga ko'paytiruvchi dekorator yoz: o'ralgan funksiya nima qaytarsa, uni 2 ga ko'paytirib qaytarsin.
 4. Generator yoz: matndagi har bir so'zni bittadan `yield` qilsin (`split` + `yield`).
 5. Dekorator yoz: funksiya xato bersa, uni ushlab "Xato yuz berdi" deb chiqarsin (funksiya qulamasin) — `try/except` ni dekorator ichida ishlat.
 6. "Faqat musbat" dekoratori yoz: funksiya uzatilgan barcha sonlar musbat bo'lishini tekshirsin; biror salbiy bo'lsa `ValueError` chaqirsin, aks holda funkisiyani normal ishlatsin.
-

☒ Yechimlar



```
# 1
def sanagich(n):
    for i in range(n):
        yield i
for s in sanagich(5):
    print(s)                # 0,1,2,3,4

# 2
print(list(sanagich(5)))    # [0, 1, 2, 3, 4]

# 3
for x in (x * 2 for x in range(5)):
    print(x)                # 0,2,4,6,8

# 4
print(sum(x for x in range(1, 101)))    # 5050

# 5
def kvadratlar(n):
    for i in range(n):
        yield i * i
print(list(kvadratlar(5)))    # [0, 1, 4, 9, 16]

# 6
def kopaytiruvchi(n):
    def kopaytir(x):
        return x * n
    return kopaytir
ikki = kopaytiruvchi(2)
print(ikki(10))              # 20

# 7
from functools import lru_cache
@lru_cache
def fakt(n):
    return 1 if n <= 1 else n * fakt(n - 1)
print(fakt(10))              # 3628800

# 8
def juftlar(n):
    for i in range(n):
        if i % 2 == 0:
            yield i
print(list(juftlar(10)))     # [0, 2, 4, 6, 8]

# 9
def qoshuvchi(n):
    def qosh(x):
        return x + n
    return qosh
besh = qoshuvchi(5)
print(besh(10))              # 15

# 10
def salom(funk):
    def ichki(*args, **kwargs):
        print("Salom!")
```

```

        return funk(*args, **kwargs)
    return ichki
@salom
def ish():
    print("Ish bajarildi")
ish() # Salom! / Ish bajarildi

# 11
def nom_chop(funk):
    def ichki(*args, **kwargs):
        print(f"Chaqirilmoqda: {funk.__name__}")
        return funk(*args, **kwargs)
    return ichki
@nom_chop
def hisob():
    return 42
print(hisob())

# 12
import time
def vaqt_olchash(funk):
    def ichki(*args, **kwargs):
        b = time.perf_counter()
        n = funk(*args, **kwargs)
        print(f"{funk.__name__}: {time.perf_counter() - b:.4f}s")
        return n
    return ichki
@vaqt_olchash
def sekin():
    time.sleep(0.5)
sekin()

# 13
def musbatlar(royxat):
    for s in royxat:
        if s > 0:
            yield s
print(list(musbatlar([-2, 3, -1, 5, 0, 8]))) # [3, 5, 8]

# 14
import time
from functools import lru_cache
def fib1(n):
    return n if n < 2 else fib1(n - 1) + fib1(n - 2)
@lru_cache
def fib2(n):
    return n if n < 2 else fib2(n - 1) + fib2(n - 2)
b = time.perf_counter(); fib1(30); print("keshsiz:",
round(time.perf_counter() - b, 3))
b = time.perf_counter(); fib2(30); print("keshli:",
round(time.perf_counter() - b, 6))

# 15
def sanoq(boshlanish):
    son = boshlanish
    while True:
        yield son
        son += 1
natija = []
for s in sanoq(10):
    if len(natija) >= 5:

```

```

        break
    natija.append(s)
print(natija)                # [10, 11, 12, 13, 14]

# 16
def sanagich_dek(funk):
    funk.soni = 0
    def ichki(*args, **kwargs):
        funk.soni += 1
        print(f"{funk.soni}-marta chaqirildi")
        return funk(*args, **kwargs)
    return ichki
@sanagich_dek
def salom():
    pass
salom(); salom()            # 1-marta / 2-marta

# 17
def ikki_barobar(funk):
    def ichki(*args, **kwargs):
        return funk(*args, **kwargs) * 2
    return ichki
@ikki_barobar
def qosh(a, b):
    return a + b
print(qosh(3, 4))           # 14 ((3+4)*2)

# 18
def sozlar(matn):
    for soz in matn.split():
        yield soz
print(list(sozlar("Python juda zor"))) # ['Python', 'juda', 'zor']

# 19
def xatoni_ushla(funk):
    def ichki(*args, **kwargs):
        try:
            return funk(*args, **kwargs)
        except Exception:
            print("Xato yuz berdi")
    return ichki
@xatoni_ushla
def bol(a, b):
    return a / b
print(bol(10, 2))           # 5.0
bol(10, 0)                  # Xato yuz berdi

# 20
def faqat_musbat(funk):
    def ichki(*args, **kwargs):
        for a in args:
            if isinstance(a, (int, float)) and a < 0:
                raise ValueError("Faqt musbat sonlar")
        return funk(*args, **kwargs)
    return ichki
@faqat_musbat
def qosh(a, b):
    return a + b
print(qosh(2, 3))           # 5
try:
    qosh(2, -1)

```

```
except ValueError as e:  
    print(e)                # Faqat musbat sonlar
```

[← Fayllar, JSON, CSV](#) | [Boshlovchilar README ↑](#) | [Keyingi: Tip ko'rsatmalari →](#)

10 — Tip ko'rsatmalari va `map` / `filter`

Python o'zgaruvchi turini oldindan e'lon qilishni talab qilmaydi. Lekin **tip ko'rsatmalari** (type hints) yozish — kodni o'qishni osonlashtiradi, muharrir (VS Code va h.k.) yordamini yaxshilaydi va xatolarni erta topishga ko'maklashadi. Bu modulda type hints'ni va funksional uslubdagi `map` / `filter` ni o'rganasan.

Bu modulda: tip ko'rsatmalari (`int`, `str`, `list[int]` ...), `Optional` / `| None`, va `map` / `filter`.

10.1 Tip ko'rsatmalari — asoslari

O'zgaruvchi yoki funksiya parametriga uning turini yozib qo'yasan. Bu Python'ni o'zgartirmaydi — faqat "bu yerga shu tur kutilmoqda" deb belgilaydi:

```
ism: str = "Aziz"
yosh: int = 25
narx: float = 19.99
faolmi: bool = True

def salomla(ism: str) -> str:      # ism - str, qaytaradi - str
    return f"Salom, {ism}"
```

`-> str` qismi — funksiya **nima qaytarishini** bildiradi. Hech narsa qaytarmasa `-> None`:

```
def chiqar(matn: str) -> None:    # hech narsa qaytarmaydi
    print(matn)
```

Juda muhim: tip ko'rsatmalari Python tomonidan **tekshirilmaydi**. Quyidagi kod xato bermaydi:

```
def qosh(a: int, b: int) -> int:
    return a + b
qosh("salom", "dunyo")    # ishlaydi! "salomdunyo" qaytaradi
```

Ular faqat **hujjat** va **muharrir/vosita** uchun. Lekin baribir doim yoz — kod ancha tushunarli bo'ladi va muharrir xatolarni ko'rsatadi.

Quyidagi diagramma annotatsiyaning har bir qismi nimani bildirishini va Python uni tekshirmasligini ko'rsatadi:

Tip ko'rsatmasi nimani bildiradi

annotatsiya = "bu yerga shu tur kutilmoqda" degan belgi

```
def salomla(ism: str) -> str:
```

: str

parametr turi — matn kutiladi

-> str

funksiya matn qaytaradi

Python annotatsiyani TEKSHIRMAYDI

salomla(123) # xato bermaydi — ishlab ketadi

Foydasi: hujjat + muharrir yordami + xatolarni erta sezish

10.2 To'plamlar uchun tip ko'rsatmalari

Ro'yxat, lug'at va boshqalar uchun ichidagi turni ham ko'rsatasan:

```
ismlar: list[str] = ["Ali", "Vali"]          # str'lardan iborat
ro'yxat
sonlar: list[int] = [1, 2, 3]                # int'lardan
narxlar: dict[str, float] = {"olma": 1.5}    # kalit str, qiymat float
koordinata: tuple[int, int] = (10, 20)       # ikkita int

def jami(sonlar: list[int]) -> int:
    return sum(sonlar)
```

10.3 Optional / | None — "qiymat bo'lmasligi ham mumkin"

Ba'zan funksiya qiymat qaytaradi yoki hech narsa (None) qaytaradi. Buni | None bilan ko'rsatasan:

```
def topish(ismlar: list[str], qidiruv: str) -> int | None:
    for i, ism in enumerate(ismlar):
        if ism == qidiruv:
            return i          # topildi — indeks
    return None              # topilmadi — None
```

```
# `enumerate` – ro'yxat bo'ylab indeks bilan birga aylanish:  
for indeks, qiymat in enumerate(["a", "b", "c"]):  
    print(indeks, qiymat)      # 0 a, 1 b, 2 c
```

`int | None` — "int yoki None" degani. Bu "qiymat bor yoki yo'q" holatlarni aniq ifodalaydi. (Eski kodda `Optional[int]` ko'rishing mumkin — bir xil ma'no.)

10.4 `map` — har elementga funksiya qo'llash

`map` ro'yxatning har bir elementiga funksiyani qo'llaydi:

```
sonlar = [1, 2, 3, 4]  
  
# Har birini kvadratga:  
kvadratlar = list(map(lambda x: x ** 2, sonlar))  
print(kvadratlar)      # [1, 4, 9, 16]  
  
# Tayyor funksiya bilan – har birini matnga aylantirish:  
matnlar = list(map(str, sonlar))  
print(matnlar)         # ['1', '2', '3', '4']
```

`map` har elementga bir xil funksiyani qo'llaydi va element soni o'zgarmaydi — quyidagi diagrammada bu ko'rinadi:



lambda nima? `lambda` — nomsiz, qisqa funksiya. `lambda x: x ** 2` = "x ni olib, kvadratini qaytaradi". Faqat bitta ifoda uchun ishlatiladi. Ko'p hollarda comprehension o'qilishi osonroq (pastga qara).

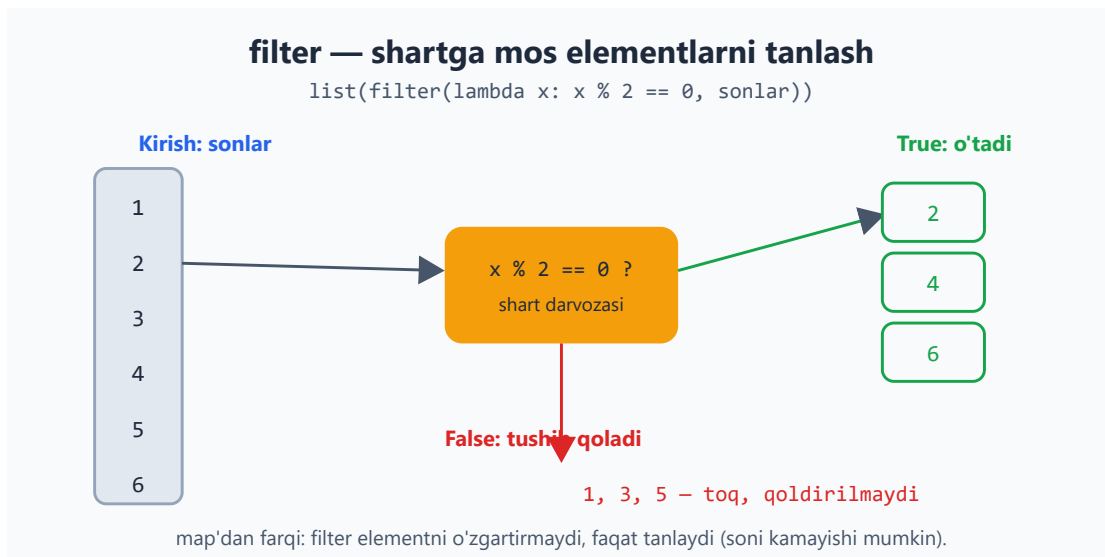
10.5 filter — shartga mos elementlarni tanlash

`filter` faqat shart `True` bo'lgan elementlarni qoldiradi:

```
sonlar = [1, 2, 3, 4, 5, 6]

juftlar = list(filter(lambda x: x % 2 == 0, sonlar))
print(juftlar)          # [2, 4, 6]
```

Diagrammada shart `True` bo'lgan elementlar o'tishi, qolganlari tushib qolishi ko'rsatilgan:



Maslahat: ko'p hollarda `map` / `filter` o'rniga **comprehension** (3-moduldan) o'qilishi osonroq:

```
kvadratlar = [x ** 2 for x in sonlar]          # map o'rniga
juftlar = [x for x in sonlar if x % 2 == 0]    # filter o'rniga
```

`map` / `filter` ni asosan tayyor funksiya bor bo'lganda ishlat (`map(str, sonlar)`). Comprehension'ni esa odat qil — u Python'da eng keng tarqalgan.

Comprehension qanday o'qilishini quyidagi oqim diagrammasi yordamida tushunish oson — aylanish, ixtiyoriy shart va ifoda ketma-ketligi:

List comprehension oqimi

```
[ x ** 2 for x in sonlar if x % 2 == 0 ]
```

ifoda

aylanish

shart (ixtiyoriy)

1) for x in sonlar

ro'yxatning har elementini navbat bilan x ga oladi

2) if x % 2 == 0 ?

shart False bo'lsa — bu element o'tkazib yuboriladi

3) x ** 2 hisoblanadi

natija yangi ro'yxatga qo'shiladi

yangi ro'yxat = [4, 16, 36, ...]

10.6 Ilg'or mavzular (hozircha qisqacha)

Tip ko'rsatmalari bo'yicha yana ko'p narsa bor — keyinroq o'rganasan: - `mypy` — kodni ishga tushirmasdan tip xatolarini topuvchi vosita. - `Generic`, `Protocol`, `TypedDict` — murakkab loyihalar uchun kuchli tip vositalari. - `functools` moduli — `reduce`, `partial` kabi funksional vositalar.

Hozircha asosiy ko'rsatmalar (`int`, `str`, `list[int]`, `| None`) va `map` / `filter` yetadi.



Masalalar (20 ta)

Bu masalalar oldingi modullarga tayanadi.

Oson (1–7):

1. `kop(a: int, b: int) -> int` funksiyasini tip ko'rsatmalari bilan yoz, ko'paytirsin.
2. `salomla(ism: str) -> str` yoz: "Salom, {ism}" qaytarsin.
3. `ortacha(sonlar: list[float]) -> float` yoz: o'rtachani qaytarsin.
4. `map` bilan `["1", "2", "3"]` ni `[1, 2, 3]` ga aylantir (`int` ishlat).
5. `filter` bilan `range(20)` dan 3 ga bo'linadiganlarini ol.
6. 4-masalani comprehension bilan ham yoz, ikkalasini taqqosla.
7. `chiqar(matn: str) -> None` yoz: matnni chop etsin (hech narsa qaytarmasin).

O'rta (8–14):

1. `list[int]` qabul qiladigan funksiya yoz: ro'yxatning yig'indisini qaytarsin.
2. `topish(ismlar: list[str], qidiruv: str) -> int | None` yoz: indeksni yoki `None` qaytarsin.
3. `enumerate` bilan ro'yxat elementlarini "0: olma", "1: banan" ko'rinishida chiqar.
4. `map` bilan har bir ismni katta harfga aylantir (`str.upper` ishlat).
5. `filter` bilan ismlar ro'yxatidan uzunligi 4 dan katta bo'lganlarini ol.
6. `dict[str, int]` qaytaradigan funksiya yoz: so'zlar ro'yxatidan har so'z uzunligini lug'at qilib qaytarsin.
7. `map` va `lambda` bilan har bir narxga 20% chegirma qo'llab yangi ro'yxat yasa.

Murakkab (15–20):

1. `Callable` ishlat: `qolla(funk, royxat)` yoz — har elementga funksiyani qo'llab yangi ro'yxat qaytarsin (`from typing import Callable`).
2. `filter` + `map` ni birga ishlat: ro'yxatdagi juft sonlarni olib, har birini kvadratga aylantir.
3. Tip ko'rsatmalari bilan to'liq funksiya yoz: `talabalar: list[dict]` qabul qilsin, ballari 60 dan yuqorilarning ismlarini `list[str]` qilib qaytarsin.
4. `int | None` qaytaradigan "xavfsiz bo'lish" funksiyasini yoz: nolga bo'lishda `None` qaytarsin.
5. Comprehension va `map` / `filter` bilan bir xil natijani uch xil usulda yoz (1–100 dan juft sonlarning kvadratlari), uchala bir xil natija berishini tekshir.
6. To'liq tipli kichik dastur: `Talaba` ma'lumotlari (`dict`) ro'yxatini olib, o'rtacha ball, eng yuqori ballli talaba ismi va o'tganlar (60+) sonini qaytaradigan funksiyalar yoz.



Yechimlar



```
# 1
def kop(a: int, b: int) -> int:
    return a * b
print(kop(3, 4))          # 12

# 2
def salomla(ism: str) -> str:
    return f"Salom, {ism}"

# 3
def ortacha(sonlar: list[float]) -> float:
    return sum(sonlar) / len(sonlar)

# 4
print(list(map(int, ["1", "2", "3"])))    # [1, 2, 3]

# 5
print(list(filter(lambda x: x % 3 == 0, range(20))))    # [0, 3, 6, 9, 12, 15, 18]

# 6
print([int(x) for x in ["1", "2", "3"]])    # comprehension - o'qilishi
osonroq

# 7
def chiqar(matn: str) -> None:
    print(matn)

# 8
def jami(sonlar: list[int]) -> int:
    return sum(sonlar)
print(jami([10, 20, 30]))    # 60

# 9
def topish(ismlar: list[str], qidiruv: str) -> int | None:
    for i, ism in enumerate(ismlar):
        if ism == qidiruv:
            return i
    return None
print(topish(["a", "b", "c", "b"]))    # 1

# 10
for i, x in enumerate(["olma", "banan"]):
    print(f"{i}: {x}")

# 11
print(list(map(str.upper, ["ali", "vali"])))    # ['ALI', 'VALI']

# 12
ismlar = ["Ali", "Malika", "Bobur", "Gul"]
print(list(filter(lambda s: len(s) > 4, ismlar)))    # ['Malika', 'Bobur']

# 13
def uzunliklar(sozlar: list[str]) -> dict[str, int]:
    return {s: len(s) for s in sozlar}
print(uzunliklar(["olma", "non"]))    # {'olma': 4, 'non': 3}

# 14
```

```

narxlar = [100, 200, 300]
print(list(map(lambda n: n * 0.8, narxlar))) # [80.0, 160.0, 240.0]

# 15
from typing import Callable
def qolla(funk: Callable[[int], int], royxat: list[int]) -> list[int]:
    return [funk(x) for x in royxat]
print(qolla(lambda x: x + 1, [1, 2, 3])) # [2, 3, 4]

# 16
sonlar = [1, 2, 3, 4, 5, 6]
natija = list(map(lambda x: x ** 2, filter(lambda x: x % 2 == 0, sonlar)))
print(natija) # [4, 16, 36]

# 17
def otganlar(talabalar: list[dict]) -> list[str]:
    return [t["ism"] for t in talabalar if t["ball"] >= 60]
data = [{"ism": "Aziz", "ball": 85}, {"ism": "Gul", "ball": 50}]
print(otganlar(data)) # ['Aziz']

# 18
def xavfsiz_bol(a: int, b: int) -> float | None:
    if b == 0:
        return None
    return a / b
print(xavfsiz_bol(10, 2), xavfsiz_bol(10, 0)) # 5.0 None

# 19
a = [x ** 2 for x in range(1, 101) if x % 2 == 0]
b = list(map(lambda x: x ** 2, filter(lambda x: x % 2 == 0, range(1, 101))))
c = []
for x in range(1, 101):
    if x % 2 == 0:
        c.append(x ** 2)
print(a == b == c) # True

# 20
def ortacha_ball(talabalar: list[dict]) -> float:
    return sum(t["ball"] for t in talabalar) / len(talabalar)
def eng_yuqori(talabalar: list[dict]) -> str:
    return max(talabalar, key=lambda t: t["ball"])["ism"]
def otganlar_soni(talabalar: list[dict]) -> int:
    return len([t for t in talabalar if t["ball"] >= 60])
data = [{"ism": "Aziz", "ball": 85}, {"ism": "Gul", "ball": 55}, {"ism": "Bobur", "ball": 72}]
print(ortacha_ball(data)) # 70.66...
print(eng_yuqori(data)) # Aziz
print(otganlar_soni(data)) # 2

```


11 — Standart kutubxona ("hammasi ichida")

Python "hammasi ichida" (batteries included) tamoyili bilan keladi: ko'p foydali vositalar o'rnatmasdan, til bilan birga keladi. Bu modulda eng ko'p ishlatiladigan standart modullar bilan tanishasan — ular kundalik ishingni ancha osonlashtiradi.

Bu modulda: `collections` (`Counter`, `defaultdict`), `datetime` (sana/vaqt), `random` (tasodif), va `math` (matematika).

11.1 `collections` — qulay to'plamlar

Counter — narsalarni sanash uchun (3-moduldagi qo'lda sanashning tayyor varianti):

```
from collections import Counter

ovozlar = ["ha", "yo'q", "ha", "ha", "yo'q"]
hisob = Counter(ovozlar)
print(hisob)                # Counter({'ha': 3, 'yo'q': 2})
print(hisob["ha"])          # 3
print(hisob.most_common(1)) # [('ha', 3)] - eng ko'p uchragani

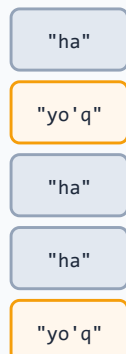
# Matndagi harflarni sanash:
print(Counter("mississippi")) # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
```

`Counter` ro'yxat bo'ylab yurib, har bir elementni avtomatik sanaydi va natijani "element → nechta marta" lug'ati ko'rinishida qaytaradi:

Counter — elementlarni sanash

Counter(["ha", "yo'q", "ha", "ha", "yo'q"])

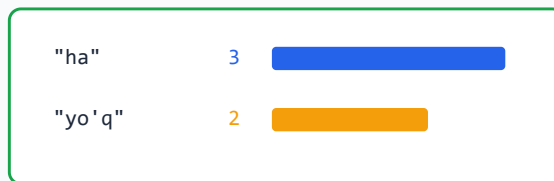
Kirish ro'yxati



sanaydi

har xil qiymatni

Natija (chastota)



most_common(1) -> [('ha', 3)]

Counter — bu dict: kalit = element, qiymat = nechta marta uchragani.

defaultdict — yo'q kalitga avtomatik standart qiymat beradi (`KeyError` bo'lmaydi):

```
from collections import defaultdict

guruh = defaultdict(list)      # yo'q kalit avtomatik bo'sh ro'yxat bo'ladi
for ism, sinf in [("Ali", "A"), ("Vali", "B"), ("Gul", "A")]:
    guruh[sinf].append(ism)    # KeyError yo'q — kalit avtomatik yaratiladi

print(dict(guruh))              # {'A': ['Ali', 'Gul'], 'B': ['Vali']}
```

Oddiy `dict` bilan buni `if kalit not in lugat: lugat[kalit] = []` qilib yozarding — `defaultdict` shuni avtomatlashtiradi.

11.2 `datetime` — sana va vaqt

```
from datetime import datetime, date, timedelta

hozir = datetime.now()          # hozirgi sana va vaqt
bugun = date.today()           # bugungi sana

print(hozir.year, hozir.month, hozir.day)  # 2026 6 9
print(bugun)                    # 2026-06-09

# Chiroyli formatlash (strftime):
print(hozir.strftime("%d.%m.%Y"))          # 09.06.2026
print(hozir.strftime("%H:%M"))             # 14:30

# Matndan sana o'qish (strptime):
```

```

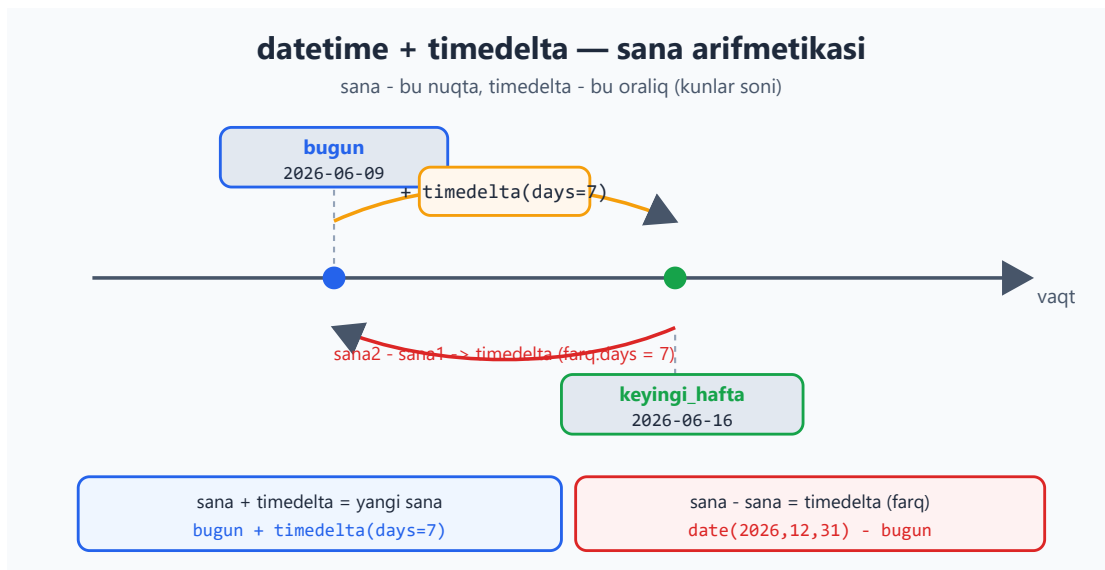
sana = datetime.strptime("2026-01-15", "%Y-%m-%d")

# timedelta — vaqt qo'shish/ayirish:
keyingi_hafta = bugun + timedelta(days=7)
print(keyingi_hafta)          # 7 kun keyin

farq = date(2026, 12, 31) - bugun
print(farq.days, "kun qoldi")

```

Sana — bu vaqt o'qidagi nuqta, `timedelta` esa ikki nuqta orasidagi oraliq: sanaga `timedelta` qo'shsang yangi sana chiqadi, ikki sanani ayirsang `timedelta` (farq) chiqadi:



`strptime` (sanani matnga) va `strftime` (matnni sanaga) — kodlar: `%Y`=yil, `%m`=oy, `%d`=kun, `%H`=soat, `%M`=daqiqqa. Hammasini yodlash shart emas, kerak bo'lganda qaraysan.

11.3 random — tasodifiy sonlar

```

import random

print(random.randint(1, 6))          # 1..6 oraliq'ida (zar)
print(random.random())                # 0.0-1.0 oraliq'ida kasr
print(random.choice(["olma", "banan", "uzum"])) # tasodifiy bittasi

karta = [1, 2, 3, 4, 5]
random.shuffle(karta)                # ro'yxatni joyida
aralashtiradi
print(karta)

```

```
# Ro'yxatdan bir nechta tasodifiy (takrorsiz):  
print(random.sample(range(1, 50), 5)) # 5 ta tasodifiy son
```

Diqqat: `random` o'yinlar va simulyatsiya uchun. Parol, token, xavfsizlik uchun **ishlatma** — uning natijasi bashorat qilinadi. Xavfsizlik kerak bo'lganda `secrets` moduli bor.

11.4 `math` — matematik funksiyalar

```
import math  
  
print(math.sqrt(16))          # 4.0    - kvadrat ildiz  
print(math.ceil(4.1))        # 5      - yuqoriga yaxlitlash  
print(math.floor(4.9))       # 4      - pastga yaxlitlash  
print(math.pi)              # 3.14159...  
print(math.factorial(5))     # 120    - faktorial  
print(math.gcd(24, 36))      # 12     - eng katta umumiy bo'luvchi  
print(round(3.14159, 2))     # 3.14   - yaxlitlash (math emas, lekin  
                             foydali)
```

11.5 Boshqa foydali modullar (qisqacha)

Python'da yana ko'p tayyor modul bor. Bilib qo'ysang foydali: - `os` — operatsion tizim bilan ishlash (papkalar, muhit o'zgaruvchilari). - `statistics` — o'rtacha, mediana, standart og'ish. - `time` — vaqtni o'lchash (`time.perf_counter`), kutish (`time.sleep`). - `logging` — `print` o'rniga professional log yozish.

```
import statistics  
baholar = [85, 90, 78, 92]  
print(statistics.mean(baholar)) # o'rtacha: 86.25  
print(statistics.median(baholar)) # mediana
```

Masalalar (20 ta)

Bu masalalar oldingi modullarga tayanadi.

Oson (1–7):

1. `Counter` bilan `"banana"` so'zidagi har harf nechta ekanini sana.

2. `Counter.most_common(1)` bilan `[1, 2, 2, 3, 3, 3]` dagi eng ko'p uchragan sonni top.
3. `datetime.now()` dan bugungi sanani `"DD.MM.YYYY"` formatida chiqar.
4. `random.randint` bilan 1–100 oralig'ida 5 ta tasodifiy son ro'yxatini yasa.
5. `math.sqrt` va `math.factorial` bilan 144 ning ildizini va $5!$ ni chiqar.
6. `random.choice` bilan ro'yxatdan tasodifiy element tanla.
7. `random.shuffle` bilan `[1, 2, 3, 4, 5]` ni aralashtir.

O'rta (8–14):

1. `defaultdict(int)` bilan so'zlar ro'yxatidagi har so'z chastotasini hisobla.
2. `defaultdict(list)` bilan talabalarni sinflar bo'yicha guruhla.
3. `timedelta` bilan bugundan 30 kun keyingi sanani hisobla.
4. `strptime` bilan `"2026-03-15"` matnini sanaga aylantirib, hafta kunini (`%A`) chiqar.
5. `statistics` bilan baholar ro'yxatining o'rtacha va medianasini chiqar.
6. `Counter` bilan matndagi eng ko'p uchragan 3 ta so'zni top.
7. `random.sample` bilan 1–49 dan 6 ta takrorlanmas tasodifiy son tanla (lotereya).

Murakkab (15–20):

1. `Counter` bilan ikki ro'yxatning umumiy elementlarini chastotasi bilan top (`&` ishlat).
2. Tug'ilgan kuninggacha necha kun qolganini hisobla (`date` va `timedelta`).
3. `defaultdict` bilan jumladagi har bir harf bilan boshlanadigan so'zlarni guruhla.
4. Oddiy "tanga tashlash" simulyatsiyasi: `random` bilan 1000 marta tashlab, "orol"/"yozuv" nisbatini chiqar.
5. `datetime` bilan ikki sana orasidagi farqni kun, hafta ko'rinishida chiqar.
6. So'zlar ro'yxatining chastota tahlili: `Counter` bilan hisobla, eng ko'p va eng kam uchraganini top, va natijani chiroyli jadval ko'rinishida chiqar.

 Yechimlar



```
# 1
from collections import Counter
print(Counter("banana"))      # Counter({'a': 3, 'n': 2, 'b': 1})

# 2
print(Counter([1, 2, 2, 3, 3, 3]).most_common(1))  # [(3, 3)]

# 3
from datetime import datetime
print(datetime.now().strftime("%d.%m.%Y"))

# 4
import random
print([random.randint(1, 100) for _ in range(5)])

# 5
import math
print(math.sqrt(144), math.factorial(5))  # 12.0 120

# 6
import random
print(random.choice(["olma", "banan", "uzum"]))

# 7
import random
sonlar = [1, 2, 3, 4, 5]
random.shuffle(sonlar)
print(sonlar)

# 8
from collections import defaultdict
chastota = defaultdict(int)
for soz in ["non", "suv", "non", "non", "suv"]:
    chastota[soz] += 1
print(dict(chastota))          # {'non': 3, 'suv': 2}

# 9
from collections import defaultdict
guruh = defaultdict(list)
for ism, sinf in [("Ali", "A"), ("Vali", "B"), ("Gul", "A")]:
    guruh[sinf].append(ism)
print(dict(guruh))             # {'A': ['Ali', 'Gul'], 'B': ['Vali']}

# 10
from datetime import date, timedelta
print(date.today() + timedelta(days=30))

# 11
from datetime import datetime
d = datetime.strptime("2026-03-15", "%Y-%m-%d")
print(d.strftime("%A"))        # Sunday

# 12
import statistics
baholar = [85, 90, 78, 92, 88]
print(statistics.mean(baholar), statistics.median(baholar))
```

```

# 13
from collections import Counter
matn = "olma non olma suv non olma"
print(Counter(matn.split()).most_common(3))

# 14
import random
print(random.sample(range(1, 50), 6))

# 15
from collections import Counter
a = Counter([1, 2, 2, 3])
b = Counter([2, 3, 3, 4])
print(a & b)                # Counter({2: 1, 3: 1})

# 16
from datetime import date
bugun = date.today()
tugilgan = date(bugun.year, 12, 31) # masalan 31-dekabr
if tugilgan < bugun:
    tugilgan = date(bugun.year + 1, 12, 31)
print((tugilgan - bugun).days, "kun qoldi")

# 17
from collections import defaultdict
jumla = "olma anor banan behi olcha"
guruh = defaultdict(list)
for soz in jumla.split():
    guruh[soz[0]].append(soz)
print(dict(guruh))

# 18
import random
orol = 0
for _ in range(1000):
    if random.choice(["orol", "yozuv"]) == "orol":
        orol += 1
print(f"Orol: {orol}, Yozuv: {1000 - orol}")

# 19
from datetime import date
d1 = date(2026, 1, 1)
d2 = date(2026, 12, 31)
farq = (d2 - d1).days
print(f"{farq} kun = {farq // 7} hafta {farq % 7} kun")

# 20
from collections import Counter
sozlar = "olma non olma suv non olma choy".split()
hisob = Counter(sozlar)
print("Eng ko'p:", hisob.most_common(1)[0])
print("Eng kam:", hisob.most_common()[-1])
for soz, son in hisob.most_common():
    print(f"{soz:<6} {'*' * son}")

```

12 — Parallel ishlash va `asyncio`

Hozirgacha dasturlarimiz **ketma-ket** ishladi: bir ish tugaguncha keyingisi kutadi. Lekin ba'zan bir nechta ishni **bir vaqtda** qilish kerak — masalan, 100 ta internet sahifani yuklab olishda har birini navbatma-navbat kutish sekin. Bu modulda bir vaqtda bir nechta ishni boshqarishni o'rganasan. Bu mavzu biroz murakkab — tushunchalar darajasida, sodda misollar bilan ko'ramiz.

Bu modulda: parallel ishlash nima, threadlar (I/O kutish uchun), GIL tushunchasi, va `asyncio` (`async` / `await`) asoslari.

12.1 Nega parallel ishlash kerak?

Ishlar ikki xil bo'ladi:

- **Kutish ishi (I/O)** — internet so'rovi, fayl o'qish, baza so'rovi. Dastur ko'p vaqtni shunchaki **kutib** o'tkazadi.
- **Hisob ishi (CPU)** — og'ir matematik hisob-kitob. Protsessor band bo'ladi.

Kutish ishida parallellik juda foydali: bitta so'rovni kutib turganda, boshqasini boshlash mumkin. Misol uchun, 3 ta sahifani har biri 2 soniyada yuklasa — ketma-ket 6 soniya, parallel esa ~2 soniya.

12.2 Threadlar — bir vaqtda bir nechta ish

`threading` moduli bir nechta ishni "bir vaqtda" boshqarish imkonini beradi (ayniqsa kutish ishlarida foydali):

```
import threading
import time

def yuklab_ol(nom):
    print(f"{nom} boshlandi")
    time.sleep(2)          # kutishni taqlid qiladi (masalan
internet)
    print(f"{nom} tugadi")

# Uchta ishni parallel boshlash:
threadlar = []
for nom in ["A", "B", "C"]:
    t = threading.Thread(target=yuklab_ol, args=(nom,))
```



```
threadlar.append(t)
t.start()                # boshlanadi

for t in threadlar:
    t.join()              # hammasi tugashini kutamiz

# Ketma-ket bo'lsa 6 soniya, threadlar bilan ~2 soniya
```

12.3 GIL — Python'ning muhim xususiyati

Python'da bir muhim qoida bor: **GIL** (Global Interpreter Lock). Soddaroq aytganda — Python'da bir vaqtda faqat **bitta** thread haqiqiy hisob-kitob qiladi.

Bundan kelib chiqadigan oddiy qoida:

Ish turi	Misol	Threadlar yordam beradimi?
Kutish (I/O)	internet, fayl, baza	✅ Ha — kutishda boshqa thread ishlaydi
Hisob (CPU)	og'ir matematika	❌ Yo'q — GIL tufayli navbatga turadi

Ya'ni: agar dasturing ko'p **kutsa** (internet/fayl) — threadlar tezlashtiradi. Agar ko'p **hisoblasa** (matematika) — threadlar yordam bermaydi (buning uchun alohida usul, `multiprocessing`, bor, lekin u ilg'orroq mavzu). Hozircha shu qoidani bilib qo'y.

Quyidagi diagramma GIL qulfi qanday ishlashini ko'rsatadi — bir vaqtda faqat bitta thread Python bytecode bajaradi:

GIL - Global Interpreter Lock

Bir vaqtda faqat BITTA thread Python bytecode bajaradi



12.4 `asyncio` — minglab kutish ishini boshqarish

`asyncio` — ayniqsa ko'p sonli kutish ishlari (masalan minglab internet so'rovi) uchun zamonaviy usul. `async def` bilan maxsus funksiya yozasan, `await` bilan "kutilayotgan" joyda boshqarivni bo'shatasan:

```
import asyncio

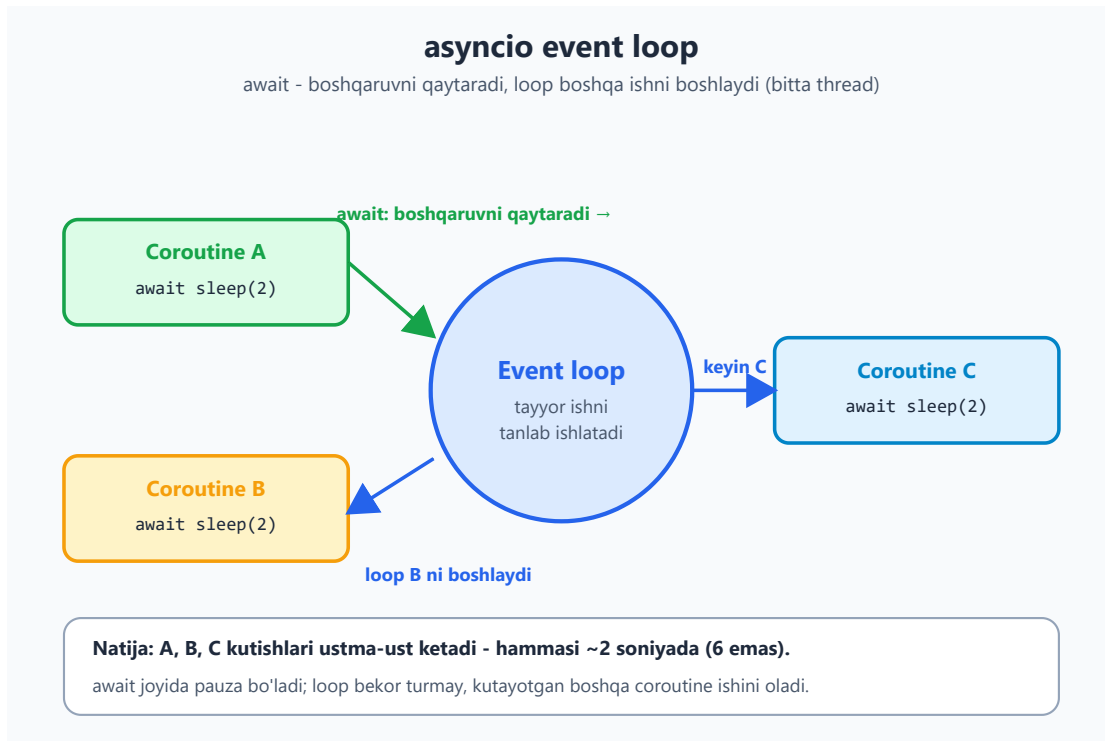
async def yuklab_ol(nom):
    print(f"{nom} boshlandi")
    await asyncio.sleep(2)      # NONBLOCKING kutish (time.sleep
    # emas!)
    print(f"{nom} tugadi")
    return f"{nom} natijasi"

async def main():
    # gather — bir nechta ishni BIR VAQTDA boshlaydi:
    natijalar = await asyncio.gather(
        yuklab_ol("A"),
        yuklab_ol("B"),
        yuklab_ol("C"),
    )
    print(natijalar)            # ~2 soniyada hammasi (6 emas)

asyncio.run(main())            # async dasturni shunday ishga
                                # tushirasan
```

Eng ko'p uchraydigan xato: `async` funksiya ichida oddiy `time.sleep()` ishlatish — u hammasini bloklaydi. `asyncio.sleep()` ishlat. Async kodda kutish uchun maxsus, "nomblocking" funksiyalar bo'ladi.

Mana `asyncio` qanday ishlaydi: `await` ga yetganda coroutine boshqaruvni **event loop**'ga qaytaradi, loop esa kutib turgan boshqa coroutine ishini boshlaydi (hammasi bitta thread'da):



12.5 Qaysi usulni qachon ishlataman?

Sodda yo'riqnoma:

```
Bitta ish, parallel kerak emas?  
→ oddiy ketma-ket kod (murakkablashtirma!)
```

```
Bir nechta kutish ishi (internet/fayl)?  
→ threading (oson) yoki asyncio (ko'p son uchun samarali)
```

```
Og'ir hisob-kitob?  
→ multiprocessing (ilg'or mavzu, keyinroq)
```

`threading` va `asyncio` ikkalasi ham kutish (I/O) ishlari uchun, lekin ishlash modeli farq qiladi — threadlarda bir nechta OS thread bo'ladi, `asyncio` da esa bitta thread ichida event loop navbatlashtiradi:

Threadlar va asyncio - farqi

Ikkalasi ham KUTISH (I/O) ishlari uchun; ishlash modeli boshqacha

threading

Bir nechta OS thread; OS almashtiradi

Thread A - mustaqil ishlaydi

Thread B - mustaqil ishlaydi

Thread C - mustaqil ishlaydi

- + Oson, mavjud kodga mos.
- + Kam sonli ishlar uchun qulay.
- Har thread xotira/almashtiruv xarajati.
- GIL: CPU hisobini tezlashtirmaydi.

asyncio

Bitta thread; event loop navbatlashtiradi

Bitta thread + event loop

coro A

coro B

coro C

await da navbat boshqasiga o'tadi

await asyncio.sleep(...)

- + Minglab kutish ishi uchun samarali.
- + Kam xotira xarajati.
- async/await yozuvini talab qiladi.
- await joyida blok qilsang (time.sleep) - hamma to'xtaydi.

Og'ir HISOB (CPU) uchun ikkalasi ham yaramaydi - multiprocessing kerak.

Eng muhim maslahat: parallellik kodni murakkablashtiradi va yangi xatolar (masalan bir vaqtda bir ma'lumotni o'zgartirish) keltiradi. Faqat **haqiqatan kerak bo'lganda** ishlat. Avvalo oddiy ketma-ket kod yoz — sekin bo'lsa, keyin optimallashtir.



Masalalar (20 ta)

Bu mavzu murakkab — masalalar asosan tushunish va oddiy misollar uchun.

Oson (1–7):

1. Bitta `async def salom()` yoz: "salom" chop etsin. `asyncio.run` bilan ishga tushir.
2. `await asyncio.sleep(1)` bilan 1 soniya kutib, keyin xabar chiqaruvchi coroutine yoz.
3. `threading.Thread` bilan bitta funksiyani ishga tushir (ichida `time.sleep(1)`).
4. GIL nima ekanini o'z so'zlaring bilan (kod izohi sifatida) yoz.
5. Kutish ishi (I/O) va hisob ishi (CPU) farqini misol bilan izohda yoz.
6. `asyncio.gather` bilan ikkita oddiy coroutine'ni birga ishga tushir.
7. `threading.Thread` bilan ikkita funksiyani parallel boshla (`start` + `join`).

O'rta (8–14):

1. `asyncio.gather` bilan 3 ta coroutine'ni birga ishga tushir, har biri turli muddat uxlasin.

2. Async funksiya yoz: parametr sifatida nom va kutish vaqtini olsin, kutgach nomni qaytarsin.
3. 3 ta thread'ni ro'yxatda yaratib, hammasini `start` qil, keyin `join` bilan kut.
4. Async coroutine natijasini `await` bilan olib, o'zgaruvchiga saqlab chiqar.
5. `time.sleep` (oddiy) va `asyncio.sleep` (async) farqini izohda tushuntir.
6. `asyncio.gather` natijasini ro'yxatga olib, har birini chiqar.
7. Bir vaqtda 5 ta "yuklab olish"ni (har biri `asyncio.sleep`) ishga tushirib, umumiy vaqtni `time` bilan o'lcha.

Murakkab (15–20):

1. Ketma-ket va parallel (gather bilan) bajarishni taqqosla: 3 ta 1-soniyalik ishni ikki usulda bajarib, vaqtlarini chiqar.
2. Async funksiyalar zanjiri: bittasi natija qaytarsin, ikkinchisi shu natijani ishlatsin (`await` bilan).
3. `threading` bilan umumiy hisoblagichni bir nechta thread'dan oshir; natija to'g'ri chiqishini kuzat (bu yerda `Lock` kerakligini izohda ayt).
4. `asyncio.gather` bilan 10 ta ishni parallel bajar, har biri o'z nomini va natijasini qaytarsin.
5. Oddiy "yuklab oluvchi" simulyator: nomlar ro'yxatini async tarzda "yuklab ol" (har biri tasodifiy vaqt uxlasin, `asyncio.sleep + random`).
6. Threadli va asynclioli variantni yonma-yon yozib, qaysi qaysi holatda mosligini izohda tushuntir (kutish vs hisob).

 Yechimlar



```
import asyncio
import threading
import time

# 1
async def salom():
    print("salom")
asyncio.run(salom())

# 2
async def kut():
    await asyncio.sleep(1)
    print("1 soniya o'tdi")
asyncio.run(kut())

# 3
def ish():
    time.sleep(1)
    print("ish tugadi")
t = threading.Thread(target=ish)
t.start(); t.join()

# 4
# GIL: Python'da bir vaqtda faqat bitta thread haqiqiy hisob-kitob qiladi.
# Shuning uchun og'ir CPU ishida threadlar tezlashtirmaydi,
# lekin kutish (I/O) ishida foydali – kutishda boshqa thread ishlay oladi.

# 5
# I/O ishi: internet so'rovi, fayl o'qish – dastur ko'p kutadi.
# CPU ishi: katta matematik hisob – protsessor band bo'ladi.
# Parallelllik I/O da ko'proq foyda beradi.

# 6
async def a():
    await asyncio.sleep(0.5); return "a"
async def b():
    await asyncio.sleep(0.5); return "b"
async def m6():
    print(await asyncio.gather(a(), b()))
asyncio.run(m6())

# 7
def ish2(nom):
    time.sleep(1)
    print(f"{nom} tugadi")
t1 = threading.Thread(target=ish2, args=("A",))
t2 = threading.Thread(target=ish2, args=("B",))
t1.start(); t2.start(); t1.join(); t2.join()

# 8
async def uxla(nom, sek):
    await asyncio.sleep(sek)
    return f"{nom}: {sek}s"
async def m8():
    print(await asyncio.gather(uxla("A", 1), uxla("B", 0.5), uxla("C", 0.2)))
asyncio.run(m8())
```

```

# 9
async def yuklab(nom, vaqt):
    await asyncio.sleep(vaqt)
    return nom
async def m9():
    print(await yuklab("Sahifa", 0.3))
asyncio.run(m9())

# 10
def ish3(nom):
    time.sleep(0.5)
    print(nom)
threadlar = [threading.Thread(target=ish3, args=(f"T{i}",)) for i in
range(3)]
for t in threadlar: t.start()
for t in threadlar: t.join()

# 11
async def hisob():
    await asyncio.sleep(0.2)
    return 42
async def m11():
    natija = await hisob()
    print("Natija:", natija)
asyncio.run(m11())

# 12
# time.sleep - butun dasturni (yoki event loop'ni) bloklaydi.
# asyncio.sleep - faqat shu coroutine'ni "uxlatadi", boshqalar ishlay
beradi.
# Async kodda DOIM asyncio.sleep ishlat.

# 13
async def m13():
    natijalar = await asyncio.gather(*[uxla(f"ish{i}", 0.1) for i in
range(3)])
    for n in natijalar:
        print(n)
asyncio.run(m13())

# 14
async def m14():
    b = time.perf_counter()
    await asyncio.gather(*[asyncio.sleep(1) for _ in range(5)])
    print(f"5 ta ish: {time.perf_counter() - b:.2f}s") # ~1s (parallel)
asyncio.run(m14())

# 15
async def bir_ish():
    await asyncio.sleep(1)
async def m15():
    # ketma-ket:
    b = time.perf_counter()
    for _ in range(3):
        await bir_ish()
    print(f"ketma-ket: {time.perf_counter() - b:.2f}s") # ~3s
    # parallel:
    b = time.perf_counter()
    await asyncio.gather(bir_ish(), bir_ish(), bir_ish())
    print(f"parallel: {time.perf_counter() - b:.2f}s") # ~1s

```

```

asyncio.run(m15())

# 16
async def birinchi():
    await asyncio.sleep(0.2)
    return 10
async def ikkinchi(x):
    await asyncio.sleep(0.2)
    return x * 2
async def m16():
    a = await birinchi()
    b = await ikkinchi(a)
    print(b) # 20
asyncio.run(m16())

# 17
hisoblagich = 0
lock = threading.Lock()
def oshir():
    global hisoblagich
    for _ in range(10000):
        with lock: # Lock – bir vaqtda faqat bitta thread
            o'zgartirsin
            hisoblagich += 1
ts = [threading.Thread(target=oshir) for _ in range(3)]
for t in ts: t.start()
for t in ts: t.join()
print(hisoblagich) # 30000 (Lock'siz kamroq chiqishi mumkin edi)

# 18
async def vazifa(i):
    await asyncio.sleep(0.1)
    return f"ish{i} tayyor"
async def m18():
    natijalar = await asyncio.gather(*[vazifa(i) for i in range(10)])
    print(natijalar)
asyncio.run(m18())

# 19
import random
async def yuklab_ol(nom):
    await asyncio.sleep(random.uniform(0.1, 0.5))
    return f"{nom} yuklandi"
async def m19():
    nomlar = ["sahifa1", "sahifa2", "sahifa3"]
    print(await asyncio.gather(*[yuklab_ol(n) for n in nomlar]))
asyncio.run(m19())

# 20
# threading – oddiy kod bilan, kam sonli kutish ishlari uchun qulay.
# asyncio – ko'p sonli kutish ishlari (minglab so'rov) uchun samarali.
# Ikkalasi ham KUTISH (I/O) uchun. Og'ir HISOB uchun multiprocessing kerak.

```


13 — Dasturni test qilish (`pytest`)

Kod yozdik — lekin u to'g'ri ishlayaptimi? Har safar qo'lda tekshirish (dasturni ishga tushirib, natijani ko'rib) zerikarli va ishonchsiz. **Testlar** — kodning to'g'ri ishlashini avtomatik tekshiradigan kod. Python'da buning eng mashhur vositasi — `pytest`. Bu modulda ishonchli test yozishni o'rganasan.

Bu modulda: nega test kerak, `pytest` asoslari (`assert` , test funksiyalari), fixture'lar, parametrash, va xatolarni test qilish.

13.1 Nega test yozamiz?

Tasavvur qil: katta dasturda bir joyni o'zgartirding. Boshqa joy buzilmaganini qanday bilasan? Qo'lda hammasini sinab ko'rish — uzoq va xatoga yo'l qo'yiladi. Testlar bir buyruq bilan hamma narsani tekshiradi.

`pytest` ni o'rnatish (terminalda):

```
pip install pytest
```

13.2 Birinchi test

Test — `assert` ishlatadigan oddiy funksiya. `assert` shartni tekshiradi: agar `True` bo'lsa hech narsa, `False` bo'lsa xato (test "yiqiladi"):

```
# matematika.py
def qosh(a, b):
    return a + b
```

```
# test_matematika.py ← fayl nomi test_ bilan boshlanishi SHART
from matematika import qosh

def test_qosh():
    assert qosh(2, 3) == 5
    # "qosh(2,3) natijasi 5 ga teng bo'lishi kerak"

def test_qosh_manfiy():
    assert qosh(-1, -1) == -2
```

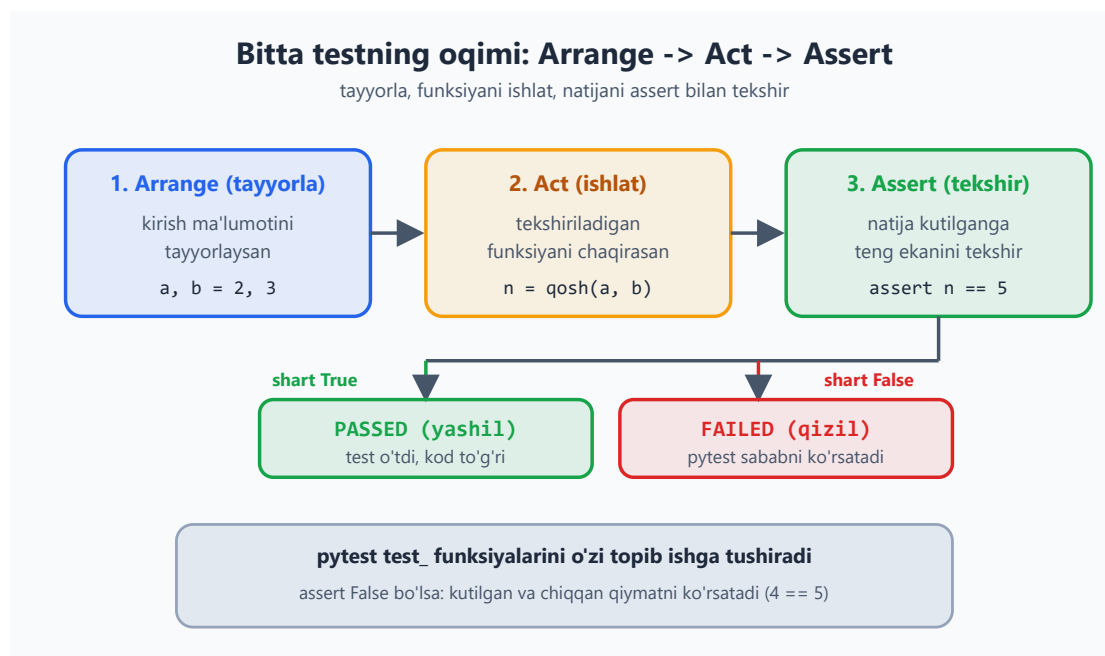
Testlarni ishga tushirish (terminalda, fayl turgan papkada):

```
pytest # barcha test_*.py fayllarni topib ishga tushiradi
pytest -v # batafsil (har test nomi ko'rinadi)
```

Hammasi to'g'ri bo'lsa, yashil **PASSED** chiqadi. Xato bo'lsa, qaysi test va nima sababdan yiqilganini ko'rsatadi.

Discovery qoidasi: `pytest` `test_` bilan boshlanadigan fayllardagi `test_` bilan boshlanadigan funksiyalarni avtomatik topadi. Hech narsa ro'yxatga olish kerak emas.

Har bir test odatda bir xil uch bosqichdan iborat bo'ladi: tayyorla, ishlat, tekshir.



13.3 `assert` — turli tekshiruvlar

```
def test_misolilar():
    assert 2 + 2 == 4
    assert "salom" in "salom dunyo" # ichida bormi
    assert len([1, 2, 3]) == 3
    assert [1, 2] == [1, 2] # ro'yxatlar teng
    natija = None
    assert natija is None
    assert 5 > 3
```

`pytest` aqlli — test yiqilsa, nima kutilgani va nima chiqqanini ko'rsatadi:

```
def test_xato():
    assert qosh(2, 2) == 5
# pytest chiqaradi:
#   assert 4 == 5
```

13.4 Fixture — test uchun tayyor ma'lumot

Ko'p testga bir xil tayyorgarlik kerak bo'lsa, **fixture** ishlatasan. Fixture — testga argument sifatida uzatiladigan tayyor ma'lumot:

```
import pytest

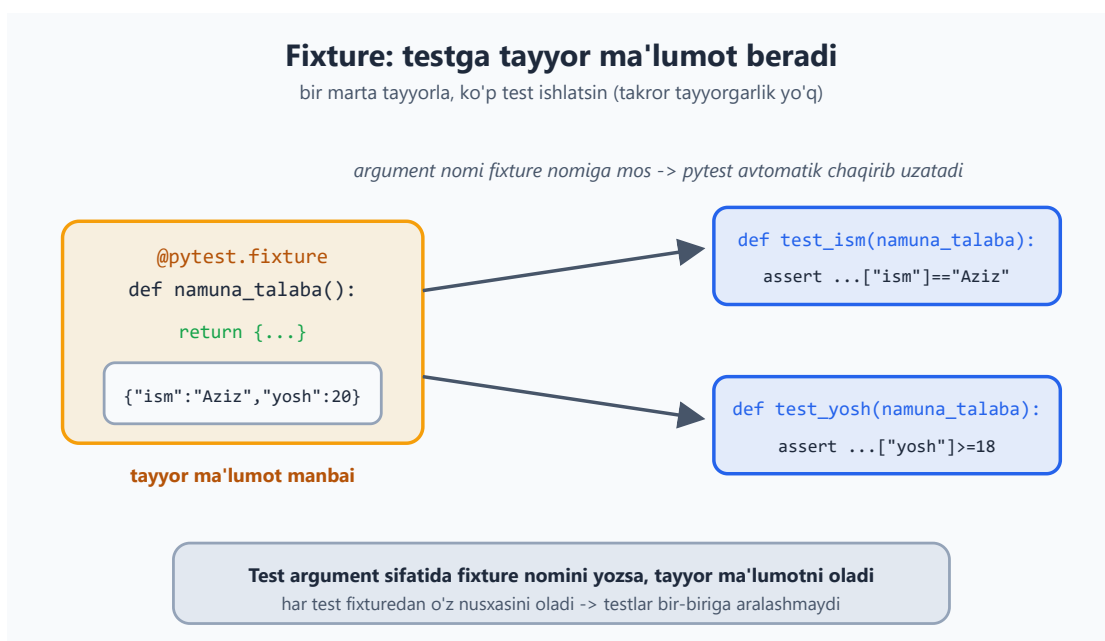
@pytest.fixture
def namuna_talaba():
    return {"ism": "Aziz", "yosh": 20}

def test_ism(namuna_talaba):
    # fixture nomini argument
    # qilib ol
    assert namuna_talaba["ism"] == "Aziz"

def test_yosh(namuna_talaba):
    assert namuna_talaba["yosh"] >= 18
```

pytest `namuna_talaba` ni avtomatik chaqirib, natijasini testga uzatadi. Bu takror tayyorgarlikni kamaytiradi.

Quyidagi diagramma bitta fixture qanday qilib bir nechta testga tayyor ma'lumot berishini ko'rsatadi.



13.5 `parametrize` — bitta test, ko'p holat

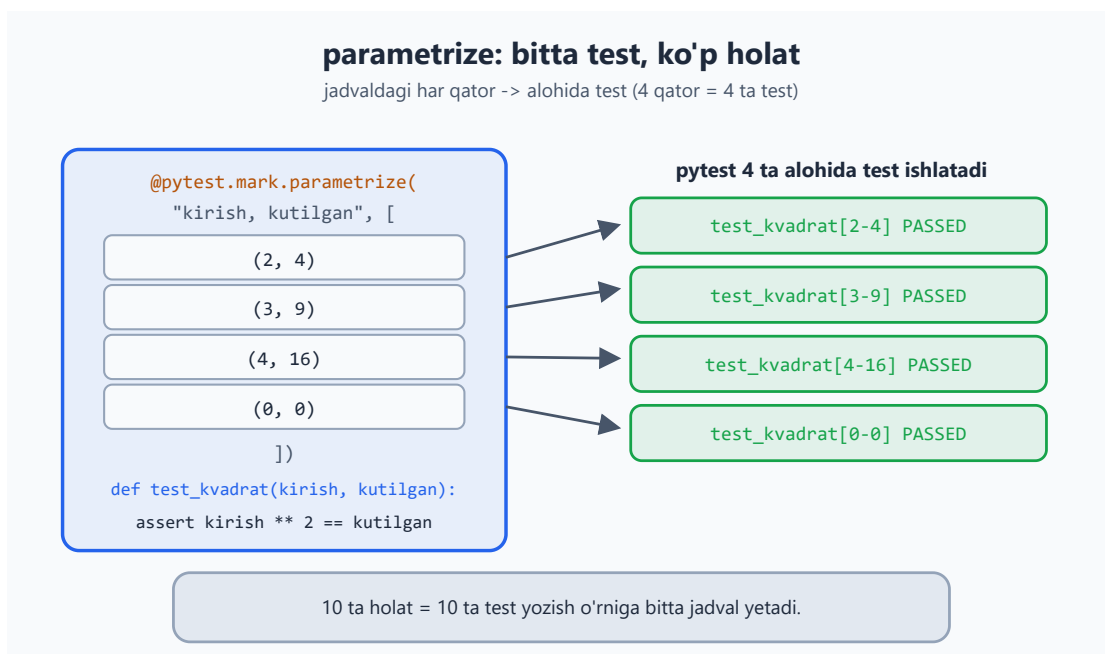
Bir testni turli kirishlar bilan tekshirish uchun:

```
import pytest

@pytest.mark.parametrize("kirish, kutilgan", [
    (2, 4),
    (3, 9),
    (4, 16),
    (0, 0),
])
def test_kvadrat(kirish, kutilgan):
    assert kirish ** 2 == kutilgan
# Bu 4 ta alohida test sifatida ishlaydi
```

Bu juda foydali: bir funksiyani 10 ta holat bilan sinashing kerak bo'lsa, 10 ta alohida test yozish o'rniga bitta `parametrize` jadval yozasan.

Diagrammada jadvaldagi har bir qator alohida test sifatida ishlashini ko'rishing mumkin.



13.6 Xatolarni test qilish

Funksiya kerakli vaqtda xato chaqirishini ham test qilish kerak (7-modulni esla):

```
import pytest

def bol(a, b):
    if b == 0:
        raise ValueError("nolga bo'lish mumkin emas")
    return a / b

def test_normal():
    assert bol(10, 2) == 5

def test_nolga_bolish():
    with pytest.raises(ValueError):          # "bu yerda ValueError
chiqishi KERAK"
        bol(10, 0)
```

`pytest.raises` ichidagi kod kutilgan xatoni chaqirmasa, test yiqiladi. Ya'ni "xato chiqishi shart" ekanini tekshiradi.

13.7 TDD — avval test, keyin kod

Ilg'or amaliyot **TDD** (Test-Driven Development): avval test yozasan (u yiqiladi, chunki kod yo'q), keyin testni o'tkazadigan kod yozasan. Bu kodni aniq talablar asosida yozishga majbur qiladi.

```
# 1. Avval test (yiqiladi):
def test_juft():
    assert juftmi(4) is True
    assert juftmi(3) is False

# 2. Keyin testni o'tkazadigan kod:
def juftmi(n):
    return n % 2 == 0
```

Hozircha TDD'ni qat'iy qo'llash shart emas. Lekin "kodimni qanday test qilaman?" deb o'ylash — kodni yaxshiroq yozishga yordam beradi.



Masalalar (20 ta)

Bu masalalarda funksiya yozib, keyin uning testini yoz. `pytest` ni terminalda ishga tushirib tekshir.

Oson (1–7):

1. `kop(a, b)` funksiyasini yoz va `test_kop` testini yoz (`assert` bilan).

2. `juftmi(n)` yoz; juft va toq holat uchun ikkita test yoz.
3. `salom(ism)` yoz (matn qaytarsin); test'da natijada ism borligini `in` bilan tekshir.
4. Ro'yxat qaytaruvchi funksiya yoz; test'da uzunligini va birinchi elementni tekshir.
5. `assert` bilan oddiy holatlarni tekshir: `2+2==4`, `"a" in "abc"`, `len([1,2])==2`.
6. Oddiy `@pytest.fixture` yoz (lug'at qaytarsin), uni testda ishlat.
7. `@pytest.mark.parametrize` bilan `qosh(a,b)` ni 3 xil holatda test qil.

O'rta (8–14):

1. `bol(a, b)` (nolga bo'lganda `ValueError`) yoz, `pytest.raises` bilan test qil.
2. Normal holatni va xato holatini ikki alohida test bilan tekshir.
3. `kvadrat(n)` funksiyasini `parametrize` bilan 4 ta holatda sina.
4. Fixture yoz: talabalar ro'yxati qaytarsin, uni bir nechta testda ishlat.
5. `ortacha(sonlar)` yoz; bo'sh ro'yxat berilganda `ZeroDivisionError` chiqishini test qil.
6. `palindrommi(soz)` yoz; `parametrize` bilan palindrom va emas holatlarni test qil.
7. `eng_katta(sonlar)` yoz; turli ro'yxatlar bilan `parametrize` orqali test qil.

Murakkab (15–20):

1. `Hisob` klassini (5-moduldagi bank hisobi) yozib, `qoy` va `yech` metodlarini test qil (fixture bilan yangi hisob yarat).
2. `yech` da yetarli mablag' bo'lmasa xato chiqishini `pytest.raises` bilan test qil.
3. `slugify(matn)` yoz (`"Salom Dunyo" → "salom-dunyo"`); `parametrize` bilan turli holatlarni test qil.
4. `Kalkulyator` klassini yozib (`qosh`, `ayir`, `kop`, `bol`), barcha metodlarni test bilan qopla (nolga bo'lish ham).
5. Fixture kompozitsiyasi: bir fixture (bo'sh savat) va undan foydalanadigan boshqa fixture (to'la savat) yoz, ikkalasini testlarda ishlat.
6. To'liq test to'plami: `Talaba` klassi (`ball_qosh`, `ortacha`) uchun normal holatlar, chegara holatlar (bo'sh ballar) va xato holatlarini qoplaydigan testlar yoz.

 Yechimlar



```
import pytest

# 1
def kop(a, b):
    return a * b
def test_kop():
    assert kop(3, 4) == 12

# 2
def juftmi(n):
    return n % 2 == 0
def test_juft():
    assert juftmi(4) is True
def test_toq():
    assert juftmi(3) is False

# 3
def salom(ism):
    return f"Salom, {ism}!"
def test_salom():
    assert "Aziz" in salom("Aziz")

# 4
def sonlar():
    return [10, 20, 30]
def test_sonlar():
    r = sonlar()
    assert len(r) == 3
    assert r[0] == 10

# 5
def test_oddiy():
    assert 2 + 2 == 4
    assert "a" in "abc"
    assert len([1, 2]) == 2

# 6
@pytest.fixture
def user():
    return {"ism": "Vali", "rol": "admin"}
def test_user(user):
    assert user["rol"] == "admin"

# 7
@pytest.mark.parametrize("a, b, kutilgan", [(1, 1, 2), (5, 5, 10), (-1, 1, 0)])
def test_qosh(a, b, kutilgan):
    assert a + b == kutilgan

# 8
def bol(a, b):
    if b == 0:
        raise ValueError("nolga bo'lish mumkin emas")
    return a / b
def test_bol_nol():
    with pytest.raises(ValueError):
        bol(10, 0)
```

```

# 9
def test_normal():
    assert bol(10, 2) == 5
def test_xato():
    with pytest.raises(ValueError):
        bol(5, 0)

# 10
def kvadrat(n):
    return n ** 2
@pytest.mark.parametrize("n, kutilgan", [(2, 4), (3, 9), (4, 16), (0, 0)])
def test_kvadrat(n, kutilgan):
    assert kvadrat(n) == kutilgan

# 11
@pytest.fixture
def talabalar():
    return [{"ism": "Aziz", "ball": 85}, {"ism": "Gul", "ball": 70}]
def test_talabalar_soni(talabalar):
    assert len(talabalar) == 2
def test_birinchii(talabalar):
    assert talabalar[0]["ism"] == "Aziz"

# 12
def ortacha(sonlar):
    return sum(sonlar) / len(sonlar)
def test_ortacha_bosh():
    with pytest.raises(ZeroDivisionError):
        ortacha([])

# 13
def palindrommi(soz):
    return soz == soz[::-1]
@pytest.mark.parametrize("soz, kutilgan", [("radar", True), ("salom", False), ("aba", True)])
def test_palindrom(soz, kutilgan):
    assert palindrommi(soz) == kutilgan

# 14
def eng_katta(sonlar):
    return max(sonlar)
@pytest.mark.parametrize("sonlar, kutilgan", [([1, 5, 3], 5), ([10], 10), ([-1, -5], -1)])
def test_eng_katta(sonlar, kutilgan):
    assert eng_katta(sonlar) == kutilgan

# 15
class Hisob:
    def __init__(self, balans=0):
        self.balans = balans
    def qoy(self, s):
        self.balans += s
    def yech(self, s):
        if s > self.balans:
            raise ValueError("mablag' yetarli emas")
        self.balans -= s
@pytest.fixture
def hisob():
    return Hisob(100)
def test_qoy(hisob):

```



```

        hisob.qoy(50)
        assert hisob.balans == 150
def test_yech(hisob):
    hisob.yech(40)
    assert hisob.balans == 60

# 16
def test_yech_xato(hisob):
    with pytest.raises(ValueError):
        hisob.yech(1000)

# 17
import re
def slugify(matn):
    matn = matn.strip().lower()
    matn = re.sub(r"^\w\s-", "", matn)
    return re.sub(r"[\s]+", "-", matn)
@pytest.mark.parametrize("kirish, kutilgan", [
    ("Salom Dunyo", "salom-dunyo"),
    ("Python", "python"),
    ("Bir Ikki Uch", "bir-ikki-uch"),
])
def test_slugify(kirish, kutilgan):
    assert slugify(kirish) == kutilgan

# 18
class Kalkulyator:
    def qosh(self, a, b): return a + b
    def ayir(self, a, b): return a - b
    def kop(self, a, b): return a * b
    def bol(self, a, b):
        if b == 0:
            raise ValueError("nolga bo'lish")
        return a / b
@pytest.fixture
def kalk():
    return Kalkulyator()
def test_qosh_k(kalk):
    assert kalk.qosh(2, 3) == 5
def test_bol_k(kalk):
    assert kalk.bol(10, 2) == 5
def test_bol_nol_k(kalk):
    with pytest.raises(ValueError):
        kalk.bol(1, 0)

# 19
@pytest.fixture
def bosh_savat():
    return []
@pytest.fixture
def tola_savat(bosh_savat):
    bosh_savat.append("olma")
    bosh_savat.append("non")
    return bosh_savat
def test_bosh(bosh_savat):
    assert len(bosh_savat) == 0
def test_tola(tola_savat):
    assert len(tola_savat) == 2

# 20
class Talaba:

```

```
def __init__(self, ism):
    self.ism = ism
    self.ballar = []
def ball_qosh(self, b):
    self.ballar.append(b)
def ortacha(self):
    if not self.ballar:
        raise ValueError("ballar yo'q")
    return sum(self.ballar) / len(self.ballar)
@pytest.fixture
def talaba():
    return Talaba("Aziz")
def test_ball_qosh(talaba):
    talaba.ball_qosh(80)
    assert talaba.ballar == [80]
def test_ortacha(talaba):
    talaba.ball_qosh(80)
    talaba.ball_qosh(90)
    assert talaba.ortacha() == 85
def test_ortacha_bosh(talaba):
    with pytest.raises(ValueError):
        talaba.ortacha()
```

14 — FastAPI bilan veb API

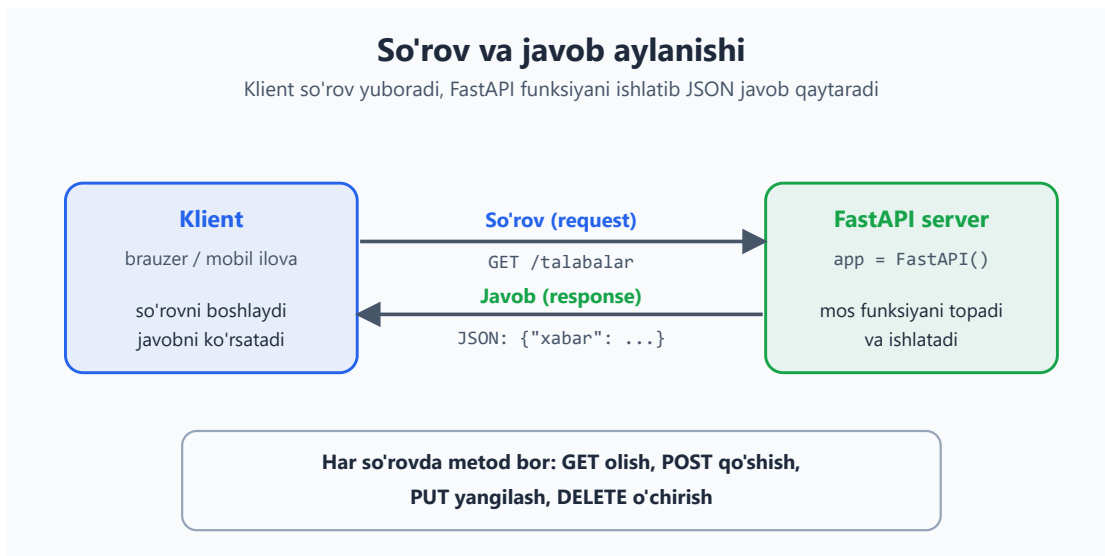
Hozirgacha yozgan dasturlarimiz faqat o'z kompyuteringda, terminalda ishladi. Lekin haqiqiy ilovalar (mobil ilova, sayt) ko'pincha **internet orqali** ma'lumot almashadi — buning uchun **veb API** kerak. Bu modulda **FastAPI** bilan oddiy, lekin ishlaydigan API yasashni o'rganasan. Bu — kursning kichik yakuniy loyihasi: oldingi modullardagi bilim (funksiyalar, lug'atlar, tip ko'rsatmalari) shu yerda birlashadi.

Bu modulda: veb API nima, FastAPI'da birinchi API, parametrlar (path va query), Pydantic bilan ma'lumotni qabul qilish, va oddiy CRUD (qo'shish/o'qish/o'zgartirish/o'chirish).

14.1 Veb API nima?

API — bu dasturlar bir-biri bilan "gaplashadigan" til. Veb API esa internet orqali ishlaydi: mijoz (brauzer, mobil ilova) **so'rov** (request) yuboradi, server **javob** (response) qaytaradi.

Quyidagi diagramma bu aylanishni ko'rsatadi: klient so'rov yuboradi, FastAPI mos funksiyani ishlatib JSON javob qaytaradi.



So'rovlarning asosiy turlari (HTTP metodlari):

Metod	Ma'nosi	Misol
GET	ma'lumot olish	talabalar ro'yxatini ko'rish

Metod	Ma'nosi	Misol
POST	yangi ma'lumot qo'shish	yangi talaba qo'shish
PUT	mavjudni yangilash	talaba ma'lumotini o'zgartirish
DELETE	o'chirish	talabani o'chirish

Javob odatda **JSON** ko'rinishida bo'ladi (8-moduldan tanish).

14.2 O'rnatish va birinchi API

```
pip install "fastapi[standard]"
```

Eng oddiy API — `main.py`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def bosh_sahifa():
    return {"xabar": "Salom, dunyo!"}
```

Ishga tushirish (terminalda):

```
fastapi dev main.py
```

Brauzerda `http://127.0.0.1:8000` ni och — `{"xabar": "Salom, dunyo!"}` chiqadi.

Bonus: `http://127.0.0.1:8000/docs` manzilini och — FastAPI **avtomatik** interaktiv hujjat yaratadi! U yerda API'ngni brauzerdan sinab ko'rishing mumkin. Bu FastAPI'ning eng yoqimli xususiyatlaridan biri.

`@app.get("/")` ni shunday o'qi: "kimdir / manziliga GET so'rov yuborsa, shu funksiyani ishlatib, natijasini qaytar". Funksiya lug'at qaytarsa, FastAPI uni avtomatik JSON ga aylantiradi.

14.3 Path parametrlari — manzildagi qiymat

Manzilning bir qismini o'zgaruvchi qilish mumkin:

```
@app.get("/salom/{ism}")
def salomla(ism: str):
    return {"xabar": f"Salom, {ism}!"}

# /salom/Aziz → {"xabar": "Salom, Aziz!"}
```

Tip ko'rsatmasi (`ism: str`, `id: int`) bu yerda **haqiqatan tekshiriladi** — FastAPI uni avtomatik tekshiradi va aylantiradi:

```
@app.get("/talaba/{id}")
def talaba_olish(id: int):          # id avtomatik int ga aylanadi
    return {"id": id, "tur": str(type(id))}

# /talaba/5 → {"id": 5, ...} (int)
# /talaba/abc → XATO (422): int kutilgan edi — FastAPI o'zi
# tekshiradi!
```

10-modulda "tip ko'rsatmalari tekshirilmaydi" degandik — bu sof Python'da. FastAPI esa ularni **ishlatadi**: so'rovdagi ma'lumotni tekshirish va aylantirish uchun. Bu juda qulay.

14.4 Query parametrlari — `?kalit=qiymat`

Manzil oxiridagi `?` dan keyingi qiymatlar — query parametrlari. Funksiya parametri sifatida olasan (standart qiymat bilan — ixtiyoriy bo'ladi):

```
@app.get("/qidiruv")
def qidiruv(q: str, limit: int = 10):
    return {"qidiruv": q, "limit": limit}

# /qidiruv?q=python → {"qidiruv": "python", "limit": 10}
# /qidiruv?q=python&limit=5 → {"qidiruv": "python", "limit": 5}
```

Endi ikkala turdagi parametrni yonma-yon solishtiramiz: path parametr manzilning bir qismi (`{}` ichida), query parametr esa `?` dan keyin `kalit=qiymat` ko'rinishida keladi.

Path parametr va Query parametr

Bir manzilda ikkala turdagi qiymat ham bo'lishi mumkin

/talaba/ 5 /kurslar ? limit=10&tur=faol

Path parametr

manzilning bir qismi ({} ichida)

```
@app.get("/talaba/{id}")
def f(id: int):
```

odatda MAJBURIY — qaysi
resurs kerakligini ko'rsatadi

Query parametr

? dan keyin kalit=qiymat

```
@app.get("/qidiruv")
def f(q: str, limit: int = 10):
```

standart qiymatli bo'lsa —
IXTIYORIY (filtrlash uchun)

14.5 Pydantic — ma'lumotni qabul qilish (POST)

Yangi ma'lumot qabul qilishda (POST), kelgan JSON qanday ko'rinishda bo'lishini **model** orqali belgilaysan. Buning uchun Pydantic ishlatasan:

```
from pydantic import BaseModel

class Talaba(BaseModel):          # ma'lumot "shakli"
    ism: str
    yosh: int
    ball: float = 0               # standart qiymatli - ixtiyoriy

@app.post("/talabalar")
def qoshish(talaba: Talaba):     # kelgan JSON avtomatik Talaba'ga
    aylanadi                     aylanadi
    return {"qoshildi": talaba}
```

Endi {"ism": "Aziz", "yosh": 20} JSON yuborsang, FastAPI uni avtomatik tekshiradi: - Maydon yetishmasa yoki tur noto'g'ri bo'lsa — o'zi xato (422) qaytaradi. - Hammasi to'g'ri bo'lsa — funksiyaga tayyor Talaba obyekti keladi.

Pydantic 5-moduldagi klasslarga o'xshaydi, lekin u qo'shimcha ravishda **kelgan ma'lumotni avtomatik tekshiradi**. Bu xavfsiz API yozishni juda osonlashtiradi — kirish ma'lumotini o'zing tekshirib o'tirishing shart emas.

14.6 To'liq misol — oddiy CRUD

Talabalarni xotirada (oddiy lug'atda) saqlovchi to'liq API. Bu — barcha asoslarni birlashtiradi:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class Talaba(BaseModel):
    ism: str
    yosh: int

# Xotirada saqlash (oddiy lug'at — dastur o'chsa yo'qoladi):
talabalar: dict[int, dict] = {}
keyingi_id = 1

@app.get("/talabalar") # HAMMASINI o'qish
def royxat():
    return talabalar

@app.get("/talabalar/{id}") # BITTASINI o'qish
def bittasi(id: int):
    if id not in talabalar:
        raise HTTPException(status_code=404, detail="Talaba topilmadi")
    return talabalar[id]

@app.post("/talabalar") # QO'SHISH
def qoshish(talaba: Talaba):
    global keyingi_id
    talabalar[keyingi_id] = talaba.model_dump()
    keyingi_id += 1
    return {"id": keyingi_id - 1, **talaba.model_dump()}

@app.put("/talabalar/{id}") # YANGILASH
def yangilash(id: int, talaba: Talaba):
    if id not in talabalar:
        raise HTTPException(status_code=404, detail="Talaba topilmadi")
    talabalar[id] = talaba.model_dump()
    return talabalar[id]

@app.delete("/talabalar/{id}") # O'CHIRISH
def ochirish(id: int):
    if id not in talabalar:
        raise HTTPException(status_code=404, detail="Talaba topilmadi")
    del talabalar[id]
    return {"ochirildi": id}
```

Quyidagi xaritada yuqoridagi endpointlar jamlangan: e'tibor ber — bir xil manzil (/talabalar/{id}) turli HTTP metod bilan turli amalni bajaradi.

REST endpoint xaritasi (CRUD)

Metod + manzil birikmasi bitta amalni belgilaydi

Metod	Manzil	Amal
GET	/talabalar	hammasini o'qish
GET	/talabalar/{id}	bittasini o'qish
POST	/talabalar	yangi qo'shish
PUT	/talabalar/{id}	yangilash
DELETE	/talabalar/{id}	o'chirish

Bir xil manzil (/talabalar/{id}) turli metod bilan turli amal bajaradi.

HTTPException — ma'lumot topilmasa, mijozga to'g'ri xato (404 "topilmadi") qaytarish uchun. `model_dump()` — Pydantic obyektni oddiy lug'atga aylantiradi.

Eslatma: bu yerda ma'lumot oddiy lug'atda — dastur o'chsa yo'qoladi. Keyingi modulda ma'lumotni **bazaga** (doimiy) saqlashni ko'rasan. Shuningdek, katta loyihalarda kodni qatlamlarga ajratish, xavfsizlik, autentifikatsiya kabi mavzular bor — ular ilg'orroq, vaqt o'tib o'rganasan.

Masalalar (20 ta)

Bu masalalar uchun `fastapi dev main.py` bilan serverni ishga tushirib, `/docs` orqali sina.

Oson (1–7):

1. / manzilida `{"xabar": "Salom"}` qaytaruvchi API yoz.
2. `/salom/{ism}` yoz: "Salom, {ism}!" qaytarsin.
3. `/kvadrat/{son}` yoz: sonning kvadratini qaytarsin (`son: int`).
4. `/qoshish?a=2&b=3` ko'rinishida ikki sonni qo'shadigan endpoint yoz (query params).
5. `/vaqt` yoz: hozirgi sanani qaytarsin (`datetime` ishlat).
6. `/docs` ni ochib, yozgan endpointlaringni brauzerdan sina.
7. `/salom` endpointiga ixtiyoriy `til` query parametrini qo'sh (standart "uz").

O'rta (8–14):

1. `Mahsulot` Pydantic modeli yoz (`nom: str`, `narx: float`), `POST /mahsulotlar` bilan qabul qilib qaytar.
2. `GET /talabalar` yoz: oldindan to'ldirilgan ro'yxatni (lug'atlar) qaytarsin.
3. `GET /talabalar/{id}` yoz: berilgan id'li talabani qaytarsin, topilmasa 404.
4. Query parametrli qidiruv: `GET /qidiruv?q=...` — mahsulotlardan nomida `q` bo'lganlarni qaytarsin.
5. `POST` bilan kelgan Pydantic modelni tekshir: noto'g'ri ma'lumot yuborib, 422 xatosini ko'r.
6. `limit` va `skip` query parametrlari bilan ro'yxatni "sahifalash" (qism qaytarish).
7. `HTTPException` bilan: id topilmasa 404, manfiy id berilsa 400 qaytar.

Murakkab (15–20):

1. To'liq CRUD yoz: `Kitob` (`nom`, `muallif`) uchun `GET` (hammasi), `GET` (bittasi), `POST`, `DELETE`.
2. `PUT` qo'sh: mavjud kitobni yangilash, topilmasa 404.
3. Avtomatik `id` berish: yangi yozuvga ketma-ket id ber (yuqoridagi `keyingi_id` namunasiday).
4. Validatsiya qo'sh: Pydantic modelda `yosh` manfiy bo'lmasligini ta'minla (`field_validator` yoki shartli tekshiruv bilan).
5. Statistika endpointi: `GET /statistika` — talabalar soni va o'rtacha yoshni qaytarsin.
6. Mini "vazifa boshqaruvchi" (todo) API: vazifa qo'shish, ro'yxatni ko'rish, bajarildi deb belgilash (`PUT`), o'chirish — to'liq CRUD bilan.



Yechimlar



Ko'rsatish uchun ochish



```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from datetime import date

app = FastAPI()

# 1
@app.get("/")
def bosh():
    return {"xabar": "Salom"}

# 2
@app.get("/salom/{ism}")
def salom(ism: str):
    return {"xabar": f"Salom, {ism}!"}

# 3
@app.get("/kvadrat/{son}")
def kvadrat(son: int):
    return {"natija": son ** 2}

# 4
@app.get("/qoshish")
def qoshish(a: int, b: int):
    return {"natija": a + b}

# 5
@app.get("/vaqt")
def vaqt():
    return {"sana": str(date.today())}

# 7
@app.get("/salom2")
def salom2(ism: str = "dunyo", til: str = "uz"):
    matnlar = {"uz": "Salom", "en": "Hello", "ru": "Privet"}
    return {"xabar": f"{matnlar.get(til, 'Salom')}, {ism}!"}

# 8
class Mahsulot(BaseModel):
    nom: str
    narx: float
@app.post("/mahsulotlar")
def mahsulot_qosh(m: Mahsulot):
    return {"qoshildi": m}

# 9, 10, 11, 13 - namuna ma'lumot bilan:
talabalar_db = {
    1: {"ism": "Aziz", "yosh": 20},
    2: {"ism": "Malika", "yosh": 22},
}

@app.get("/talabalar")
def talabalar_royxat(skip: int = 0, limit: int = 10):
    qiymatlar = list(talabalar_db.values())
    return qiymatlar[skip: skip + limit]

@app.get("/talabalar/{id}")
```

```

def talaba_bittasi(id: int):
    if id < 0:
        raise HTTPException(status_code=400, detail="id manfiy bo'lmaydi")
    if id not in talabalar_db:
        raise HTTPException(status_code=404, detail="Topilmadi")
    return talabalar_db[id]

mahsulotlar_db = [{"nom": "Olma", "narx": 12000}, {"nom": "Non", "narx": 4000}]
@app.get("/qidiruv")
def qidiruv(q: str):
    return [m for m in mahsulotlar_db if q.lower() in m["nom"].lower()]

# 15, 16, 17 - Kitob uchun to'liq CRUD:
class Kitob(BaseModel):
    nom: str
    muallif: str

kitoblar: dict[int, dict] = {}
kitob_id = 1

@app.get("/kitoblar")
def kitoblar_royxat():
    return kitoblar

@app.get("/kitoblar/{id}")
def kitob_bittasi(id: int):
    if id not in kitoblar:
        raise HTTPException(status_code=404, detail="Topilmadi")
    return kitoblar[id]

@app.post("/kitoblar")
def kitob_qosh(k: Kitob):
    global kitob_id
    kitoblar[kitob_id] = k.model_dump()
    kitob_id += 1
    return {"id": kitob_id - 1, **k.model_dump()}

@app.put("/kitoblar/{id}")
def kitob_yangila(id: int, k: Kitob):
    if id not in kitoblar:
        raise HTTPException(status_code=404, detail="Topilmadi")
    kitoblar[id] = k.model_dump()
    return kitoblar[id]

@app.delete("/kitoblar/{id}")
def kitob_ochir(id: int):
    if id not in kitoblar:
        raise HTTPException(status_code=404, detail="Topilmadi")
    del kitoblar[id]
    return {"ochirildi": id}

# 18 - validatsiyali model:
from pydantic import field_validator
class TalabaV(BaseModel):
    ism: str
    yosh: int
    @field_validator("yosh")
    @classmethod
    def yosh_musbat(cls, v):
        if v < 0:

```

```

        raise ValueError("yosh manfiy bo'lmaydi")
    return v

# 19
@app.get("/statistika")
def statistika():
    yoshlar = [t["yosh"] for t in talabalar_db.values()]
    return {
        "soni": len(yoshlar),
        "ortacha_yosh": sum(yoshlar) / len(yoshlar) if yoshlar else 0,
    }

# 20 - todo API:
class Vazifa(BaseModel):
    matn: str
    bajarildi: bool = False

vazifalar: dict[int, dict] = {}
vazifa_id = 1

@app.get("/vazifalar")
def vazifalar_royxat():
    return vazifalar

@app.post("/vazifalar")
def vazifa_qosh(v: Vazifa):
    global vazifa_id
    vazifalar[vazifa_id] = v.model_dump()
    vazifa_id += 1
    return {"id": vazifa_id - 1, **v.model_dump()}

@app.put("/vazifalar/{id}/bajarildi")
def vazifa_belgila(id: int):
    if id not in vazifalar:
        raise HTTPException(status_code=404, detail="Topilmadi")
    vazifalar[id]["bajarildi"] = True
    return vazifalar[id]

@app.delete("/vazifalar/{id}")
def vazifa_ochir(id: int):
    if id not in vazifalar:
        raise HTTPException(status_code=404, detail="Topilmadi")
    del vazifalar[id]
    return {"ochirildi": id}

```

15 — Ma'lumotlar bazasi va SQL

Oldingi modulda ma'lumotni oddiy lug'atda saqladik — lekin dastur o'chsa, hammasi yo'qoldi. **Ma'lumotlar bazasi** ma'lumotni doimiy, tartibli va tez qidiriladigan ko'rinishda saqlaydi. Bu yakuniy modulda **SQL** tilining asoslarini va Python'ning o'rnatilgan **sqlite3** modulini o'rganasan — qo'shimcha hech narsa o'rnatmasdan ishlaydigan, to'liq baza.

Bu modulda: ma'lumotlar bazasi nima, **sqlite3** bilan jadval yaratish, ma'lumot qo'shish/o'qish/yangilash/o'chirish (SQL), va xavfsiz so'rovlar (parametrlash).

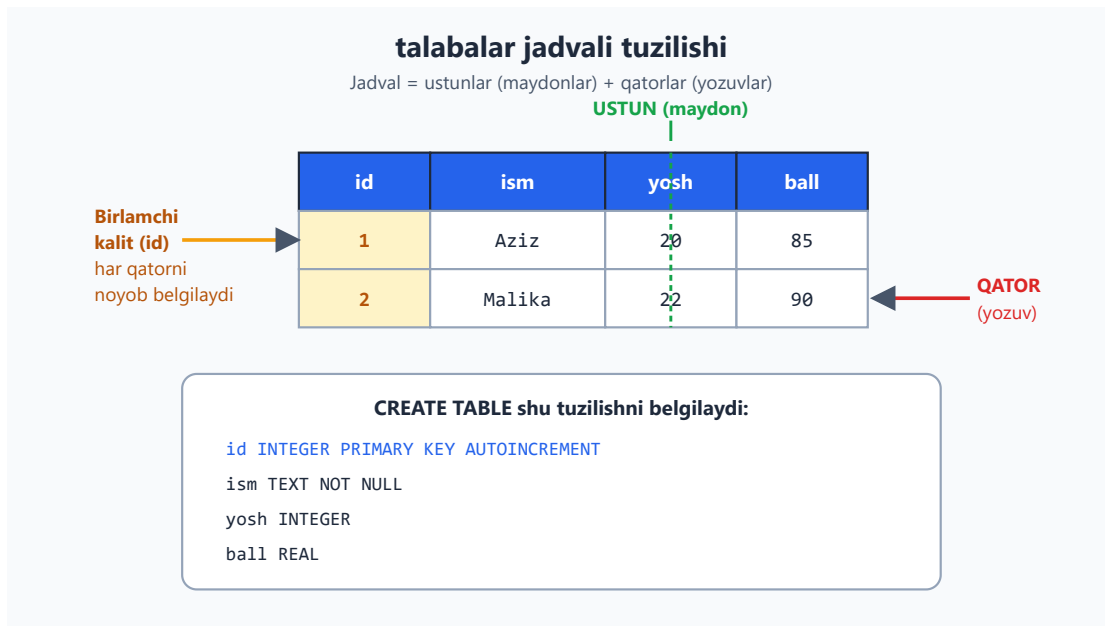
15.1 Ma'lumotlar bazasi nima?

Ma'lumotlar bazasi — ma'lumotni **jadvallar** ko'rinishida saqlaydi (Excel jadvaliga o'xshash, lekin ancha kuchli). Jadval **ustunlar** (maydonlar) va **qatorlardan** (yozuvlar) iborat.

Misol — **talabalar** jadvali:

id	ism	yosh	ball
1	Aziz	20	85
2	Malika	22	90

Jadvalning ustun (maydon), qator (yozuv) va birlamchi kalit (**id**) qismlarini quyidagicha tasavvur qil:



Baza bilan **SQL** (Structured Query Language) tilida "gaplashasan". `sqlite3` Python bilan birga keladi — o'rnatish shart emas, va butun baza bitta faylda saqlanadi. Boshlash uchun ideal.

15.2 Bazaga ulanish va jadval yaratish

```
import sqlite3

# Bazaga ulanish (fayl yo'q bo'lsa, yaratiladi):
conn = sqlite3.connect("maktab.db")
cursor = conn.cursor()

# Jadval yaratish (SQL):
cursor.execute("""
    CREATE TABLE IF NOT EXISTS talabalar (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        ism TEXT NOT NULL,
        yosh INTEGER,
        ball REAL
    )
""")

conn.commit()      # o'zgarishlarni saqlash
conn.close()       # ulanishni yopish
```

Tushuntirish: - `connect("maktab.db")` — baza fayliga ulanadi (yo'q bo'lsa yaratadi). - `cursor` — buyruqlarni bajaruvchi "kursor". - `CREATE TABLE IF NOT EXISTS` — jadval yo'q bo'lsa yaratadi (bor bo'lsa xato bermaydi). - `INTEGER PRIMARY KEY AUTOINCREMENT` — har yozuvga avtomatik o'sib boruvchi `id`. - `TEXT`, `INTEGER`, `REAL` — ustun turlari

(matn, butun son, kasr). - `conn.commit()` — o'zgarishlarni faylga **yozadi** (busiz saqlanmaydi!).

15.3 Ma'lumot qo'shish (INSERT)

```
import sqlite3
conn = sqlite3.connect("maktab.db")
cursor = conn.cursor()

cursor.execute(
    "INSERT INTO talabalar (ism, yosh, ball) VALUES (?, ?, ?)",
    ("Aziz", 20, 85.5)
)
conn.commit()
conn.close()
```

? belgilari — JUDA muhim (xavfsizlik!): qiymatlarni SQL matniga to'g'ridan-to'g'ri yozma, doim `?` ishlatib, qiymatlarni alohida uzat. Aks holda **SQL injection** degan jiddiy xavfsizlik zaifligi paydo bo'ladi. Bu eng muhim qoidalardan biri.

```
# ❌ HECH QACHON BUNDAY QILMA:
cursor.execute(f"INSERT INTO talabalar (ism) VALUES ('{ism}')" )
# ✅ DOIM BUNDAY:
cursor.execute("INSERT INTO talabalar (ism) VALUES (?)", (ism,))
```

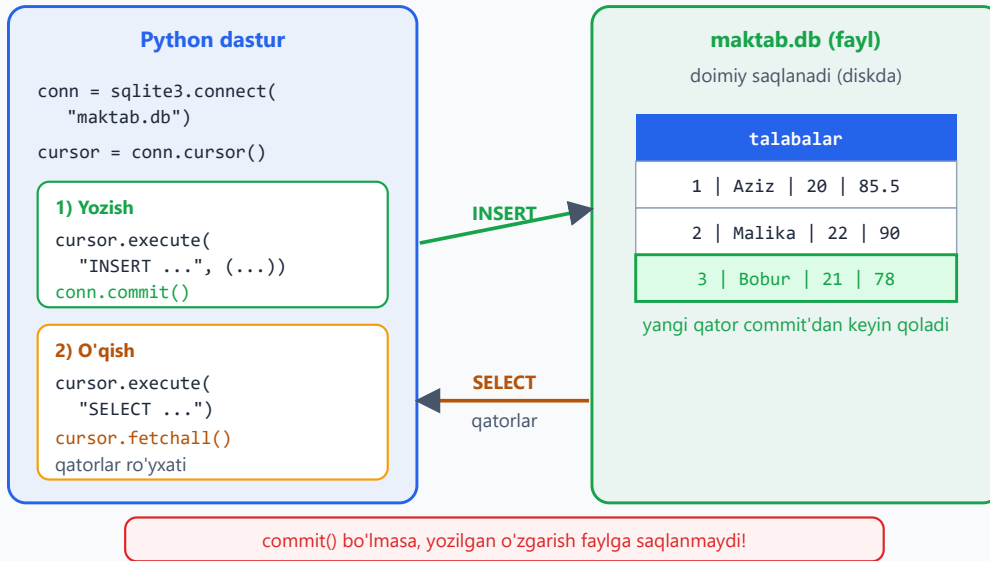
Ko'p yozuvni birdan qo'shish (`executemany`):

```
talabalar = [("Malika", 22, 90), ("Bobur", 21, 78)]
cursor.executemany(
    "INSERT INTO talabalar (ism, yosh, ball) VALUES (?, ?, ?)",
    talabalar
)
conn.commit()
```

Yozish (`INSERT + commit`) va o'qish (`SELECT + fetchall`) dastur bilan baza fayli orasida shunday aylanadi:

Dastur ↔ baza: yozish va o'qish sikli

cursor buyruq yuboradi, commit yozadi, fetchall natijani qaytaradi



15.4 Ma'lumot o'qish (SELECT)

```
import sqlite3
conn = sqlite3.connect("maktab.db")
cursor = conn.cursor()

# Barcha yozuvlar:
cursor.execute("SELECT * FROM talabalar")
qatorlar = cursor.fetchall()      # barcha natijalar ro'yxati
for qator in qatorlar:
    print(qator)                  # (1, 'Aziz', 20, 85.5)

# Bitta yozuv:
cursor.execute("SELECT * FROM talabalar WHERE id = ?", (1,))
print(cursor.fetchone())          # bitta qator yoki None

conn.close()
```

Foydali SQL so'rovlari:

```
# Shart bilan:
cursor.execute("SELECT * FROM talabalar WHERE yosh > ?", (20,))

# Tartiblash:
cursor.execute("SELECT * FROM talabalar ORDER BY ball DESC")

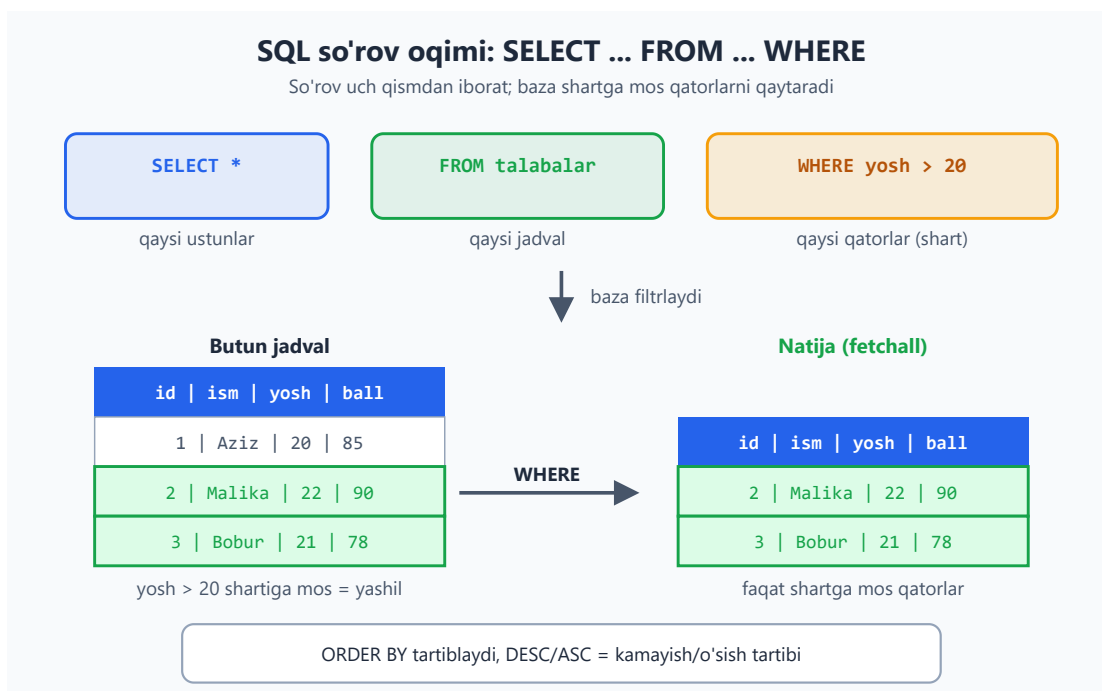
# Faqat kerakli ustunlar:
cursor.execute("SELECT ism, ball FROM talabalar")
```



```
# Sanash:
cursor.execute("SELECT COUNT(*) FROM talabalar")
print(cursor.fetchone()[0])      # nechta talaba bor
```

SQL kalit so'zlari: `SELECT` (tanlash) ... `FROM` (qaysi jadval) ... `WHERE` (shart) ... `ORDER BY` (tartiblash) ... `DESC / ASC` (kamayish/o'sish). Bular barcha SQL bazalarda deyarli bir xil.

So'rovning uch qismi — `SELECT` (qaysi ustunlar), `FROM` (qaysi jadval), `WHERE` (qaysi qatorlar) — qanday ishlashini quyidagi oqim ko'rsatadi:



15.5 Yangilash (UPDATE) va o'chirish (DELETE)

```
# Yangilash:
cursor.execute(
    "UPDATE talabalar SET ball = ? WHERE id = ?",
    (95, 1)
)
conn.commit()

# O'chirish:
cursor.execute("DELETE FROM talabalar WHERE id = ?", (2,))
conn.commit()
```

Diqqat — `WHERE` ni unutma! `UPDATE` yoki `DELETE` da `WHERE` yozmasang, **butun jadvalga** ta'sir qiladi (hamma yozuv o'chadi/o'zgaradi). Bu juda keng tarqalgan,

achchiq xato. Doim `WHERE` borligini tekshir.

15.6 `with` bilan toza ishlash

8-moduldagi `with` bu yerda ham ishlaydi — ulanishni avtomatik boshqaradi:

```
import sqlite3

with sqlite3.connect("maktab.db") as conn:
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM talabalar")
    for qator in cursor.fetchall():
        print(qator)
# with bloki commit'ni avtomatik bajaradi
```

15.7 Keyingi qadam: katta loyihalar uchun

`sqlite3` o'rganish va kichik loyihalar uchun ajoyib. Loyihalarning kattalashganda quyidagilarni o'rganasan (ular ilg'orroq, hozir shart emas): - **PostgreSQL / MySQL** — ko'p foydalanuvchili, katta loyihalar uchun kuchli bazalar (alohida server sifatida ishlaydi). Python'da ularga ulanish uchun maxsus kutubxonalar bor. - **ORM** (masalan SQLAlchemy) — SQL yozmasdan, baza bilan Python obyektlari orqali ishlash usuli. Klasslar jadvalga, obyektlar qatorlarga mos keladi.

Bularning hammasi shu modulda o'rgangan asoslarga (jadval, qator, SELECT/INSERT/UPDATE/DELETE) tayanadi — shuning uchun bu poydevor muhim.



Masalalar (20 ta)

Bu masalalar `sqlite3` bilan ishlaydi — har birini ishga tushirib, yaratilgan `.db` faylni sina.

Oson (1–7):

1. `test.db` bazasiga ulanib, `talabalar` jadvalini yarat (`id`, `ism`, `yosh`).
2. Jadvalga bitta talaba qo'sh (`INSERT` + ? parametrlar bilan).
3. `SELECT * FROM talabalar` bilan barcha yozuvlarni o'qib chiqar.
4. `executemany` bilan 3 ta talabani birdan qo'sh.

5. `WHERE` bilan id'si 1 bo'lgan talabani top.
6. `COUNT(*)` bilan jadvaldagi yozuvlar sonini chiqar.
7. `ORDER BY` bilan talabalarni yoshi bo'yicha tartiblab chiqar.

O'rta (8–14):

1. `UPDATE` bilan bitta talabaning yoshini o'zgartir (`WHERE` bilan).
2. `DELETE` bilan bitta talabani o'chir (`WHERE` bilan).
3. `WHERE yosh > ?` bilan yoshi 20 dan katta talabalarni top.
4. Foydalanuvchidan ism va yosh so'rab (`input`), bazaga qo'sh (`?` parametrlar bilan).
5. `SELECT ism, yosh` bilan faqat kerakli ustunlarni o'qi.
6. `mahsulotlar` jadvali yarat (`nom` , `narx`), bir nechta mahsulot qo'sh, narxi bo'yicha kamayish tartibida chiqar.
7. `with sqlite3.connect(...)` ishlatib, jadvalni o'qib chiqar.

Murakkab (15–20):

1. To'liq CRUD funksiyalari yoz: `qosh(ism, yosh)` , `royxat()` , `yangila(id, yosh)` , `ochir(id)` — har biri bazaga ulanib ishlasin.
2. Oddiy "telefon kitobchasi" dasturi: menyu (qo'shish/ko'rish/o'chirish) bilan, ma'lumot `sqlite3` bazada saqlansin.
3. `WHERE ism LIKE ?` bilan qidiruv: ismida berilgan harf(lar) bor talabalarni top (`LIKE '%a%'`).
4. Ballar jadvali yaratib, `SELECT AVG(ball)` , `MAX(ball)` , `MIN(ball)` bilan statistika chiqar.
5. Oddiy "vazifalar" (todo) dasturi: vazifa qo'shish, bajarildi deb belgilash (`UPDATE`), o'chirish — bazada saqlansin.
6. Mini inventarizatsiya: `mahsulotlar` jadvali (`nom` , `soni` , `narx`); qo'shish, sotish (sonni kamaytirish, `UPDATE`), umumiy qiymatni hisoblash (`SELECT SUM(soni * narx)`) funksiyalari bilan to'liq dastur yoz.

 Yechimlar



```
import sqlite3

# 1
conn = sqlite3.connect("test.db")
cur = conn.cursor()
cur.execute("""CREATE TABLE IF NOT EXISTS talabalar (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    ism TEXT NOT NULL,
    yosh INTEGER
)""")
conn.commit()

# 2
cur.execute("INSERT INTO talabalar (ism, yosh) VALUES (?, ?)", ("Aziz", 20))
conn.commit()

# 3
cur.execute("SELECT * FROM talabalar")
for q in cur.fetchall():
    print(q)

# 4
cur.executemany(
    "INSERT INTO talabalar (ism, yosh) VALUES (?, ?)",
    [("Malika", 22), ("Bobur", 21), ("Gul", 19)]
)
conn.commit()

# 5
cur.execute("SELECT * FROM talabalar WHERE id = ?", (1,))
print(cur.fetchone())

# 6
cur.execute("SELECT COUNT(*) FROM talabalar")
print("Jami:", cur.fetchone()[0])

# 7
cur.execute("SELECT * FROM talabalar ORDER BY yosh")
for q in cur.fetchall():
    print(q)

# 8
cur.execute("UPDATE talabalar SET yosh = ? WHERE id = ?", (25, 1))
conn.commit()

# 9
cur.execute("DELETE FROM talabalar WHERE id = ?", (4,))
conn.commit()

# 10
cur.execute("SELECT * FROM talabalar WHERE yosh > ?", (20,))
print(cur.fetchall())

# 11
ism = input("Ism: ")
yosh = int(input("Yosh: "))
cur.execute("INSERT INTO talabalar (ism, yosh) VALUES (?, ?)", (ism, yosh))
```

```

conn.commit()

# 12
cur.execute("SELECT ism, yosh FROM talabalar")
for q in cur.fetchall():
    print(q)

# 13
cur.execute("""CREATE TABLE IF NOT EXISTS mahsulotlar (
    id INTEGER PRIMARY KEY AUTOINCREMENT, nom TEXT, narx REAL
)""")
cur.executemany("INSERT INTO mahsulotlar (nom, narx) VALUES (?, ?)",
                [("Olma", 12000), ("Non", 4000), ("Go'sht", 90000)])
conn.commit()
cur.execute("SELECT * FROM mahsulotlar ORDER BY narx DESC")
print(cur.fetchall())
conn.close()

# 14
with sqlite3.connect("test.db") as conn:
    cur = conn.cursor()
    cur.execute("SELECT * FROM talabalar")
    for q in cur.fetchall():
        print(q)

# 15
def ulanish():
    return sqlite3.connect("test.db")
def qosh(ism, yosh):
    with ulanish() as c:
        c.execute("INSERT INTO talabalar (ism, yosh) VALUES (?, ?)", (ism,
yosh))
def royxat():
    with ulanish() as c:
        return c.execute("SELECT * FROM talabalar").fetchall()
def yangila(id, yosh):
    with ulanish() as c:
        c.execute("UPDATE talabalar SET yosh = ? WHERE id = ?", (yosh, id))
def ochir(id):
    with ulanish() as c:
        c.execute("DELETE FROM talabalar WHERE id = ?", (id,))
qosh("Kamol", 23)
print(royxat())

# 16
def init():
    with sqlite3.connect("tel.db") as c:
        c.execute("""CREATE TABLE IF NOT EXISTS kontaktlar (
            id INTEGER PRIMARY KEY AUTOINCREMENT, ism TEXT, raqam TEXT)""")
init()
while True:
    amal = input("qosh / korish / ochirish / chiqish: ")
    if amal == "chiqish":
        break
    with sqlite3.connect("tel.db") as c:
        if amal == "qosh":
            ism = input("Ism: "); raqam = input("Raqam: ")
            c.execute("INSERT INTO kontaktlar (ism, raqam) VALUES (?, ?)",
(ism, raqam))
        elif amal == "korish":
            for q in c.execute("SELECT * FROM kontaktlar").fetchall():

```

```

        print(q)
    elif amal == "ochirish":
        id = int(input("id: "))
        c.execute("DELETE FROM kontaktlar WHERE id = ?", (id,))

# 17
with sqlite3.connect("test.db") as conn:
    cur = conn.cursor()
    cur.execute("SELECT * FROM talabalar WHERE ism LIKE ?", ("%a%",))
    print(cur.fetchall())

# 18
with sqlite3.connect("ball.db") as conn:
    cur = conn.cursor()
    cur.execute("CREATE TABLE IF NOT EXISTS b (id INTEGER PRIMARY KEY, ball
REAL)")
    cur.executemany("INSERT INTO b (ball) VALUES (?)", [(85,), (90,), (78,),
(92,)])
    cur.execute("SELECT AVG(ball), MAX(ball), MIN(ball) FROM b")
    print(cur.fetchone())      # (86.25, 92.0, 78.0)

# 19
def todo_init():
    with sqlite3.connect("todo.db") as c:
        c.execute("""CREATE TABLE IF NOT EXISTS vazifalar (
            id INTEGER PRIMARY KEY AUTOINCREMENT, matn TEXT, bajarildi
INTEGER DEFAULT 0)""")
    todo_init()
def vazifa_qosh(matn):
    with sqlite3.connect("todo.db") as c:
        c.execute("INSERT INTO vazifalar (matn) VALUES (?)", (matn,))
def vazifa_belgila(id):
    with sqlite3.connect("todo.db") as c:
        c.execute("UPDATE vazifalar SET bajarildi = 1 WHERE id = ?", (id,))
def vazifa_ochir(id):
    with sqlite3.connect("todo.db") as c:
        c.execute("DELETE FROM vazifalar WHERE id = ?", (id,))
vazifa_qosh("Python o'rganish")
vazifa_belgila(1)

# 20
def inv_init():
    with sqlite3.connect("inv.db") as c:
        c.execute("""CREATE TABLE IF NOT EXISTS mahsulotlar (
            id INTEGER PRIMARY KEY AUTOINCREMENT, nom TEXT, soni INTEGER,
narx REAL)""")
    inv_init()
def mahsulot_qosh(nom, soni, narx):
    with sqlite3.connect("inv.db") as c:
        c.execute("INSERT INTO mahsulotlar (nom, soni, narx) VALUES (?, ?,
?)", (nom, soni, narx))
def sotish(id, miqdor):
    with sqlite3.connect("inv.db") as c:
        c.execute("UPDATE mahsulotlar SET soni = soni - ? WHERE id = ?",
(miqdor, id))
def umumiy_qiyamat():
    with sqlite3.connect("inv.db") as c:
        return c.execute("SELECT SUM(soni * narx) FROM
mahsulotlar").fetchone()[0]
mahsulot_qosh("Olma", 100, 12000)
mahsulot_qosh("Non", 50, 4000)

```

```
sotish(1, 10)
print("Umumiy qiymat:", umumiy_qiymat())
```

🎉 Tabriklayman — kursni tugatding!

Bu — boshlovchilar yo'nalishining **oxirgi moduli**. Noldan boshlab, sen quyidagilarni o'rganding:

- **Asoslar:** o'zgaruvchilar, tiplar, shartlar, sikllar, funksiyalar (01–02)
- **Ma'lumot:** ro'yxat, lug'at, to'plam, stringlar (03–04)
- **OOP:** klasslar, obyektlar, meros (05)
- **Tashkil etish:** modullar, virtual muhit, xatolarni boshqarish (06–07)
- **Ma'lumot bilan ishlash:** fayllar, JSON, CSV (08)
- **Ilg'or vositalar:** generatorlar, dekoratorlar, tip ko'rsatmalari, standart kutubxona (09–11)
- **Professional ko'nikmalar:** parallel ishlash, test yozish, veb API, ma'lumotlar bazasi (12–15)

Endi nima qilish kerak? 1. **Loyiha yoz.** Eng yaxshi o'rganish — amaliyot. Kichik bir narsa qil: CLI dastur, telegram bot, yoki kichik veb API (14 + 15 modullarni birlashtirib). 2.

Rasmiy hujjatni o'qi. docs.python.org — eng ishonchli manba. Endi uni o'qiy oladigan darajadasan. 3. **Mashq qil.** Har modulning 20 ta masalasini yechib chiqqaningga ishonch hosil qil.

Omad! 🚀

← [FastAPI veb API](#) | [Boshlovchilar README](#) ↑